

Frequently Asked Questions About iDRAC Redfish

This technical whitepaper provides an overview of Dell implementation of the iDRAC Redfish Application Programming Interface (API). It describes some best practices and addresses Frequently Asked Questions (FAQs) about Redfish.

December 2024

Revisions

Date/Revision	Description
Oct 2022/ 1.0	Initial release
Oct 2022/ 1.1	- Changed font type for Redfish examples. - Changed body format to JSON for action examples. - Added version to title page.
Oct 2022/ 1.2	- Added links for Powershell gallery and Redfish user guide. - Added section describing how to figure out supported parameters and values for an action.
Nov 2022/1.3	Added 23.8 supported parameters for creating RAID volume.
Nov 2022/1.4	- Added 16.3 and 16.4, DMTF properties for server location. - Added section 22, iDRAC user SSH key management. - Added 23.9 and 23.10 FAQ question for checking server status.
Dec 2022/1.5	- Added 4.8 for setting storage controller rate attributes. - Added 4.9 for changing storage controller mode attribute.
March 2023/1.6	- Added 24.4 FAQ question. - Added 14.3 set BIOS pending attribute values only workflow. - Added 14.6 BIOS reset to defaults workflow.
March 2023/1.7	Added section 23, monitor health state for server hardware.
Nov 2023/1.8	- Updated configuring DMTF Network Device Properties section. - Added Configuring OSM Network Device Properties section. - Added Repurpose Retire Server using SystemErase section. - Added SecureErase individual drive section. - Added Export iDRAC LC Log section. - Added section get/set iDRAC time.
Dec 2024/1.9	- Updated all Redfish OEM URIs to be compliant with DMTF. - Updated iDRAC user account management section, call out differences between iDRAC9 and iDRAC10 for iDRAC10. - Updated Export and Import System Configuration Profile sections, call out URI differences between iDRAC9 and iDRAC10. - Added style tag Note to all Notes mentioned in the doc - Updated URI in section 3.4 to get iDRAC MAC address - Added 4.4 to get storage controller battery details

	• Added new sections 4.4 to 4.7 to get internal and external storage enclosure details • Added new section 9, configure iDRAC User SNMPv3 settings • Updated section 14 to leverage non deprecated property • Added more property details to section 20.1 • Added section 20.4 for preventing iDRAC LC logs overflow • Added note about accepting EULA to SupportAssist section 22.4 • Added section 23 to capture server screenshot details • Added note to section 24.1 about \$expand behavior • Added new section 24.2, get server device sideband firmware update support • Added new section 31 for iDRAC10 AC power recovery server settings • Added new section 32 for getting Dell system ID • Added new section 33 to get server generation details • Added section 34 update Active Directory Kerberos Key Tab • Added new section 35 to help get / debug iDRAC memory / performance issues • Added new section 36, reset iDRAC to default custom settings • Added section 37 to get server GPU details • Added section 38 get operating system network interface details • Added new section 39 set server onetime boot device • Added new section 40 block / disable OS in-band commands to iDRAC • Added new section 41 change iDRAC IPv4 network settings from DHCP to static • Added new section 42, Redfish query parameters • Added 44.12 and 44.13 questions
--	--

Acknowledgments

Author: Richard Schnur, A. Anis Ahmed

Support: Texas Roemer, Ajay Shenoy, Neeraja Kothala

Others: Samiksha Aggarwal, Technical Content Developer

The information in this publication is provided "as is." Dell Inc. makes no representations or warranties of any kind with respect to the information in this publication, and specifically disclaims implied warranties of merchantability or fitness for a particular purpose.

Use, copying, and distribution of any software described in this publication requires an applicable software license.

This document may contain certain words that are not consistent with Dell's current language guidelines. Dell plans to update the document over subsequent future releases to revise these words accordingly.

This document may contain language from third party content that is not under Dell's control and is not consistent with Dell's current guidelines for Dell's own content. When such third party content is updated by the relevant third parties, this document will be revised accordingly.

Copyright © 2024 Dell Inc. or its subsidiaries. All Rights Reserved. Dell Technologies, Dell, and other trademarks are trademarks of Dell Inc. or its subsidiaries. Other trademarks may be trademarks of their respective owners. [12/20/2024] [Technical Whitepaper] [641]

Contents

Revisions.....	2
Acknowledgments.....	3
Contents.....	4
Executive summary.....	12
1 Introduction.....	13
1.1 Getting started with the Redfish API	13
1.2 Redfish key technologies.....	13
1.3 HTTPS communication – Redfish is secure	13
1.4 RESTful application programming interface	14
1.5 JSON data	14
1.6 OData	14
1.7 Eventing.....	14
1.8 Terms and Definitions.....	14
1.9 Run iDRAC Redfish Commands	16
1.10 GET Request Example Using Postman	17
1.11 POST Request Example Using Postman	18
1.12 PATCH Request Example Using Postman	19
1.13 GET Request Example Using CURL.....	20
1.14 POST Request Example Using CURL	21
1.15 PATCH Request Example Using CURL.....	22
1.16 GET Request Example Using Python	22
1.17 POST Request Example Using Python	23
1.18 PATCH Request Example Using Python	23
1.19 Create iDRAC X-auth Token Session	24
1.20 Get Command Example Using X-auth Token Session	25
1.21 Enable/Disable System Lockdown Mode	25
1.22 Enable/Disable iDRAC Redfish Support	26
1.23 Change Redfish URI Computer System ID	26
1.24 Enable/Disable iDRAC USB-NIC.....	27
1.25 Enable/Disable iDRAC IPMI	27
1.26 Set iDRAC NIC Selection/Failover Network	28
2 Viewing iDRAC Information.....	29
3 iDRAC Basics	30
3.1 Get iDRAC Firmware Version.....	30

3.2	Get iDRAC Redfish Version	30
3.3	Get iDRAC Current IPv4 Address	30
3.4	Get iDRAC MAC Address.....	31
3.5	Get Server Model.....	31
3.6	Get Server Service Tag	31
3.7	Get Server OEM Branded Status (ID Module Installed).....	32
4	Configuring Storage Devices using Redfish.....	33
4.1	Get Storage Controllers.....	33
4.2	Get Storage Controller Virtual Disks.....	33
4.3	Get Storage Controller Disks.....	33
4.4	Get Storage Controller Battery Details	34
4.5	Get current enclosure details (internal and external (if supported)).....	34
4.6	Get external enclosure EMM details.....	34
4.7	Get enclosure details (DMTF and OEM)	34
4.8	Get additional OEM enclosure details	34
4.9	Reset Storage Controller	35
4.10	Create Virtual Disk.....	35
4.11	Create Virtual Disk Along with Setting Cache Policy Attributes	36
4.12	Change Virtual Disk Attributes.....	37
4.13	Change Controller Rate Attributes.....	39
4.14	Change Controller Mode Attribute	39
4.15	Secure Virtual Disk	40
4.16	Initialize Virtual Disk	41
4.17	Set Local Key Management (LKM) Encryption	42
4.18	Remove Local Key Management (LKM) Encryption	43
4.19	Get OEM Storage Controller Attributes	44
4.20	Set OEM Storage Controller Attributes	44
5	Configuring DMTF Network Device Properties (Attributes)	46
5.1	Get Network Device IDs	46
5.2	Get Network Device ID Details	46
5.3	Get Network Port Details	47
5.4	Get Network Device Attribute Registry Details.....	47
5.5	Get Network Device Properties	48
5.6	Set Network Device Properties.....	48
6	Configuring OEM Network Device Properties (Attributes)	51
6.1	Get Network Device IDs	51

6.2	Get Network Device ID Details	51
6.3	Get Network Port Details	51
6.4	Get Network Device Attribute Registry Details	52
6.5	Get Network Device Properties	52
6.6	Set Network Device Properties.....	53
7	iDRAC9 User Account Management.....	55
7.1	Create New iDRAC User Account:	55
7.2	Get iDRAC User Account Details:	55
7.3	Change iDRAC User Account Password:	55
7.4	Change iDRAC User Account Privilege:	56
7.5	Delete iDRAC User Account:.....	56
8	iDRAC10 User Account Management.....	58
8.1	Create New iDRAC User Account:	58
8.2	Get iDRAC User Account Details:	58
8.3	Change iDRAC User Account Password:	58
8.4	Change iDRAC User Account Privilege:	59
8.5	Delete iDRAC User Account:.....	59
9	Configure iDRAC User SNMPv3 Settings	60
9.1	Get iDRAC user current SNMPv3 settings	60
9.2	Get iDRAC user SNMPv3 attribute supported possible values	60
9.3	Set iDRAC user SNMPv3 settings.....	61
10	Manage iDRAC Syslog Settings.....	62
10.1	Get iDRAC Syslog Attributes:.....	62
10.2	Get iDRAC Attribute Registry:	62
10.3	Set iDRAC Syslog Attributes:	63
11	Manage iDRAC SNMP Settings	64
11.1	Get iDRAC SNMP Attributes:	64
11.2	Set iDRAC SNMP Attributes:.....	64
11.3	Set iDRAC SNMP Alert for Specific Components:	65
12	Manage iDRAC Power	67
12.1	Get and SET iDRAC Power Cap attributes	67
12.1.1	Get iDRAC Power Cap attributes	67
12.1.2	Set iDRAC Power Cap attributes	67
12.2	iDRAC Power Configuration (Redundancy Policy, HotSpare, Primary PSU)	68
12.2.1	Get iDRAC Power Configuration attributes	68
12.2.2	Set iDRAC Power Configuration attributes	69

13	Get/Set iDRAC NTP settings.....	70
13.1	Get iDRAC NTP settings	70
13.2	Set iDRAC NTP settings.....	70
14	Get/Set Chassis Indicator LED.....	72
14.1	Get Chassis Indicator LED Status.....	72
14.2	Set Chassis Indicator LED.....	72
15	Enabled/Configure iDRAC Alerts	73
15.1	Enable iDRAC Alert Setting	73
15.2	Configure iDRAC Alerts.....	73
16	Redfish Operations Management.....	75
16.1	Manage iDRAC GUI Login Security Message String	75
16.1.1	Get iDRAC Login Security Message String	75
16.1.2	Set iDRAC Login Security Message String.....	75
16.2	Server Processor Information.....	76
16.2.1	Get All Processor Details:	76
16.3	Get Specific Processor Property	76
16.4	Server Memory Information	76
16.4.1	Get All Memory Details.....	76
16.4.2	Get Specific Memory Property	77
17	Redfish BIOS Settings.....	78
17.1	Configure BIOS Settings	78
17.1.1	Get BIOS Attributes.....	78
17.1.2	Get BIOS attribute registry	78
17.2	Set BIOS Attributes (set pending values and create config job one PATCH call)	78
17.3	Set BIOS Attributes (set pending values only)	80
17.4	Configure BIOS Boot Order.....	82
17.4.1	Get Current BIOS Boot Order	82
17.4.2	Set BIOS Boot Order.....	83
17.5	Enabling/Disabling BIOS Boot Order Devices.....	84
17.5.1	Get Current BIOS Boot Order Enabled/Disabled State	84
17.5.2	Disable BIOS Boot Order Device	85
17.6	BIOS Reset to Default Settings (RTD)	86
17.6.1	Get Current BIOS RTD setting:	86
17.6.2	Set BIOS RTD	86
18	Boot images: Physical and Virtual.....	88
18.1	Attach ISO as Virtual CD and Onetime Boot Virtual CD	88

18.2	Attach Multiple Virtual Media Devices	90
18.3	Eject (Detach) Virtual Media Devices	92
18.4	Boot to Network ISO Using OEM Extension	92
18.5	Configure BIOS Attributes and Set Onetime Boot to HTTP Boot Device.....	94
18.5.1	Enable HTTP Device 1 and Set HTTP Device 1 Path BIOS Attributes	95
18.5.2	Get Current Boot Order, Locate HTTP Device 1	96
18.5.3	Set Next Onetime Boot to HTTP Boot Entry Device and Reboot Server.....	97
19	Server Physical Location Attributes.....	98
19.1	Get Server Location Attributes (OEM).....	98
19.2	Set Server Location Attributes (OEM)	98
19.3	Get Server Location Attributes (DMTF).....	99
19.4	Set Server Location Attributes (DMTF)	99
19.5	Get Server Slot Details	99
20	iDRAC Lifecycle Controller	101
20.1	Verify Server Status/Lifecycle Controller Status/Real Time Storage Monitor Status	101
20.2	Insert iDRAC Lifecycle Logs Custom Comment.....	102
20.3	View iDRAC Lifecycle Logs	102
20.4	Prevent iDRAC Lifecycle Logs overflow	103
21	Server POST Code	104
21.1	Get Server Last System Boot Timestamp	105
21.2	Determine Linux OS Run-Level Prior to the Last Reboot	105
21.3	Get iDRAC First Boot Up Time.....	106
21.4	Get/Set iDRAC Date Time Local Offset	106
21.5	Get/Set iDRAC Date Time	107
22	iDRAC Reporting	109
22.1	Job Queue Management	109
22.2	Export Server Hardware Inventory	110
22.3	Export iDRAC Lifecycle Log	110
22.4	SupportAssist Collection.....	111
23	Capture server screenshot details.....	113
23.1	Get last crash screenshot.....	113
23.2	Preview server screenshot	113
23.3	Get current server screenshot.....	113
24	iDRAC Firmware Operations.....	114
24.1	Get Server Current Firmware Inventory All Devices	114
24.2	Get Server Device Sideband Firmware Update Support	114

24.3	Single Device Firmware Update (Immediate).....	115
24.4	Single Device Firmware Update (OnReset)	116
24.5	Rollback Device Firmware	117
24.6	Repository Firmware Updates	119
25	iDRAC Sessions/Webserver Management	126
25.1	Get all active sessions	126
25.2	Delete one active session.....	126
25.3	Get iDRAC webserver session timeout setting	126
25.4	Set iDRAC webserver session timeout setting	127
25.5	iDRAC maximum simultaneous sessions.....	127
25.6	Simultaneous console sessions maximum.....	127
26	iDRAC User SSH Key Management	128
26.1	Get current iDRAC user SSH key strings.....	128
26.2	Set iDRAC user SSH key	130
27	Monitor Health State for Server Hardware (RollupStatus)	131
27.1	Get current rollup status for hardware devices.....	131
27.2	Get additional health information for the devices which support rollup health data	132
28	Repurpose/Retire The Server Using SystemErase.....	134
28.1	Get supported SystemErase Components	134
28.2	Perform SystemErase Action	134
29	SecureErase Individual Drive	136
29.1	Check if Drive Supports ISE	136
29.2	Secure Erase Drive	137
30	Manage iDRAC Time	138
30.1	Get iDRAC Time	138
30.2	Set iDRAC Time	138
31	iDRAC10 AC power recovery server settings	139
31.1	Get AC power recovery settings.....	139
31.2	Set AC power recovery settings	139
32	Dell System ID.....	140
32.1	Get Dell System ID	140
33	Server generation details	141
33.1	Get server generation details	141
34	Update Active Directory Kerberos key tab	142
34.1	Set Kerberos key tab	142
35	Debug iDRAC System Management Memory and processor performance	143

35.1	Get iDRAC kernel memory statistics	143
35.2	Get iDRAC kernel processor statistics	143
36	Reset iDRAC to custom default settings	144
37	Graphics Processing Unit (GPU) details	147
37.1	Get overall GPU details	147
37.2	Get GPU sensors.....	147
37.3	Get GPU power metrics.....	147
37.4	Get GPU thermal metrics	148
38	Get Operating System (OS) network interface details	149
38.1	Get OS network interface details.....	149
39	Set the server onetime boot device.....	150
39.1	Get the supported onetime boot device possible values.....	150
39.2	Get the current onetime boot device setting.....	150
39.3	Set the server onetime boot device to PXE.....	150
39.4	Set the onetime boot device to BIOS setup (F2).....	151
39.5	Set the onetime boot device to Virtual CD (ISO Image).....	151
39.6	Set onetime boot device to Virtual Floppy (IMG Image).....	152
40	Block or Disable OS in-band commands to iDRAC	153
41	Change iDRAC IPv4 network settings from DHCP to Static.....	155
42	Redfish Query Parameters	156
42.1	\$select	156
42.2	\$top	157
42.3	\$expand	158
42.4	\$filter	166
43	Redfish Error Codes.....	172
43.1	HTTP status codes and error messages.....	172
43.2	Message Registry/Error Message Index Error Codes.....	172
43.3	Redfish Service Error payload	173
44	Frequently Asked Questions (FAQ)	174
44.1	Strings the API is expecting for the ComputerSystem Reset action?	174
44.2	Set BIOS attributes but no changes applied, no config job created.....	174
44.3	Created a BIOS config job but no changes were applied.....	175
44.4	BIOS config job created which is still in scheduled state, unable to set more BIOS attributes.....	175
44.5	What version of iDRAC supports configuring OEM attributes using PATCH?	175
44.6	Details on YAML spec/schema?.....	175
44.7	What type of methods are supported per URI?.....	175

44.8 What are the supported parameters/values for an action?	176
44.9 What are the supported parameters for creating a RAID volume using POST?	177
44.10 Operation not starting, Lifecycle Controller in use error message returned	178
44.11 All supported controller attributes are not listed under URI “redfish/v1/Systems/System.Embedded.1/Storage/{controller FQDD}/Controllers/{controller FQDD}?\$select=Oem/Dell/DellStorageController”	178
44.12 redfish/v1/TaskService/Tasks/<task ID> is getting auto deleted after a certain period of time	178
44.13 Configuration or Update job stuck in running state, takes 2 hours to get marked as failed due to timeout ...	178
A Technical support and resources	180

Executive summary

This technical whitepaper provides an overview of the Dell Redfish Application Program Interface (API). This paper describes the best practices and address some of the Frequently Asked Questions (FAQ).

Introduction

The Redfish Scalable Platforms Management API is an Open Standard that is defined by the Distributed Management Task Force (DMTF). Redfish is a next-generation systems management interface standard that enables scalable, secure, and open server management. The Redfish API uses RESTful interface semantics to access data that is defined in model format to perform out-of-band systems management. It is suitable for a wide range of servers ranging from stand-alone servers to rack mount and bladed environments and for large-scale cloud environments.

Dell PowerEdge servers offer a comprehensive range of embedded systems management functions enabled by the Integrated Dell Remote Access Controller (iDRAC). These functions are designed by adhering to industry standard application programming interfaces (APIs) including Redfish. iDRAC technology is part of a larger data center solution that helps keep business critical applications and workloads available always. The technology allows administrators to deploy, monitor, manage, configure, update, troubleshoot, and remediate Dell servers from any location, and without the use of agents. It accomplishes this regardless of an operating system or a Hypervisor presence or state.

This document provides a brief overview on Redfish and information about various aspects of Redfish protocol, supported schema, and Redfish Eventing implemented in iDRAC. Its main purpose is to provide a reference to Frequently Asked Questions and best practices.

The iDRAC Redfish API product team members are interested in your direct feedback. This email address: Redfish.FAQ.Council@dell.com is intended for Dell Technologies' Redfish user community to provide direct feedback and submit new workflow requests.

1.1 Getting started with the Redfish API

Redfish API has been defined and in use since 2015. What follows in the below sections are the key things that you need to know about Redfish to help get started. We break out technologies and terms that help you get ready to capture the power of what is available from the PowerEdge iDRAC Redfish API implementation.

1.2 Redfish key technologies

Redfish is a RESTful interface over HTTPS in JSON format based on OData usable by clients, scripts, and browser-based GUIs. It uses a range of IT technologies that have been selected because of their widespread use. By adopting these accepted technologies, administrators will find it easier to use Redfish. Taken together, these technologies create a foundation from which servers can be managed by using common programming and scripting languages.

1.3 HTTPS communication – Redfish is secure

The Hypertext Transfer Protocol or HTTP is an application protocol for distributed, collaborative, hypermedia information systems and forms the foundation of data communication for the World Wide Web. Secure HTTP or HTTPS is a secure version of HTTP that enables secure communications by operating HTTP within a network connection encrypted by TLS or SSL. By using HTTPS, Redfish significantly enhances the security of server management especially in comparison to legacy server management protocols.

1.4 RESTful application programming interface

REpresentational State Transfer (REST) is a software architectural style. REST has been applied for a range of purposes including the definition of web-based APIs. Systems that adhere to REST practices are often referred to as RESTful interfaces and typically use the HTTP Methods (GET, POST, DELETE, and more) that web browsers use to access web pages. RESTful architectures are commonly used by many IT solutions. Leveraging this standardized approach, Redfish implements a RESTful API for accessing management information and for issuing commands to change the configuration or operational state of a server.

1.5 JSON data

Redfish represents data using JSON, which is JavaScript Object Notation. JSON is a lightweight data-interchange format that is easy for users to read and write, and for machines to parse. JSON uses a text format that is completely language independent but uses conventions familiar to programmers. These properties make JSON an ideal data-interchange language.

1.6 OData

OData is an open protocol standard for the definition and exchange of information using RESTful APIs. OData provides Redfish with the required framework to ensure that the data structures remain interchangeable between server vendors.

1.7 Eventing

The Redfish specification includes support for eventing that enables the notification to a management client of significant events occurring in a server. Redfish provides push style event notifications to an event listener, defined as a Redfish compliant HTTPS server. The listener subscribes to the events of interest based on the event types defined in the Redfish specification. Event subscriptions remain in place until deleted or until the Redfish manager such as iDRAC is reset to its default configuration.

Upon receiving an event subscription request, iDRAC will add the hostname of the requestor to the list of targets to be notified when the event occurs.

1.8 Terms and Definitions

For this technical white paper, the following terms and definitions apply:

AR:	Attribute Registry
BIOS:	Basic Input Output System
BMC:	Baseboard Management Controller
CLI:	Command Line Interface

CMOS:	Complementary Metal–Oxide–Semiconductor
CNAs:	Converged Network Adapters
CPU:	Central Processing Unit
DMA:	Direct Memory Access
DMTF:	Distributed Management Task Force
FCs:	Fiber Channel cards
FQDD:	Fully Qualified Device Descriptor
FRU:	Field Replaceable Unit
GUI:	Graphical User Interface
HTTP:	Hypertext Transfer Protocol
HTTPS:	Hypertext Transfer Protocol over TLS (secure)
iDRAC:	Integrated Dell Remote Access Controller
IP:	Internet Protocol – logical address
IPMI:	Intelligent Platform Management Interface
ISM:	iDRAC Service Module
JSON:	JavaScript Object Notation
LOMs:	LAN on Motherboards
MAC:	Media Access Control – physical address
MRC:	Memory Reference Code
NDCs:	Network Daughter Cards
NICs:	Network Interface Cards
OData:	Open Data Protocol
OEM:	Original Equipment Manufacturer
OS:	Operating System
PCI:	Peripheral Component Interconnect
PCIe:	PCI Express
PXE:	Preboot eXecution Environment
RACADM:	RACADM is the command line interface (CLI) to the iDRAC.

RAID:	Redundant Array of Independent Disks
REST:	REpresentational State Transfer
RPM:	RedHat Packet Manager
SCP:	Server Configuration Profile
SEKM:	Secure Encrypted Key Management
SIO:	Serial Input-Output
SSH:	Secure Shell
SSL:	Secure Socket Layer
TCP:	Transmission Control Protocol
TLS:	Transport Layer Security
UEFI:	Unified Extensible Firmware Interface
URI:	Uniform Resource Identifier
WS-MAN:	Web Services-Management
YAML:	Yet Another Markup Language

1.9 Run iDRAC Redfish Commands

iDRAC Redfish commands can be run from either a remote management station such as a laptop or desktop, or locally from the server operating system using the USB-NIC interface. Redfish commands can be run using several diverse ways such as using a REST client (Example: Postman), curl, Python, or Powershell.

The Redfish commands issued perform a TLS handshake with the iDRAC webserver and the interaction happens through the HTTPS protocol.

If using a browser to run GET requests, you may need to install the JSON plugin to put the output in human readable format. Note that Firefox by default already has this plugin installed. Other browsers like Chrome and Edge you may need to install.

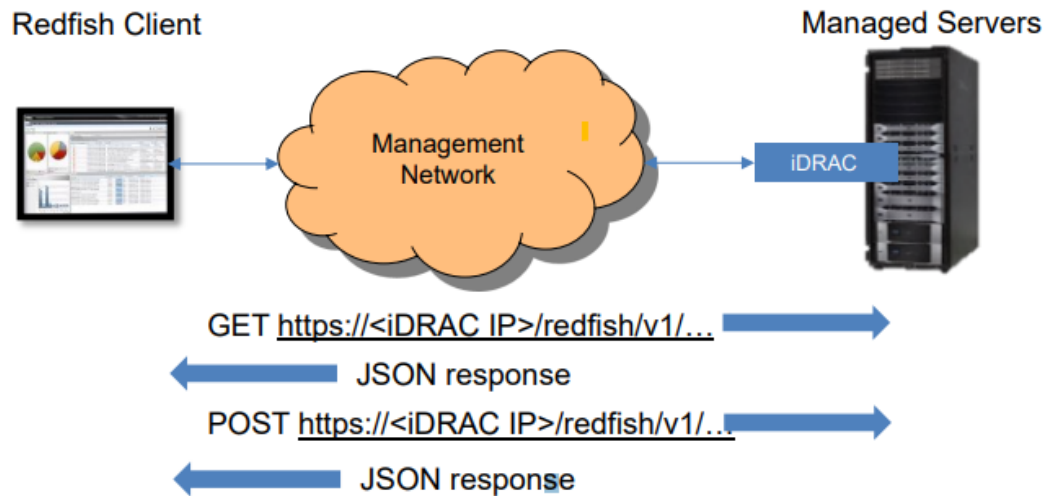


Figure 1 Redfish communication workflow

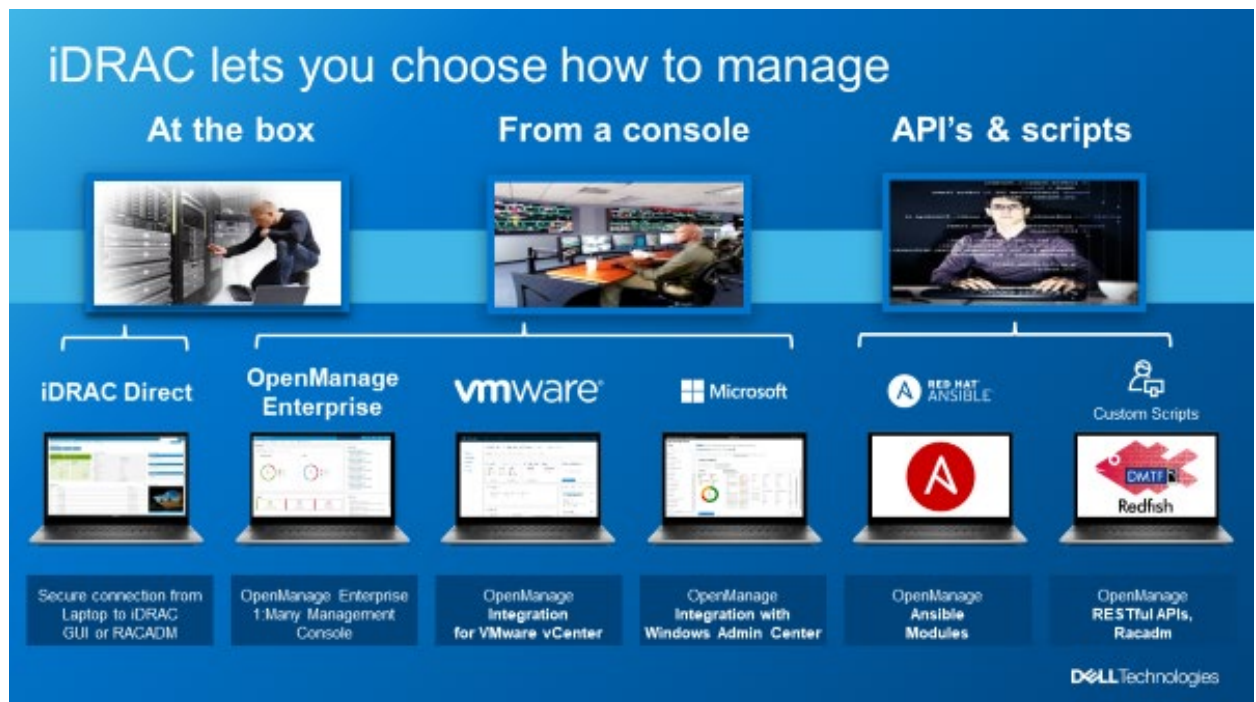


Figure 2 Server management solutions

1.10 GET Request Example Using Postman

Command: GET

URI: `redfish/v1/Systems`

Auth: Basic (passed in iDRAC username and password)

Header: content-type application/json

NOTE: Status code of 200 is returned for success.

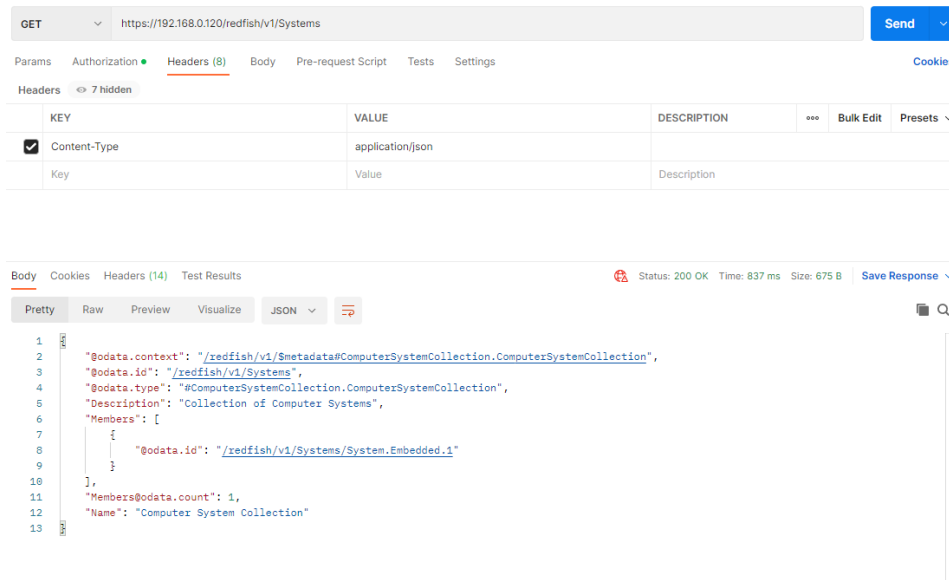


Figure 3 Postman GET example

1.11 POST Request Example Using Postman

Command: POST

URI: redfish/v1/Systems/System.Embedded.1/Actions/ComputerSystem.Reset

Header: content-type application/json

Body: {
 "ResetType": "ForceRestart"
}

NOTE: Status code of 204 is returned for success.

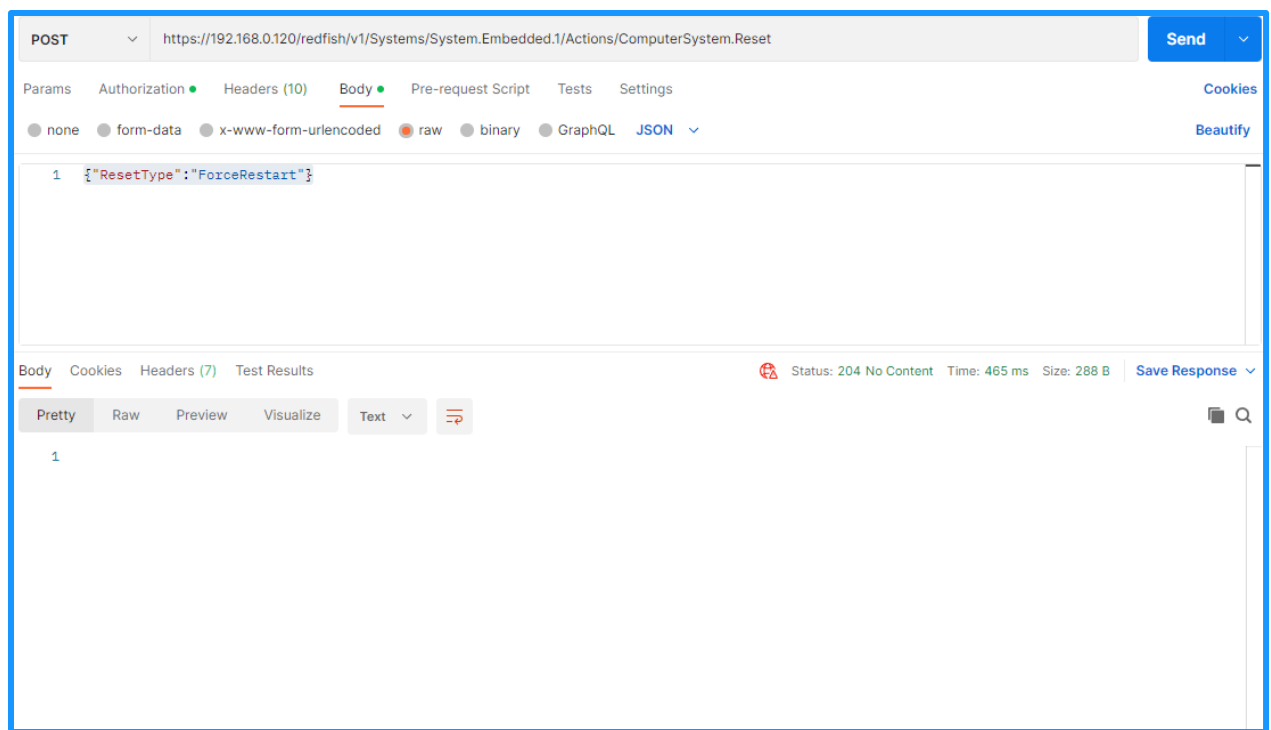


Figure 4 Postman POST example

1.12 PATCH Request Example Using Postman

Command: PATCH

URI:
redfish/v1/Managers/iDRAC.Embedded.1/Oem/Dell/DellAttributes/iDRAC.Embedded.1

Header: content-type application/json

```
Body: {
  "Attributes":{
    "Redfish.1.NumericDynamicSegmentsEnable":"Enabled"
  }
}
```

NOTE: Status code of 200 is returned for success.

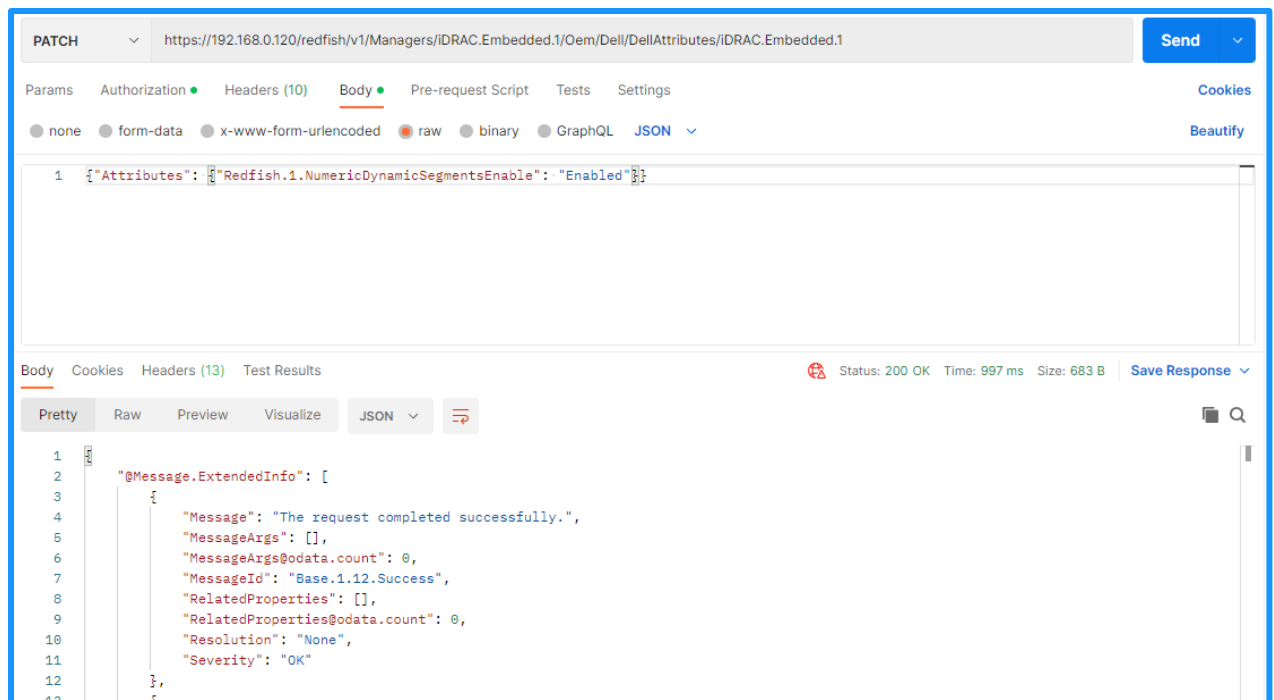


Figure 5 Postman PATCH example

1.13 GET Request Example Using CURL

```
[root@localhost /]# curl -GET "https://192.168.0.120/redfish/v1/Managers/iDRAC.Embedded.1/Oem/Dell/DellAttributes/iDRAC.Embedded.1/Attributes?$select=Attributes/IPv6.1.Enable" -u root:calvin --insecure -H "Content-Type: application/json" -i
```

HTTP/1.1 200 OK

Date: Tue, 11 Oct 2022 21:45:46 GMT

Server: Apache

Allow: PATCH

Link: </redfish/v1/Schemas/DellAttributes.v1_0_0.json>;rel=describedby

OData-Version: 4.0

Access-Control-Allow-Origin: *

Cache-Control: no-cache

X-Frame-Options: DENY

Strict-Transport-Security: max-age=63072000; includeSubDomains; preload

Content-Length: 245

Vary: Accept-Encoding

Content-Type: application/json;odata.metadata=minimal;charset=utf-8

```
{"@odata.context":"/redfish/v1/$metadata#DellAttributes.DellAttributes","@odata.id":"/redfish/v1/Managers/1/Oem/Dell/DellAttributes/iDRAC.Embedded.1","@odata.type":"#DellAttributes.v1_0_0.DellAttributes","Attributes":{"IPv6.1.Enable":"Enabled"}}
```

1.14 POST Request Example Using CURL

```
[root@localhost /]# curl -k "https://192.168.0.120/redfish/v1/Managers/iDRAC.Embedded.1/Oem/Dell/DellJobService/Actions/DellJobService.DeleteJobQueue" -d '{"JobID":"JID_CLEARALL"}' -X POST -u root:calvin -H "Content-Type: application/json" -i
```

HTTP/1.1 200 OK

Date: Tue, 11 Oct 2022 22:15:11 GMT

Server: Apache

OData-Version: 4.0

Access-Control-Allow-Origin: *

Cache-Control: no-cache

X-Frame-Options: DENY

Strict-Transport-Security: max-age=63072000; includeSubDomains; preload

Content-Length: 498

Vary: Accept-Encoding

Content-Type: application/json;odata.metadata=minimal;charset=utf-8

```
{"@Message.ExtendedInfo":[{"Message":"The request completed successfully.","MessageArgs":[],"MessageArgs@odata.count":0,"MessageId":"Base.1.12.Success","RelatedProperties":[],"RelatedProperties@odata.count":0,"Resolution":"None","Severity":"OK"},{"Message":"The specified job was deleted","MessageArgs":[],"MessageArgs@odata.count":0,"MessageId":"iDRAC.2.7.SUP020","RelatedProperties":[],"RelatedProperties@odata.count":0,"Resolution":"No response action is required.","Severity":"Informational"}]}
```

1.15 PATCH Request Example Using CURL

```
[root@localhost /]# curl -k
"https://192.168.0.120/redfish/v1/Managers/iDRAC.Embedded.1/Oem/Dell/DellAttributes/iDRAC.Embedded.1
" -X PATCH -u root:calvin --insecure -H "Content-Type: application/json" -i -d '{"Attributes":{"IPv6.1.
Enable":"Enabled"}}'

HTTP/1.1 200 OK

Date: Tue, 11 Oct 2022 22:19:47 GMT

Server: Apache

OData-Version: 4.0

Access-Control-Allow-Origin: *

Cache-Control: no-cache

X-Frame-Options: DENY

Strict-Transport-Security: max-age=63072000; includeSubDomains; preload

Content-Length: 506

Vary: Accept-Encoding

Content-Type: application/json;odata.metadata=minimal;charset=utf-8

{"@Message.ExtendedInfo":[{"Message":"The request completed
successfully.", "MessageArgs": [], "MessageArgs@odata.count": 0, "MessageId": "Base.1.12.Success", "RelatedP
roperties": [], "RelatedProperties@odata.count": 0, "Resolution": "None", "Severity": "OK"}, {"Message": "The
operation successfully
completed.", "MessageArgs": [], "MessageArgs@odata.count": 0, "MessageId": "iDRAC.2.7.SYS413", "RelatedPro
perties": [], "RelatedProperties@odata.count": 0, "Resolution": "No response action is
required.", "Severity": "Informational"}]}
```

1.16 GET Request Example Using Python

```
>>> import requests

>>> import warnings

>>> warnings.filterwarnings("ignore")

>>> response = requests.get('https://192.168.0.120/redfish/v1/Managers', auth=("root", "calvin"), verify=False)
```

```
>>> response.status_code

200

>>> response.json()

{'@odata.context': '/redfish/v1/$metadata#ManagerCollection.ManagerCollection', '@odata.id':
'/redfish/v1/Managers', '@odata.type': '#ManagerCollection.ManagerCollection', 'Description': 'BMC',
'Members': [{'@odata.id': '/redfish/v1/Managers/1'}], 'Members@odata.count': 1, 'Name': 'Manager'}
```

1.17 POST Request Example Using Python

```
>>> import requests

>>> import json

>>> import warnings

>>> warnings.filterwarnings("ignore")

>>> url = 'https://192.168.0.120/
redfish/v1/Managers/iDRAC.Embedded.1/Oem/Dell/DellLCService/Actions/DellLCService.GetRemoteService
sAPIStatus'

>>> body = {}

>>> headers = {'content-type': 'application/json'}

response = requests.post(url, data=json.dumps(body), headers=headers, verify=False, auth=("root", "calvin"))

>>> response.status_code

200

>>> response.json()

{'@Message.ExtendedInfo': [{'Message': 'The request completed successfully.', 'MessageArgs': [],
'MessageArgs@odata.count': 0, 'MessageId': 'Base.1.12.Success', 'RelatedProperties': [],
'RelatedProperties@odata.count': 0, 'Resolution': 'None', 'Severity': 'OK'}, {'Message': 'Lifecycle controller is
ready to take provisioning requests', 'MessageArgs': [], 'MessageArgs@odata.count': 0, 'MessageId':
'iDRAC.2.7.LC061', 'RelatedProperties': [], 'RelatedProperties@odata.count': 0, 'Resolution': 'No response
action is required.', 'Severity': 'Informational'}], 'LCStatus': 'Ready', 'RTStatus': 'Ready', 'ServerStatus':
'OutOfPOST', 'Status': 'Ready', 'TelemetryServiceStatus': 'Ready'}
```

1.18 PATCH Request Example Using Python

```
>>> import requests
```

```

>>> import json

>>> import warnings

>>> warnings.filterwarnings("ignore")

>>> url =
https://192.168.0.120/redfish/v1/Managers/iDRAC.Embedded.1/Oem/Dell/DellAttributes/iDRAC.Embedded.1

>>> headers = {'content-type': 'application/json'}

>>> body = {"Attributes":{"IPv6.1.Enable":"Enabled"}}

>>> response = requests.patch(url, data=json.dumps(body), headers=headers,
verify=False,auth=("root","calvin"))

>>> response.status_code

200

>>> response.json()

{'@Message.ExtendedInfo': [{'Message': 'The request completed successfully.', 'MessageArgs': [],
'MessageArgs@odata.count': 0, 'MessageId': 'Base.1.12.Success', 'RelatedProperties': [],
'RelatedProperties@odata.count': 0, 'Resolution': 'None', 'Severity': 'OK'}, {'Message': 'The operation
successfully completed.', 'MessageArgs': [], 'MessageArgs@odata.count': 0, 'MessageId':
'iDRAC.2.7.SYS413', 'RelatedProperties': [], 'RelatedProperties@odata.count': 0, 'Resolution': 'No response
action is required.', 'Severity': 'Informational'}]}

```

1.19 Create iDRAC X-auth Token Session

To perform Redfish commands, you can use either basic auth or X-auth token session. It is recommended to use X-auth token for better security, simplifies running Redfish commands. Now when running Redfish commands, you only need to pass in iDRAC IP with X-auth token session, username and password is no longer required.

Command: POST

URI: redfish/v1/SessionService/Sessions

Header: content-type application/json

Auth: Basic or X auth

Body:

```
{"UserName": "<iDRAC username>", "Password": "<iDRAC user password>"}
```

Success status code returned: 201

NOTE: In the headers, output the X-auth token session will be returned.

NOTE: Once the X-auth token is generated, there is no way to retrieve it after creation.

1.20 Get Command Example Using X-auth Token Session

Command: GET

URI: redfish/v1/Managers/iDRAC.Embedded.1

Header: content-type application/json, X-auth-token "<your X-auth token session>"

Success status code returned: 200

NOTE: For more details, examples of running GET/PATCH/POST/DELETE commands using X-auth token session, review scripts on GitHub.

<https://github.com/dell/iDRAC-Redfish-Scripting/tree/master/Redfish%20Python>

1.21 Enable/Disable System Lockdown Mode

Command: PATCH

URI:
redfish/v1/Managers/iDRAC.Embedded.1/Oem/Dell/DellAttributes/iDRAC.Embedded.1

Header: content-type application/json

Auth: Basic or X auth

Body: {
 "Attributes": {
 "Lockdown.1.SystemLockdown": "<pass in Enabled or Disabled>"
 }
}

Success status code returned: 200

NOTE: Default iDRAC setting is Disabled. If System Lockdown Mode is enabled, you cannot make any configuration changes or apply firmware updates to the server.

1.22 Enable/Disable iDRAC Redfish Support

Command: PATCH

URI:
redfish/v1/Managers/iDRAC.Embedded.1/Oem/Dell/DellAttributes/iDRAC.Embedded.1

Header: content-type application/json

Auth: Basic or X auth

```
Body: {
  "Attributes":{
    "Redfish.1.Enable":"<pass in Enabled or Disabled>"
  }
}
```

Success status code returned: 200

NOTE: Default iDRAC9 setting is Enabled, iDRAC10 this attribute has been removed, no option to disable Redfish.

1.23 Change Redfish URI Computer System ID

Command: PATCH

URI:
redfish/v1/Managers/iDRAC.Embedded.1/Oem/Dell/DellAttributes/iDRAC.Embedded.1

Header: content-type application/json

Auth: Basic or X auth

```
Body: {
  "Attributes":{
    "Redfish.1.NumericDynamicSegmentsEnable":"Enabled"
  }
}
```

Success status code returned: 200

Default behavior for iDRAC, Redfish URI Computer System ID will be:

redfish/v1/Systems/System.Embedded.1

redfish/v1/Managers/iDRAC.Embedded.1

redfish/v1/Chassis/System.Embedded.1

By setting this attribute to Enabled, the computer system ID will be updated to 1.

Example:

redfish/v1/Systems/1

redfish/v1/Managers/1

redfish/v1/Chassis/1

NOTE: Default iDRAC setting is Disabled.

NOTE: When this attribute is set to enabled, the default URI computer system IDs will still be supported.

1.24 Enable/Disable iDRAC USB-NIC

Command: PATCH

URI:

redfish/v1/Managers/iDRAC.Embedded.1/Oem/Dell/DellAttributes/iDRAC.Embedded.1

Header: content-type application/json

Auth: Basic or X auth

```
Body: {  
  "Attributes":{  
    "OS-BMC.1.AdminState":"<pass in Enabled or Disabled>"  
  }  
}
```

Success status code returned: 200

1.25 Enable/Disable iDRAC IPMI

Command: PATCH

URI:

redfish/v1/Managers/iDRAC.Embedded.1/Oem/Dell/DellAttributes/iDRAC.Embedded.1

Header: content-type application/json

Auth: Basic or X auth

Body: {

"Attributes":{

"IPMILan.1#Enable":"<pass in Enabled or Disabled>"

}

}

Success status code returned: 200

1.26 Set iDRAC NIC Selection/Failover Network

Command: PATCH

URI:

redfish/v1/Managers/iDRAC.Embedded.1/Oem/Dell/DellAttributes/iDRAC.Embedded.1

Header: content-type application/json

Auth: Basic or X auth

Body: {

"Attributes":{

"NIC.1.Selection":"LOM1",

"NIC.1.Failover":"ALL"

}

}

Success status code returned: 200

NOTE: In this example I am setting NIC section to LOM 1 port, failover to all ports.

2 Viewing iDRAC Information

iDRAC enables you to configure iDRAC properties, set up users, and alerts to perform remote management tasks. Before you configure iDRAC, ensure that the iDRAC network settings and a supported browser are configured, and the required licenses are updated.

In section 1, we covered the basics on high-level theory of operations. Starting with this section, we begin to explore the fun stuff. Examples of what you can do with Redfish and hints for you to follow to get familiar with the inner workings of the Redfish API. We will explore commands and responses. You will find a wealth of information about what you can do to swim with the Redfish.

The next few sections uncover iDRAC Redfish commands. The following hot links will start you off to your swim to configuring various Redfish capabilities:

- [iDRAC](#)
- [Storage](#)
- [Networking](#)
- [User Accounts](#)
- [Syslog](#)
- [Power](#)
- [Redfish Operations Management](#)
- [System Information](#)
- [BIOS Operations](#)
- [Boot images physical and virtual](#)
- [Server Physical Location](#)
- [iDRAC Lifecycle Controller](#)
- [Server POST Code](#)
- [iDRAC Reporting](#)
- [Webserver management](#)
- [Redfish Error Codes](#)
- [Redfish FAQ](#)
- [Technical Support and Redfish resources](#)

3 iDRAC Basics

3.1 Get iDRAC Firmware Version

Command: GET

URI: redfish/v1/Managers/iDRAC.Embedded.1?\$select=FirmwareVersion

Header: content-type application/json

Auth: Basic or X auth

Success status code returned: 200

3.2 Get iDRAC Redfish Version

Command: GET

URI: redfish/v1?\$select=RedfishVersion

Header: content-type application/json

Auth: Basic or X auth

Success status code returned: 200

NOTE: The above returns the current Redfish version for the iDRAC firmware version that is installed on the server.

3.3 Get iDRAC Current IPv4 Address

Command: GET

URI:
redfish/v1/Managers/iDRAC.Embedded.1/Oem/Dell/DellAttributes/iDRAC.Embedded.1?\$select=Attributes/CurrentIPv4.1.Address

Header: content-type application/json

Auth: Basic or X auth

Success status code returned: 200

3.4 Get iDRAC MAC Address

Command: GET

URI: redfish/v1/Managers/iDRAC.Embedded.1/EthernetInterfaces/<interface instance ID>?\$select=MACAddress

URI example:

redfish/v1/Managers/iDRAC.Embedded.1/EthernetInterfaces/NIC.1?\$select=MACAddress

Header: content-type application/json

Auth: Basic or X auth

Success status code returned: 200

3.5 Get Server Model

Command: GET

URI: redfish/v1/Systems/System.Embedded.1?\$select=Model

Header: content-type application/json

Auth: Basic or X auth

Success status code returned: 200

3.6 Get Server Service Tag

Command: GET

URI:

redfish/v1/Systems/System.Embedded.1/Bios?\$select=Attributes/SystemServiceTag

Header: content-type application/json

Auth: Basic or X auth

Success status code returned: 200

3.7 Get Server OEM Branded Status (ID Module Installed)

Command: GET

URI:

redfish/v1/Systems/System.Embedded.1?\$select=Oem/Dell/DellSystem/IsOEMBranded

Header: content-type application/json

Auth: Basic or X auth

Success status code returned: 200

NOTE: Boolean value of True means ID Module is installed.

4 Configuring Storage Devices using Redfish

4.1 Get Storage Controllers

Command: GET

URI: redfish/v1/Systems/System.Embedded.1/Storage?\$expand=*\$levels=1)

Header: content-type application/json

Auth: Basic or X auth

Success status code returned: 200

4.2 Get Storage Controller Virtual Disks

Command: GET

URI: redfish/v1/Systems/System.Embedded.1/Storage/{controller ID}/Volumes?\$expand=*\$levels=1)

URI Example: redfish/v1/Systems/System.Embedded.1/Storage/RAID.SL.3-1/Volumes?\$expand=*\$levels=1)

Header: content-type application/json

Auth: Basic or X auth

Success status code returned: 200

4.3 Get Storage Controller Disks

Command: GET

URI: redfish/v1/Systems/System.Embedded.1/Storage/{controller ID}?\$select=Drives

URI Example: redfish/v1/Systems/System.Embedded.1/Storage/RAID.SL.3-1?\$select=Drives

Header: content-type application/json

Auth: Basic or X auth

Success status code returned: 200

NOTE: JSON output will return all disk URIs. iDRAC does not support passing in multiple query parameters so to get details for each disk, run GET on the disk URI.

4.4 Get Storage Controller Battery Details

Command: GET
URI: redfish/v1/Chassis/System.Embedded.1/Oem/Dell/DellControllerBattery
Header: content-type application/json
Auth: Basic or X auth
Success status code returned: 200

4.5 Get current enclosure details (internal and external (if supported))

Command: GET
URI: redfish/v1/Chassis/System.Embedded.1/Oem/Dell/DellEnclosures
Header: content-type application/json
Auth: Basic or X auth

4.6 Get external enclosure EMM details

Command: GET
URI: redfish/v1/Chassis/System.Embedded.1/Oem/Dell/DellEnclosureEMM
Header: content-type application/json
Auth: Basic or X auth

4.7 Get enclosure details (DMTF and OEM)

Command: GET
URI: redfish/v1/Chassis/<enclosure ID>
URI example: redfish/v1/Chassis/Enclosure.Internal.0-1:NonRAID.SL.3-1
Header: content-type application/json
Auth: Basic or X auth

4.8 Get additional OEM enclosure details

Command: GET
URI: redfish/v1/Chassis/<enclosure ID>/Oem/Dell/<enclosure ID>
URI example: redfish/v1/Chassis/Enclosure.Internal.0-1:NonRAID.SL.3-1/Oem/Dell/DellEnclosures/Enclosure.Internal.0-1:NonRAID.SL.3-1
Header: content-type application/json
Auth: Basic or X auth

4.9 Reset Storage Controller

Command: POST

URI: redfish/v1/redfish/v1/Systems/System.Embedded.1/Oem/Dell/DellRaidService/Actions/DellRaidService.ResetConfig

Header: content-type application/json

Auth: Basic or X auth

Body: {
 "TargetFQDD": "<controller FQDD>"
}

Body Example: {
 "TargetFQDD": "RAID.Integrated.1-1"
}

Success status code returned: 202

JOB ID URI will be returned in JSON response headers location. Run GET command on this URI to monitor the job status to completion.

Command: GET

URI: redfish/v1/Managers/iDRAC.Embedded.1/Oem/Dell/Jobs/{job ID}

Header: content-type application/json

Auth: Basic or X auth

Success status code returned: 200

4.10 Create Virtual Disk

Command: POST
URI: redfish/v1/Systems/System.Embedded.1/Storage/{controller FQDD ID}/Volumes

Header: content-type application/json

Auth: Basic or X auth

Body: {"RAIDType": "<RAID level>", "Links": {"Drives": [{"@odata.id": "<disk URI>"}]}}

Body example to create a one disk RAID 0:

```
{"RAIDType": "RAID0", "Links": {"Drives": [{"@odata.id":
"/redfish/v1/Systems/System.Embedded.1/Storage/Drives/Disk.Bay.3:Enclosure.Internal.0-1:RAID.SL.3-1"}]}}
```

Success status code returned: 202

JOB ID URI will be returned in JSON response headers location. Run GET command on this URI to monitor the job status to completion.

Command: GET

URI: redfish/v1/Managers/iDRAC.Embedded.1/Oem/Dell/Jobs/{job ID}

Header: content-type application/json

Auth: Basic or X auth

Success status code returned: 200

4.11 Create Virtual Disk Along with Setting Cache Policy Attributes

Command: POST

URI: redfish/v1/Systems/System.Embedded.1/Storage/{controller ID}/Volumes

URI Example: redfish/v1/Systems/System.Embedded.1/Storage/RAID.SL.3-1/Volumes

Header: content-type application/json

Auth: Basic or X auth

Body: {"RAIDType": "<RAID level>", "Links": {"Drives": [{"@odata.id": "<drive URI>"}]}, "ReadCachePolicy": "Off", "WriteCachePolicy": "UnprotectedWriteBack"}

Body example:

```
{"RAIDType": "RAID0", "Links": {"Drives": [{"@odata.id":  
"/redfish/v1/Systems/System.Embedded.1/Storage/Drives/Disk.Bay.3:Enclosur  
e.Internal.0-1:RAID.SL.3-1"}]}, "ReadCachePolicy": "Off",  
"WriteCachePolicy": "UnprotectedWriteBack"}
```

Success status code returned: 202

NOTE: For Read and Write Cache policies, iDRAC leverages DMTF values, DMTF schema will have supported values. For Disk Cache policy, this is an OEM property that leverage OEM values.

NOTE: PERC controller itself must support setting these VD attributes and all possible values. See the PERC documentation for supported VD attributes or possible values per PERC model.

JOB ID URI will be returned in JSON response headers location. Run GET command on this URI to monitor the job status to completion.

Command: GET

URI: redfish/v1/Managers/iDRAC.Embedded.1/Oem/Dell/Jobs/{job ID}

Header: content-type application/json

Auth: Basic or X auth

Success status code returned: 200

4.12 Change Virtual Disk Attributes

Command: PATCH

URI: redfish/v1/Systems/System.Embedded.1/Storage/{controller
ID}/Volumes/{virtual disk ID}/Settings

URI Example:

redfish/v1/Systems/System.Embedded.1/Storage/RAID.SL.31/Volumes/Disk.Virt
ual.1:RAID.SL.3-1/Settings

Header: content-type application/json

Auth: Basic or X auth

Body: {
 "@Redfish.SettingsApplyTime": {
 "ApplyTime": "Immediate"

```

    },
    "Oem": {
      "Dell": {
        "DellVolume": {
          "DiskCachePolicy": "<attribute value>"
        }
      }
    },
    "ReadCachePolicy": "<attribute value>",
    "WriteCachePolicy": "<attribute value>"
  }
}

```

```

Body example: {
  "@Redfish.SettingsApplyTime": {
    "ApplyTime": "Immediate"
  },
  "Oem": {
    "Dell": {
      "DellVolume": {
        "DiskCachePolicy": "Enabled"
      }
    }
  },
  "ReadCachePolicy": "ReadAhead",
  "WriteCachePolicy": "WriteThrough"
}

```

Success status code returned: 202

NOTE: For Read and Write Cache policies, iDRAC leverages DMTF values, DMTF schema will have supported values. For Disk Cache policy, this is OEM property, leverage OEM values.

NOTE: PERC controller itself must support setting these VD attributes and all possible values. See the PERC documentation for supported VD attributes or possible values per PERC model.

JOB ID URI will be returned in JSON response headers location. Run GET command on this URI to monitor the job status to completion.

Command: GET

URI: redfish/v1/Managers/iDRAC.Embedded.1/Oem/Dell/Jobs/{job ID}

Header: content-type application/json

Auth: Basic or X auth

Success status code returned: 200

4.13 Change Controller Rate Attributes

Command: PATCH

URI: redfish/v1/Systems/System.Embedded.1/Storage/{controller FQDD}/Controllers/{controller FQDD}/Settings

URI Example: redfish/v1/Systems/System.Embedded.1/Storage/RAID.SL.3-1/Controllers/RAID.SL.3-1/Settings

Header: content-type application/json

Auth: Basic or X token

Body Example to set DMTF and OEM controller properties:

```
{"@Redfish.SettingsApplyTime":{"ApplyTime":"Immediate"},"ControllerRates":
{"ConsistencyCheckRatePercent":
70},"Oem":{"Dell":{"DellStorageController":{"BackgroundInitializationRatePercent
": 88,"ReconstructRatePercent": 30}}}}
```

Success status code returned: 202

NOTE: Task ID URI will be returned in the headers location (Example: /redfish/v1/TaskService/Tasks/JID_684479528789). Run GET on this URI to confirm message property reports “successfully completed”.

4.14 Change Controller Mode Attribute

NOTE: Changing the controller mode is only supported on PERC 10 or older generation cards.

Command: PATCH

URI: redfish/v1/Systems/System.Embedded.1/Storage/{controller FQDD}/Controllers/{controller FQDD}/Settings

URI Example: redfish/v1/Systems/System.Embedded.1/Storage/RAID.SL.3-1/Controllers/RAID.SL.3-1/Settings

Header: content-type application/json

Auth: Basic or X token

Body Example to set PERC 9 controller boot mode to HBA:

```
{"Oem":{"Dell":{"DellStorageController":{"ControllerMode":"HBA"}}},"@Redfish.Set
tingsApplyTime":{"ApplyTime":"OnReset"}}
```

Success status code returned: 202

NOTE: Task ID URI will be returned in the headers location (Example: /redfish/v1/TaskService/Tasks/JID_684479528789). Run GET on this URI to confirm message property reports “successfully scheduled”. Now reboot the server to run the config job.

NOTE: Server reboot is needed to change the controller boot mode. PERC controllers do not support changing controller mode in real time.

Reboot the server to run the config job:

Command: POST

URI: redfish/v1/Systems/System.Embedded.1/Actions/ComputerSystem.Reset

Header: content-type application/json

Auth: Basic or X auth

```
Body: {  
  "ResetType": "GracefulRestart"  
}
```

NOTE: Once the server has been rebooted, run GET request on the same task ID URI to monitor the progress, confirm message property reports “successfully completed”.

4.15 Secure Virtual Disk

Command: POST

iDRAC9 URI:
redfish/v1/Systems/System.Embedded.1/Oem/Dell/DellRaidService/Actions/DellRaidService.LockVirtualDisk

iDRAC10 URI:
redfish/v1/Systems/System.Embedded.1/Oem/Dell/DellRaidService/Actions/DellRaidService.SecureVirtualDisk

Header: content-type application/json

Auth: Basic or X auth

```
Body: {  
  "TargetFQDD": "<virtual disk FQDD>"  
}
```

```
Body Example: {  
  "TargetFQDD": "Disk.Virtual.0:RAID.Integrated.1-1"  
}
```

Success status code returned: 202

JOB ID URI will be returned in JSON response headers location. Run GET command on this URI to monitor the job status to completion.

Command: GET

URI: redfish/v1/Managers/iDRAC.Embedded.1/Oem/Dell/Jobs/{job ID}

Header: content-type application/json

Auth: Basic or X auth

Success status code returned: 200

4.16 Initialize Virtual Disk

Command: POST

URI: redfish/v1/Systems/System.Embedded.1/Storage/{controller FQDD ID}/Volumes/{Virtual disk FQDD ID}/Actions/Volume.Initialize

Example URI: redfish/v1/Systems/System.Embedded.1/Storage/RAID.SL.3-1/Volumes/Disk.Virtual.0:RAID.SL.3-1/Actions/Volume.Initialize

Header: content-type application/json

Auth: Basic or X auth

```
Body: {  
  "InitializeType": "<Pass in value Fast or Slow>"  
}
```

Success status code returned: 202

JOB ID URI will be returned in JSON response headers location. Run GET command on this URI to monitor the job status to completion.

Command: GET

URI: redfish/v1/Managers/iDRAC.Embedded.1/Oem/Dell/Jobs/{job ID}

Header: content-type application/json

Auth: Basic or X auth

Success status code returned: 200

4.17 Set Local Key Management (LKM) Encryption

Note: iDRAC10 does not support setting LKM using iDRAC interfaces for PERC, this can only be done from F2 Device Settings. iDRAC9 though still supports this feature.

Command: POST

URI:
redfish/v1/Systems/System.Embedded.1/Oem/Dell/DellRaidService/Actions/DellRaidService.SetControllerKey

Header: content-type application/json

Auth: Basic or X auth

Body: {
 "TargetFQDD": "<controller ID>",
 "Key": "<passphrase>",
 "Keyid": "<key id>"
}

Body Example: {
 "TargetFQDD": "RAID.SL.3-1",
 "Key": "Test123##",
 "Keyid": "testkey"
}

Success status code returned: 202

JOB ID URI will be returned in JSON response headers location. Run GET command on this URI to monitor the job status to completion.

Command: GET

URI: redfish/v1/Managers/iDRAC.Embedded.1/Oem/Dell/Jobs/{job ID}

Header: content-type application/json

Auth: Basic or X auth

Success status code returned: 200

4.18 Remove Local Key Management (LKM) Encryption

Note: iDRAC10 does not support remove LKM using iDRAC interfaces for PERC, this can only be done from F2 Device Settings. iDRAC9 though still supports this feature.

Command: POST

URI:
redfish/v1/Systems/System.Embedded.1/Oem/Dell/DellRaidService/Actions/DellRaidService.RemoveControllerKey

Header: content-type application/json

Auth: Basic or X auth

Body: {
 "TargetFQDD": "<controller ID>"
}

Body Example: {
 "TargetFQDD": "RAID.SL.3-1"
}

Success status code returned: 202

JOB ID URI will be returned in JSON response headers location. Run GET command on this URI to monitor the job status to completion.

Command: GET

URI: redfish/v1/Managers/iDRAC.Embedded.1/Oem/Dell/Jobs/{job ID}

Header: content-type application/json

Auth: Basic or X auth

Success status code returned: 200

4.19 Get OEM Storage Controller Attributes

Command: GET

URI: redfish/v1/Systems/System.Embedded.1/Storage/{controller device ID}/Controllers/{controller device ID}?\$select=Oem/Dell/DellStorageController

URI example: redfish/v1/Systems/System.Embedded.1/Storage/RAID.SL.3-1/Controllers/RAID.SL.3-1?\$select=Oem/Dell/DellStorageController

Header: content-type application/json

Auth: Basic or X auth

Success status code returned: 200

4.20 Set OEM Storage Controller Attributes

Command: PATCH

URI: redfish/v1/Systems/System.Embedded.1/Storage/{controller device ID}/Controllers/{controller device ID}?\$select=Oem/Dell/DellStorageController

URI example: redfish/v1/Systems/System.Embedded.1/Storage/RAID.SL.3-1/Controllers/RAID.SL.3-1?\$select=Oem/Dell/DellStorageController

Header: content-type application/json

Auth: Basic or X auth

```
Body: {
  "@Redfish.SettingsApplyTime":{
    "ApplyTime":"Immediate"
  },
  "Oem":{
    "Dell":{
      "DellStorageController":{
        "<attribute name>":"<attribute value>"
      }
    }
  }
}
```

```

    }
  }
}

Body Example: {{
  "@Redfish.SettingsApplyTime":{
    "ApplyTime":"Immediate"
  },
  "Oem":{
    "Dell":{
      "DellStorageController":{
        "CopybackMode":"OnWithSMART"
      }
    }
  }
}
}

```

Success status code returned: 202

JOB ID URI will be returned in JSON response headers location. Run GET command on this URI to monitor the job status to completion.

Command: GET

URI: redfish/v1/Managers/iDRAC.Embedded.1/Oem/Dell/Jobs/{job ID}

Header: content-type application/json

Auth: Basic or X auth

Success status code returned: 200

5 Configuring DMTF Network Device Properties (Attributes)

The Redfish API supports Inventorying, Monitoring, and Configuring network devices of the server. You can inventory, monitor, and configure the following network devices:

- Network Interface Cards (NICs)
- Converged Network Adapters (CNAs)
- LAN On Motherboards (LOMs)
- Network Daughter Cards (NDCs)
- Fiber channel Card (FCs)

The command subset includes a list of NICs managed by server and their statistics, configuration of NIC parameters and NIC inventory.

NOTE: This section covers GET and SET for DMTF network device properties which these properties in general are limited on what can be configured. Refer to “Configuring OEM Network Device Properties” section which supports a wide range of properties.

5.1 Get Network Device IDs

Command: GET

URI: `redfish/v1/Chassis/System.Embedded.1/NetworkAdapters`

Header: `content-type application/json`

Auth: Basic or X auth

Success status code returned: 200

5.2 Get Network Device ID Details

Command: GET

URI: `redfish/v1/Chassis/System.Embedded.1/NetworkAdapters/{network device ID}/NetworkDeviceFunctions/{network device function ID}`

URI example:

`redfish/v1/Chassis/System.Embedded.1/NetworkAdapters/NIC.Integrated.1/NetworkDeviceFunctions/NIC.Integrated.1-1-1`

Header: content-type application/json

Auth: Basic or X auth

Success status code returned: 200

5.3 Get Network Port Details

Command: GET

URI: redfish/v1/Chassis/System.Embedded.1/NetworkAdapters/{network device ID}/NetworkPorts/{network port ID}

URI example:

redfish/v1/Chassis/System.Embedded.1/NetworkAdapters/NIC.Integrated.1/NetworkPorts/NIC.Integrated.1-1

Header: content-type application/json

Auth: Basic or X auth

Success status code returned: 200

5.4 Get Network Device Attribute Registry Details

Command: GET

URI: redfish/v1/Registries/NetworkAttributesRegistry_{network device ID}/NetworkAttributesRegistry_{network device ID}.json

URI example:

redfish/v1/Registries/NetworkAttributesRegistry_NIC.Integrated.1-1-1/NetworkAttributesRegistry_NIC.Integrated.1-1-1.json

Header: content-type application/json

Auth: Basic or X auth

Success status code returned: 200

NOTE: Getting the attribute registry details is helpful for configuring attributes. Output returns possible values, if read-only or any parent/child attribute dependencies.

5.5 Get Network Device Properties

Command: GET

URI: redfish/v1/Chassis/System.Embedded.1/NetworkAdapters/{network device ID}/NetworkDeviceFunctions/{network device function ID}?\$select=FibreChannel

URI: redfish/v1/Chassis/System.Embedded.1/NetworkAdapters/{network device ID}/NetworkDeviceFunctions/{network device function ID}?\$select=iSCSIBoot

URI example:

/redfish/v1/Chassis/System.Embedded.1/NetworkAdapters/NIC.Integrated.1/NetworkDeviceFunctions/NIC.Integrated.1-1-1?\$select=FibreChannelHeader:
content-type application/json

Auth: Basic or X auth

Success status code returned: 200

NOTE: Settable attributes will be reported in either FibreChannel or iSCSIBoot array. Depending on type of network device installed array may be empty which is expected due to network device not supporting those properties.

5.6 Set Network Device Properties

Command: PATCH

URI: redfish/v1/Systems/System.Embedded.1/NetworkAdapters/{network device ID}/NetworkDeviceFunctions/{network device function ID}/Settings

URI example:

redfish/v1/Systems/System.Embedded.1/NetworkAdapters/NIC.Integrated.1/NetworkDeviceFunctions/NIC.Integrated.1-1-1/Settings

Header: content-type application/json Auth: Basic or X auth

Body: {

```
"@Redfish.SettingsApplyTime":{ "ApplyTime":"Immediate"
}, "iSCSIBoot": {}, "FibreChannel": {}}
```

Body Example: {

```
"@Redfish.SettingsApplyTime":{ "ApplyTime":"Immediate"
}, "iSCSIBoot": {"InitiatorIPAddress":"10.10.1.10"}, "FibreChannel": {}}
```

Success status code returned: 202

JOB ID URI is returned in JSON response headers location. Run GET command on this URI to monitor the job status to show Scheduled.

Command: GET

URI: redfish/v1/Managers/iDRAC.Embedded.1/Oem/Dell/Jobs/{job ID}

Header: content-type application/json

Auth: Basic or X auth

Success status code returned: 200

Reboot the server to run the config job.

Command: POST

URI: redfish/v1/Systems/System.Embedded.1/Actions/ComputerSystem.Reset

Header: content-type application/json

Auth: Basic or X auth

```
Body: {
  "ResetType":"GracefulRestart"
}
```

Success status code returned: 204

Once you reboot the server, run GET command again on the same job ID URI to monitor the status until marked completed.

Command: GET

URI: redfish/v1/Managers/iDRAC.Embedded.1/Oem/Dell/Jobs/{job ID}

Header: content-type application/json

Auth: Basic or X auth

Success status code returned: 200

6 Configuring OEM Network Device Properties (Attributes)

6.1 Get Network Device IDs

Command: GET

URI: redfish/v1/Chassis/System.Embedded.1/NetworkAdapters

Header: content-type application/json

Auth: Basic or X auth

Success status code returned: 200

6.2 Get Network Device ID Details

Command: GET

URI: redfish/v1/Chassis/System.Embedded.1/NetworkAdapters/{network device ID}/NetworkDeviceFunctions/{network device function ID}

URI example:

redfish/v1/Chassis/System.Embedded.1/NetworkAdapters/NIC.Integrated.1/NetworkDeviceFunctions/NIC.Integrated.1-1-1

Header: content-type application/json

Auth: Basic or X auth

Success status code returned: 200

6.3 Get Network Port Details

Command: GET

URI: redfish/v1/Chassis/System.Embedded.1/NetworkAdapters/{network device ID}/NetworkPorts/{network port ID}

URI example:

redfish/v1/Chassis/System.Embedded.1/NetworkAdapters/NIC.Integrated.1/NetworkPorts/NIC.Integrated.1-1

Header: content-type application/json

Auth: Basic or X auth

Success status code returned: 200

6.4 Get Network Device Attribute Registry Details

Command: GET

URI: redfish/v1/Registries/NetworkAttributesRegistry_{network device ID}/NetworkAttributesRegistry_{network device ID}.json

URI example:

redfish/v1/Registries/NetworkAttributesRegistry_NIC.Integrated.1-1-1/NetworkAttributesRegistry_NIC.Integrated.1-1-1.json

Header: content-type application/json

Auth: Basic or X auth

Success status code returned: 200

NOTE: Getting the attribute registry details is helpful for configuring attributes. Output returns possible values, if read-only or any parent/child attribute dependencies.

6.5 Get Network Device Properties

Command: GET

URI: redfish/v1/Chassis/System.Embedded.1/NetworkAdapters/{network device ID}/NetworkDeviceFunctions/{network device function ID}/Oem/Dell/DellNetworkAttributes/{network device function ID}

URI example:

/redfish/v1/Chassis/System.Embedded.1/NetworkAdapters/NIC.Integrated.1/NetworkDeviceFunctions/NIC.Integrated.1-1-

1/Oem/Dell/DellNetworkAttributes/NIC.Integrated.1-1-1Auth: Basic or X auth

Success status code returned: 200

6.6 Set Network Device Properties

1. Command: PATCH

URI: redfish/v1/Chassis/System.Embedded.1/NetworkAdapters/{network device ID}/NetworkDeviceFunctions/{network device function ID}/Oem/Dell/DellNetworkAttributes/{network device function ID}/Settings

URI example:

redfish/v1/Chassis/System.Embedded.1/NetworkAdapters/NIC.Integrated.1/NetworkDeviceFunctions/NIC.Integrated.1-1-1/Oem/Dell/DellNetworkAttributes/NIC.Integrated.1-1-1/Settings

Header: content-type application/json Auth: Basic or X auth

Body: {

"@Redfish.SettingsApplyTime":{ "ApplyTime":"OnReset"

},

"Attributes":{

"<attribute name>":"<attribute value>"

}

}

Body Example: { "@Redfish.SettingsApplyTime":{

"ApplyTime":"OnReset"

},

"Attributes":{ "WakeOnLan":"Enabled"

}

}

Success status code returned: 202

2. JOB ID URI is returned in JSON response headers location. Run GET command on this URI to monitor the job status to show Scheduled.

Command: GET

URI: redfish/v1/Managers/iDRAC.Embedded.1/Jobs/{job ID}

Header: content-type application/json

Auth: Basic or X auth

Success status code returned: 200

3. Reboot the server to run the config job.

Command: POST

URI: redfish/v1/Systems/System.Embedded.1/Actions/ComputerSystem.Reset

Header: content-type application/json

Auth: Basic or X auth

```
Body: {  
  "ResetType": "GracefulRestart"  
}
```

Success status code returned: 204

4. Once you reboot the server, run GET command again on the same job ID URI to monitor the status until marked completed.

Command: GET

URI: redfish/v1/Managers/iDRAC.Embedded.1/Jobs/{job ID}

Header: content-type application/json

Auth: Basic or X auth

Success status code returned: 200

7 iDRAC9 User Account Management

7.1 Create New iDRAC User Account:

Command: PATCH

URI: redfish/v1/Managers/iDRAC.Embedded.1/Accounts/{user account ID, value will be 2 to 16}

Header: content-type application/json

Auth: Basic or X auth

```
Body: {  
  "UserName": "<new username string>",  
  "Password": "<new user password>",  
  "RoleId": "<Supported values: Administrator, Operator, ReadOnly or None>",  
  "Enabled": true  
}
```

Success status code returned: 200

NOTE: Pass in a boolean value of False for Enabled attribute if you want to create a new user but not enable the account.

7.2 Get iDRAC User Account Details:

Command: GET

URI: redfish/v1/Managers/iDRAC.Embedded.1/Accounts?\$expand=*\$levels=1)

Header: content-type application/json

Auth: Basic or X auth

Success status code returned: 200

7.3 Change iDRAC User Account Password:

Command: PATCH

URI: redfish/v1/Managers/iDRAC.Embedded.1/Accounts/{user account ID, value will be 2 to 16}

Header: content-type application/json

Auth: Basic or X auth

```
Body: {  
    "Password": "<new user password>"  
}
```

Success status code returned: 200

7.4 Change iDRAC User Account Privilege:

Command: PATCH

URI: redfish/v1/Managers/iDRAC.Embedded.1/Accounts/{user account ID, value will be 2 to 16}

Header: content-type application/json

Auth: Basic or X auth

```
Body: {  
    "RoleId": "<Supported values are Administrator, Operator, ReadOnly or None>"  
}
```

Success status code returned: 200

7.5 Delete iDRAC User Account:

Command: PATCH

URI: redfish/v1/Managers/iDRAC.Embedded.1/Accounts/{user account ID, value will be 2 to 16}

Header: content-type application/json

Auth: Basic or X auth

```
Body: {  
    "Enabled": false,  
    "RoleId": "None"  
}
```

Success status code returned: 200

Command: PATCH

URI: redfish/v1/Managers/iDRAC.Embedded.1/Accounts/{user account ID, value will be 2 to 16}

Header: content-type application/json

Auth: Basic or X auth

```
Body: {  
    "UserName":""  
}
```

Success status code returned: 200

NOTE: You must run (2) PATCH calls to completely remove this user ID entry from the iDRAC.

8 iDRAC10 User Account Management

8.1 Create New iDRAC User Account:

Command: POST

URI: redfish/v1/AccountService/Accounts

Header: content-type application/json

Auth: Basic or X auth

```
Body: {
  "Id": "<pass in value from 2 to 16>",
  "UserName": "<new username string>",
  "Password": "<new user password>",
  "RoleId": "<Supported values: Administrator, Operator, ReadOnly or None>",
  "Enabled": true
}
```

Success status code returned: 201

NOTE: Pass in a boolean value of False for Enabled attribute if you want to create a new user but not enable the account.

8.2 Get iDRAC User Account Details:

Command: GET

URI: redfish/v1/AccountService/Accounts?\$expand=*(\$levels=1)

Header: content-type application/json

Auth: Basic or X auth

Success status code returned: 200

8.3 Change iDRAC User Account Password:

Command: PATCH

URI: redfish/v1/AccountService/Accounts/{user account ID, value will be 2 to 16}

Header: content-type application/json

Auth: Basic or X auth

```
Body: {  
  "Password": "<new user password>"  
}
```

Success status code returned: 200

8.4 Change iDRAC User Account Privilege:

Command: PATCH

URI: redfish/v1/AccountService/Accounts/{user account ID, value will be 2 to 16}

Header: content-type application/json

Auth: Basic or X auth

```
Body: {  
  "RoleId": "<Supported values are Administrator, Operator, ReadOnly or None>"  
}
```

Success status code returned: 200

8.5 Delete iDRAC User Account:

Command: DELETE

URI: redfish/v1/AccountService/Accounts/{user account ID, value will be 2 to 16}

Header: content-type application/json

Auth: Basic or X auth

Success status code returned: 204

9 Configure iDRAC User SNMPv3 Settings

9.1 Get iDRAC user current SNMPv3 settings

Command: GET
URI:
redfish/v1/Managers/iDRAC.Embedded.1/Oem/Dell/DellAttributes/iDRAC.Embedded.1?\$select=Attributes/Users.2#ProtocolEnable
Header: content-type application/json
Auth: Basic or X auth
Success status code returned: 200

To get all other SNMPv3 attributes use the same URI but replace the attribute name with one of the following attributes:

- Users.2.AuthenticationProtocol
- Users.2.PrivacyProtocol
- Users.2.EnableSNMPv3Passphrase
- Users.2.SNMPv3AuthenticationPassphrase
- Users.2.SNMPv3PrivacyPassphrase

Note: In the attribute names replace “2” with the iDRAC user you want to configure for SNMPv3.

9.2 Get iDRAC user SNMPv3 attribute supported possible values

Command: GET
URI:
redfish/v1/Registries/ManagerAttributeRegistry/ManagerAttributeRegistry.v1_0_0.json
Header: content-type application/json
Auth: Basic or X auth
Success status code returned: 200

Example of registry details for attribute “Users.2.AuthenticationProtocol”:

```
{AttributeName: Users.2.AuthenticationProtocol
CurrentValue: None
DefaultValue: 2
DisplayName: SNMP V3 Authentication Protocol
DisplayOrder: 259
HelpText: The Authentication Protocol for SNMP v3
Hidden: False
Id: iDRAC.Embedded.1#Users.2#AuthenticationProtocol
MenuPath: ./iDRAC.Embedded.1/Users
```

```
Readonly: False
Regex:
Type: Enumeration
Value: [{'ValueDisplayName': 'None', 'ValueName': '0'}, {'ValueDisplayName': 'MD5', 'ValueName': '1'}, {'ValueDisplayName': 'SHA', 'ValueName': '2'}]
WriteOnly: False}
```

Note: You must parse the JSON output and look for each SNMPv3 attribute to get supported possible values.

Note: It is recommended to get supported values first before you set the attributes. Each iDRAC version or generation may not support the same values.

9.3 Set iDRAC user SNMPv3 settings

Command: PATCH

URI:

redfish/v1/Managers/iDRAC.Embedded.1/Oem/Dell/DellAttributes/iDRAC.Embedded.1

Header: content-type application/json

Auth: Basic or X auth

```
Body: {
  "Attributes":{
    "<attribute name>":"<attribute value>"
  }
}
```

```
Body example: {
  "Attributes":{
    "Users.2.ProtocolEnable":"Enabled",
    "Users.2.AuthenticationProtocol":"MD5",
    "Users.2.PrivacyProtocol":"AES-256",
    "Users.2.EnableSNMPv3Passphrase":"Enabled",
    "Users.2.SNMPv3AuthenticationPassphrase":"test12345",
    "Users.2.SNMPv3PrivacyPassphrase":"test56789"
  }
}
```

Success status code returned: 200

Note: In the attribute names replace “2” with the iDRAC user you want to configure for SNMPv3.

10 Manage iDRAC Syslog Settings

10.1 Get iDRAC Syslog Attributes:

Command: GET

URI:

redfish/v1/Managers/iDRAC.Embedded.1/Oem/Dell/DellAttributes/iDRAC.Embedded.1

Header: content-type application/json

Auth: Basic or X auth

Success status code returned: 200

NOTE: This returns all iDRAC settings besides syslog attributes. Supported syslog attributes:

SysLog.1.Port

SysLog.1.SysLogEnable

SysLog.1.Server1

SysLog.1.Server2

SysLog.1.Server3

To get details for only a specific attribute, use the `?$select` query parameter.

Command: GET

URI:

redfish/v1/Managers/iDRAC.Embedded.1/Oem/Dell/DellAttributes/iDRAC.Embedded.1?\$select=Attributes/{attribute name}

URI example:

redfish/v1/Managers/iDRAC.Embedded.1/Oem/Dell/DellAttributes/iDRAC.Embedded.1?\$select=Attributes/SysLog.1.Port

Header: content-type application/json

Auth: Basic or X auth

Success status code returned: 200

10.2 Get iDRAC Attribute Registry:

Command: GET

URI:
redfish/v1/Registries/ManagerAttributeRegistry/ManagerAttributeRegistry.v1_0_0.json

Header: content-type application/json

Auth: Basic or X auth

Success status code returned: 200

NOTE: Getting the attribute registry details will help for setting attributes. You can find out possible values, if attribute is read-only, any dependencies, regex support.

10.3 Set iDRAC Syslog Attributes:

Command: PATCH

URI:
redfish/v1/Managers/iDRAC.Embedded.1/Oem/Dell/DellAttributes/iDRAC.Embedded.1

Header: content-type application/json

Auth: Basic or X auth

Body: {
 "Attributes":{
 "<attribute name>":"<attribute value>"
 }
}

Body example: {
 "Attributes":{
 "SysLog.1.SysLogEnable":"Enabled",
 "SysLog.1.Port":"514",
 "SysLog.1.Server1":"192.168.0.130"
 }
}

Success status code returned: 200

NOTE: Server reboot is not required to apply iDRAC attributes, these are applied in real time.

11 Manage iDRAC SNMP Settings

11.1 Get iDRAC SNMP Attributes:

Command: GET

URI:

redfish/v1/Managers/iDRAC.Embedded.1/Oem/Dell/DellAttributes/iDRAC.Embedded.1

Header: content-type application/json

Auth: Basic or X auth

Success status code returned: 200

NOTE: This output will return all iDRAC attributes. There is no option to filter for specific attributes, you will need to parse JSON output, search for “SNMP” to find all SNMP attributes.

11.2 Set iDRAC SNMP Attributes:

Command: PATCH

URI:

redfish/v1/Managers/iDRAC.Embedded.1/Oem/Dell/DellAttributes/iDRAC.Embedded.1

Header: content-type application/json

Auth: Basic or X auth

```
Body: {
  "Attributes":{
    "<attribute name>":"<attribute value>"
  }
}
```

```
Body Example: {
  "Attributes":{
    "SNMPAlert.1.State":"Enabled",
    "SNMPAlert.1.Destination":"192.168.0.140",
    "SNMP.1.AgentCommunity":"public",
    "SNMP.1.AlertPort":162,
    "Users.4.ProtocolEnable":"Enabled",
    "SNMPAlert.1.SNMPv3Username":"user4"
  }
}
```

Success status code returned: 200

In the body example, following setting is done:

- Enable SNMP 1 alert state
- Set SNMP 1 alert destination to 192.168.0.140
- Set SNMP agent community to public
- Set SNMP port to 162
- Enable SNMP support for user 4

11.3 Set iDRAC SNMP Alert for Specific Components:

NOTE: You must leverage iDRAC Server Configuration Profile (SCP) feature to set iDRAC alerts.

Command: POST

iDRAC9 URI:

redfish/v1/Managers/iDRAC.Embedded.1/Actions/Oem/EID_674_Manager.ImportSystemConfiguration

iDRAC10 URI:

redfish/v1/Managers/iDRAC.Embedded.1/Actions/Oem/OemManager.ImportSystemConfiguration

Header: content-type application/json

Auth: Basic or X token

Body: {"ShareParameters": {"Target": "EventFilters", "IPAddress": "<network share IP>", "ShareType": "<pass in share type, examples NFS or HTTP>", "ShareName": "<network share name>", "FileName": "<pass in your SCP filename which has been edited to make alert changes>"} }

Success status code returned: 202

NOTE: In the Headers Location, job ID URI will be returned. Run GET on this URI to check the job status for import operation.

NOTE: In the SCP file you will see each supported component for Alerts which may have either Info, Warning or Critical attributes. Enable or disable depending on what events you want to receive.

An example from the SCP file when the SNMP alert is enabled for AMP critical event:

```
<!-- <Attribute Name="AMP_1_1#DisplayName">AMP_1_1 (Critical)</Attribute>-->
<Attribute Name="AMP_1_1#Alert#Email">Disabled</Attribute>
<Attribute Name="AMP_1_1#Alert#SNMP">Enabled</Attribute>
<Attribute Name="AMP_1_1#Alert#IPMI">Disabled</Attribute>
<Attribute Name="AMP_1_1#Alert#SysLog">Disabled</Attribute>
<Attribute Name="AMP_1_1#Alert#WSEventing">Disabled</Attribute>
<Attribute Name="AMP_1_1#Alert#OSLog">Disabled</Attribute>
<Attribute Name="AMP_1_1#Alert#RedfishEventing">Disabled</Attribute>
<Attribute Name="AMP_1_1#Action">No Action</Attribute>
```

12 Manage iDRAC Power

12.1 Get and SET iDRAC Power Cap attributes

12.1.1 Get iDRAC Power Cap attributes

Command: GET

URI:

redfish/v1/Managers/iDRAC.Embedded.1/Oem/Dell/DellAttributes/System.Embedded.1

Header: content-type application/json

Auth: Basic or X auth

Success status code returned: 200

NOTE: This will return all iDRAC settings besides power cap attributes.

Supported power cap policy attributes:

ServerPwr.1.PowerCapSetting
ServerPwr.1.ActivePowerCapVal
ServerPwr.1.PowerCapValue
ServerPwr.1.PowerCapMinThres
ServerPwr.1.PowerCapMaxThres

NOTE: For details about getting attribute registry details or using query parameters to get a specific attribute, see section 6 Manage iDRAC Syslog settings.

12.1.2 Set iDRAC Power Cap attributes

Command: PATCH

URI:

redfish/v1/Managers/iDRAC.Embedded.1/Oem/Dell/DellAttributes/System.Embedded.1

Header: content-type application/json

Auth: Basic or X auth

Body: {
 "Attributes":{
 "<attribute name>":"<attribute value>"

```
    }  
  }  
  
Body example: {  
  "Attributes":{  
    "ServerPwr.1.PowerCapSetting":"Enabled",  
    "ServerPwr.1.PowerCapValue":"700"  
  }  
}
```

Success status code returned: 200

NOTE: Before setting power cap check attributes `ServerPwr.1.PowerCapMinThres` and `ServerPwr.1.PowerCapMaxThres`, this gives you the power cap range for your server configuration.

NOTE: Server reboot is not required to apply iDRAC attributes, these get applied in real time.

12.2 iDRAC Power Configuration (Redundancy Policy, HotSpare, Primary PSU)

12.2.1 Get iDRAC Power Configuration attributes

Command: GET

URI:
`redfish/v1/Managers/iDRAC.Embedded.1/Oem/Dell/DellAttributes/System.Embedded.1`

Header: `content-type application/json`

Auth: Basic or X auth

Success status code returned: 200

NOTE: This returns all iDRAC settings besides power configuration attributes.

Supported power configuration attributes:

ServerPwr.1.PSRedPolicy

ServerPwr.1.PSRapidOn (this attribute means HotSpare)

ServerPwr.1.RapidOnPrimaryPSU

NOTE: For details about getting attribute registry details or using query parameters to get a specific attribute, see section 6 Manage iDRAC Syslog settings.

12.2.2 Set iDRAC Power Configuration attributes

Command: PATCH

URI:

redfish/v1/Managers/iDRAC.Embedded.1/Oem/Dell/DellAttributes/System.Embedded.1

Header: content-type application/json

Auth: Basic or X auth

```
Body: {  
  "Attributes":{  
    "<attribute name>":"<attribute value>"  
  }  
}
```

```
Body example: {  
  "Attributes":{  
    "ServerPwr.1.PSRedPolicy":"A/B Grid Redundant",  
    "ServerPwr.1.PSRapidOn":"Enabled",  
    "ServerPwr.1.RapidOnPrimaryPSU":"PSU2"  
  }  
}
```

Success status code returned: 200

NOTE: Before setting power cap check attributes ServerPwr.1.PowerCapMinThres and ServerPwr.1.PowerCapMaxThres, this gives you the power cap range for your server configuration.

NOTE: Server reboot is not required to apply iDRAC attributes, these get applied in real time.

13 Get/Set iDRAC NTP settings

13.1 Get iDRAC NTP settings

Command: GET

URI:

redfish/v1/Managers/iDRAC.Embedded.1/Oem/Dell/DellAttributes/iDRAC.Embedded.1?\$select=Attributes/NTPConfigGroup.1.NTPEnable

Header: content-type application/json

Auth: Basic or X auth

Success status code returned: 200

NOTE: Replace “NTPConfigGroup.1.NTPEnable” in the URI with attributes “NTPConfigGroup.1.NTP1” and “Time.1.Timezone” to get other NTP settings.

13.2 Set iDRAC NTP settings

Command: PATCH

URI:

redfish/v1/Managers/iDRAC.Embedded.1/Oem/Dell/DellAttributes/iDRAC.Embedded.1

Header: content-type application/json

Auth: Basic or X auth

```
Body: {
  "Attributes":{
    "<attribute name>":"<attribute value>"
  }
}
```

```
Body Example: {
  "Attributes":{
    "NTPConfigGroup.1.NTPEnable":"Enabled",
    "NTPConfigGroup.1.NTP1":"192.168.0.150",
    "Time.1.Timezone":"CST6CDT"
  }
}
```

Success status code returned: 200

NOTE: This example shows enabling NTP, setting NTP server 1 to 192.168.0.150 and time zone to central standard time.

14 Get/Set Chassis Indicator LED

14.1 Get Chassis Indicator LED Status

Command: GET
URI: redfish/v1/Chassis/System.Embedded.1?\$select= LocationIndicatorActive
Header: content-type application/json
Auth: Basic or X auth

Success status code returned: 200

14.2 Set Chassis Indicator LED

Command: PATCH

URI: redfish/v1/Chassis/System.Embedded.1

Header: content-type application/json

Auth: Basic or X auth

Body: {
 " LocationIndicatorActive":True
}

Success status code returned: 200

NOTE: Supported possible value is a Boolean.

15 Enabled/Configure iDRAC Alerts

15.1 Enable iDRAC Alert Setting

Command: PATCH

URI:
redfish/v1/Managers/iDRAC.Embedded.1/Oem/Dell/DellAttributes/iDRAC.Embedded.1

Header: content-type application/json

Auth: Basic or X auth

```
Body: {
  "Attributes":{
    "IPMILan.1.AlertEnable":"Enabled"
  }
}
```

Success status code returned: 200

15.2 Configure iDRAC Alerts

NOTE: You must leverage iDRAC Server Configuration Profile (SCP) feature to set iDRAC alerts.

Command: POST

iDRAC9 URI:
redfish/v1/Managers/iDRAC.Embedded.1/Actions/Oem/EID_674_Manager.ImportSystemConfiguration

iDRAC10 URI:
redfish/v1/Managers/iDRAC.Embedded.1/Actions/Oem/OemManager.ImportSystemConfiguration

Header: content-type application/json

Auth: Basic or X token

```
Body: {
  "ShareParameters":{
    "Target":"EventFilters",
    "IPAddress":"<network share IP>",
    "ShareType":"<pass in NFS CIFS HTTP or HTTPS>",
    "ShareName":"<network share name>",
  }
}
```

```
    "FileName": "<pass in your SCP filename which has been edited to make alert
changes>"
  }
}
```

Success status code returned: 202

NOTE: In the Headers Location, job ID URI will be returned. Run GET on this URI to check the job status for import operation.

NOTE: In the SCP file, each supported component for Alerts which may have either Info, Warning or Critical attributes are displayed. Enable or disable depending on what events you want to receive.

An example from the SCP file where Redfish eventing is enabled, IPMI, and SNMP to receive events when AMP critical event occurs:

```
<!-- <Attribute Name="AMP_1_1#DisplayName">AMP_1_1 (Critical)</Attribute>-->
<Attribute Name="AMP_1_1#Alert#Email">Disabled</Attribute>
<Attribute Name="AMP_1_1#Alert#SNMP">Enabled</Attribute>
<Attribute Name="AMP_1_1#Alert#IPMI">Enabled</Attribute>
<Attribute Name="AMP_1_1#Alert#SysLog">Disabled</Attribute>
<Attribute Name="AMP_1_1#Alert#WSEventing">Disabled</Attribute>
<Attribute Name="AMP_1_1#Alert#OSLog">Disabled</Attribute>
<Attribute Name="AMP_1_1#Alert#RedfishEventing">Enabled</Attribute>
<Attribute Name="AMP_1_1#Action">No Action</Attribute>
```

16 Redfish Operations Management

16.1 Manage iDRAC GUI Login Security Message String

16.1.1 Get iDRAC Login Security Message String

Command: GET

URI:

redfish/v1/Managers/iDRAC.Embedded.1/Oem/Dell/DellAttributes/iDRAC.Embedded.1?\$select=Attributes/GUI.1.SecurityPolicyMessage

Header: content-type application/json

Auth: Basic or X auth

Success status code returned: 200

16.1.2 Set iDRAC Login Security Message String

Command: PATCH

URI:

redfish/v1/Managers/iDRAC.Embedded.1/Oem/Dell/DellAttributes/iDRAC.Embedded.1

Header: content-type application/json

Auth: Basic or X auth

```
Body: {
  "Attributes":{
    "<attribute name>":"<attribute value>"
  }
}
```

```
Body example: {
  "Attributes":{
    "GUI.1.SecurityPolicyMessage":"Test security login message"
  }
}
```

Success status code returned: 200

NOTE: Server reboot is not required to apply iDRAC attributes, these will get applied in real time.

16.2 Server Processor Information

16.2.1 Get All Processor Details:

Command: GET

URI: redfish/v1/Systems/System.Embedded.1/Processors?\$expand=*\$levels=1)

Header: content-type application/json

Auth: Basic or X auth

Success status code returned: 200

16.3 Get Specific Processor Property

Command: GET

URI: redfish/v1/Systems/System.Embedded.1/Processors/{CPU ID}?\$select={property name}

URI example:

redfish/v1/Systems/System.Embedded.1/Processors/CPU.Socket.1?\$select=Oem/Dell/DellProcessor/CPUFamily

Header: content-type application/json

Auth: Basic or X auth

Success status code returned: 200

16.4 Server Memory Information

16.4.1 Get All Memory Details

Command: GET

URI: redfish/v1/Systems/System.Embedded.1/Memory?\$expand=*\$levels=1)

Header: content-type application/json

Auth: Basic or X auth

Success status code returned: 200

16.4.2 Get Specific Memory Property

Command: GET

URI: redfish/v1/Systems/System.Embedded.1/Memory/{Memory ID}?\$select={property name}

URI example:

redfish/v1/Systems/System.Embedded.1/Memory/DIMM.Socket.A2?\$select=Manufacturer

Header: content-type application/json

Auth: Basic or X auth

Success status code returned: 200

17 Redfish BIOS Settings

17.1 Configure BIOS Settings

17.1.1 Get BIOS Attributes

Command: GET

URI: redfish/v1/Systems/System.Embedded.1/Bios

Header: content-type application/json

Auth: Basic or X auth

Success status code returned: 200

17.1.2 Get BIOS attribute registry

Command: GET

URI: redfish/v1/Systems/System.Embedded.1/Bios/BiosRegistry

Header: content-type application/json

Auth: Basic or X auth

Success status code returned: 200

NOTE: Getting the attribute registry details will help for setting attributes. You can find out possible values, if attribute is read-only, any dependencies, regex support.

17.2 Set BIOS Attributes (set pending values and create config job one PATCH call)

1. Command: PATCH

URI: redfish/v1/Systems/System.Embedded.1/Bios/Settings

Header: content-type application/json

Auth: Basic or X auth

```
Body: {
  "@Redfish.SettingsApplyTime":{
    "ApplyTime":"OnReset"
  },
  "Attributes":{
    "<attribute name>":"<attribute value>"
  }
}
```

```
Body example: {
  "@Redfish.SettingsApplyTime":{
    "ApplyTime":"OnReset"
  },
  "Attributes":{
    "EmbSata":"RaidMode",
    "NvmeMode":"Raid"
  }
}
```

Success status code returned: 202

2. JOB ID URI will be returned in JSON response headers location. Run GET command on this URI to monitor the job status to show Scheduled.

Command: GET

URI: redfish/v1/Managers/iDRAC.Embedded.1/Oem/Dell/Jobs/{job ID}

Header: content-type application/json

Auth: Basic or X auth

Success status code returned: 200

3. Reboot the server to run the config job.

Command: POST

URI: redfish/v1/Systems/System.Embedded.1/Actions/ComputerSystem.Reset

Header: content-type application/json

Auth: Basic or X auth

```
Body: {
  "ResetType":"GracefulRestart"
}
```

Success status code returned: 204

4. Once you reboot the server, run GET command again on the same job ID URI to monitor the status until marked completed.

Command: GET

URI: redfish/v1/Managers/iDRAC.Embedded.1/Jobs/{job ID}

Header: content-type application/json

Auth: Basic or X auth

Success status code returned: 200

17.3 Set BIOS Attributes (set pending values only)

Note: Dell Technologies recommends to not pass in "@Redfish.SettingsApplyTime" for the PATCH body because it only sets the attribute's pending values, and a config job is not created.

Command: PATCH

URI: redfish/v1/Systems/System.Embedded.1/Bios/Settings

Header: content-type application/json

Auth: Basic or X auth

```
Body: {  
  "Attributes":{  
    "<attribute name>":"<attribute value>"  
  }  
}
```

```
Body example: {  
  "Attributes":{  
    "EmbSata":"RaidMode",  
    "NvmeMode":"Raid"  
  }  
}
```

Success status code returned: 202

To set more pending attribute values modify the JSON body and run the same PATCH command again.

Command: PATCH

URI: redfish/v1/Systems/System.Embedded.1/Bios/Settings

Header: content-type application/json

Auth: Basic or X auth

```
Body: {  
  "Attributes":{  
    "<attribute name>":"<attribute value>"  
  }  
}
```

```
Body example: {  
  "Attributes":{  
    "MemTest":"Enabled"  
  }  
}
```

If you need to check what BIOS attribute pending values have already been set, run the GET command below.

Command: GET

URI: redfish/v1/Systems/System.Embedded.1/Bios/Settings

Header: content-type application/json

Auth: Basic or X auth

Success status code returned: 200

In the JSON output, you should see pending attributes section (example below shows SysProfile and MemTest attributes have pending values set).

```
{  
  "@odata.context": "/redfish/v1/$metadata#Bios.Bios",  
  "@odata.id": "/redfish/v1/Systems/System.Embedded.1/Bios/Settings",  
  "@odata.type": "#Bios.v1_2_0.Bios",  
  "Id": "Settings",  
  "Name": "BIOS Configuration Pending Settings",  
  "Description": "BIOS Configuration Pending Settings. These settings will be  
applied on next system reboot.",  
  "AttributeRegistry": "BiosAttributeRegistry.v1_0_3",  
  "Attributes": {  
    "SysProfile": "Custom",  
    "MemTest": "Disabled"  
  },  
}
```

After you set all pending attribute values, create the BIOS config job.

Command: POST

URI: redfish/v1/Managers/iDRAC.Embedded.1/Oem/Dell/Jobs

Header: content-type application/json

Auth: Basic or X auth

Body:

```
{"TargetSettingsURI":"/redfish/v1/Systems/System.Embedded.1/Bios/Settings"}
```

Note: Job ID URI should be returned in the headers output.

Confirm the config job is in scheduled job state.

Command: GET

URI: redfish/v1/Managers/iDRAC.Embedded.1/Oem/Dell/Jobs/{job ID}

Header: content-type application/json

Auth: Basic or X auth

Success status code returned: 200

Reboot the server to initiate the job to apply the attribute changes.

Command: POST

URI: redfish/v1/Systems/System.Embedded.1/Actions/ComputerSystem.Reset

Header: content-type application/json

Auth: Basic or X auth

Body: {"ResetType":"ForceRestart"}

17.4 Configure BIOS Boot Order

17.4.1 Get Current BIOS Boot Order

Command: GET

URI: redfish/v1/Systems/System.Embedded.1/BootOptions?\$expand=*\$levels=1)

Header: content-type application/json

Auth: Basic or X auth

Success status code returned: 200

NOTE: For each boot order device entry, property ID value will be used for changing the boot order.

17.4.2 Set BIOS Boot Order

Command: PATCH

iDRAC9 URI: redfish/v1/Systems/System.Embedded.1

iDRAC10 URI: redfish/v1/Systems/System.Embedded.1/Settings

Header: content-type application/json

Auth: Basic or X auth

```
Body: {
  "Boot":{
    "BootOrder":[
      "<boot order device id"
    ]
  }
}
```

```
Body example: {
  "Boot":{
    "BootOrder":[
      "Boot0000",
      "Boot0008",
      "Boot0009"
    ]
  }
}
```

Success status code returned: 202

NOTE: For boot order device IDs you can pass in one, multiple or all boot order devices. If you only pass in one boot order device Id, that device will get set as first device and the rest of the boot order will get shifted down.

JOB ID URI will be returned in JSON response headers location. Run GET command on this URI to monitor the job status to show Scheduled.

Command: GET

URI: redfish/v1/Managers/iDRAC.Embedded.1/Oem/Dell/Jobs/{job ID}

Header: content-type application/json

Auth: Basic or X auth

Success status code returned: 200

Reboot the server to run the config job.

Command: POST

URI: redfish/v1/Systems/System.Embedded.1/Actions/ComputerSystem.Reset

Header: content-type application/json

Auth: Basic or X auth

```
Body: {  
    "ResetType": "GracefulRestart"  
}
```

Success status code returned: 204

Once you reboot the server, run GET command again on the same job ID URI to monitor the status until marked completed.

Command: GET

URI: redfish/v1/Managers/iDRAC.Embedded.1/Oem/Dell/Jobs/{job ID}

Header: content-type application/json

Auth: Basic or X auth

Success status code returned: 200

17.5 Enabling/Disabling BIOS Boot Order Devices

17.5.1 Get Current BIOS Boot Order Enabled/Disabled State

Command: GET

URI: redfish/v1/Systems/System.Embedded.1/BootOptions?\$expand=*\$levels=1)

Header: content-type application/json

Auth: Basic or X auth

Success status code returned: 200

NOTE: For each boot order device entry, property BootOptionEnabled value is used to determine if boot entry is enabled (true) or disabled (false).

17.5.2 Disable BIOS Boot Order Device

Command: PATCH

URI: redfish/v1/Systems/System.Embedded.1/BootOptions/{boot order device ID}

URI example: /redfish/v1/Systems/System.Embedded.1/BootOptions/Boot0000

Header: content-type application/json

Auth: Basic or X auth

```
Body: {  
  "BootOptionEnabled":false  
}
```

Success status code returned: 202

NOTE: Boolean value for property BootOptionEnabled. If you want to enable a boot order device, pass in a value of True.

JOB ID URI is returned in JSON response headers location. Run GET command on this URI to monitor the job status to show Scheduled.

Command: GET

URI: redfish/v1/Managers/iDRAC.Embedded.1/Oem/Dell/Jobs/{job ID}

Header: content-type application/json

Auth: Basic or X auth

Success status code returned: 200

Reboot the server to run the config job.

Command: POST

URI: redfish/v1/Systems/System.Embedded.1/Actions/ComputerSystem.Reset

Header: content-type application/json

Auth: Basic or X auth

Body: {

 "ResetType": "GracefulRestart"

}

Success status code returned: 204

After you reboot the server, run GET command again on the same job ID URI to monitor the status until marked completed.

Command: GET

URI: redfish/v1/Managers/iDRAC.Embedded.1/Oem/Dell/Jobs/{job ID}

Header: content-type application/json

Auth: Basic or X auth

Success status code returned: 200

17.6 BIOS Reset to Default Settings (RTD)

17.6.1 Get Current BIOS RTD setting:

Command: GET

URI:

redfish/v1/Managers/iDRAC.Embedded.1/Oem/Dell/DellAttributes/LifecycleController.Embedded.1?\$select=Attributes/LCAttributes.1.BIOSRTDRequested

Header: content-type application/json

Auth: Basic or X auth

Success status code returned: 200

NOTE: Boolean value of True or False will be returned.

17.6.2 Set BIOS RTD

Command: POST

URI: redfish/v1/Systems/System.Embedded.1/Bios/Actions/Bios.ResetBios

Header: content-type application/json

Auth: Basic or X auth

Body: {}

Note: Empty payload body is required to pass in.

Success status code returned: 202

After performing the action for BIOS RTD, reboot the server for BIOS to apply RTD changes.

Command: POST

URI: redfish/v1/Systems/System.Embedded.1/Actions/ComputerSystem.Reset

Header: content-type application/json

Auth: Basic or X auth

Body: {

 "ResetType": "GracefulRestart"

}

18 Boot images: Physical and Virtual

18.1 Attach ISO as Virtual CD and Onetime Boot Virtual CD

Get Virtual Media Device Details.

Command: GET

URI: redfish/v1/Systems/System.Embedded.1/VirtualMedia?\$expand=*(\$levels=1)

Header: content-type application/json

Auth: Basic or X auth

Success status code returned: 200

NOTE: Best practice to get details for virtual media devices first, see if any devices are already attached.

Attach ISO as a virtual CD.

Command: POST

URI: redfish/v1/Systems/System.Embedded.1/VirtualMedia/{virtual device index ID}/Actions/VirtualMedia.InsertMedia

URI Example:

redfish/v1/Systems/System.Embedded.1/VirtualMedia/1/Actions/VirtualMedia.InsertMedia

Header: content-type application/json

Auth: Basic or X auth

Body: {

 "Image": "{URI path of the ISO}",

 "Inserted": true,

 "WriteProtected": true

}

Body Example: {

```
    "Image": "http://128.173.237.17/pub/CentOS/8-stream/isos/x86_64/CentOS-Stream-8-x86_64-latest-boot.iso",
    "Inserted": true,
    "WriteProtected": true
}
```

Success status code returned: 204

Set CD as next onetime boot device.

```
Command: PATCH
URI: redfish/v1/Systems/System.Embedded.1
Header: content-type application/json
Auth: Basic or X auth
Body: {
    "Boot": {
        "BootSourceOverrideTarget": "Cd",
        "BootSourceOverrideEnabled": "Once"
    }
}
```

Success status code returned: 200

Reboot server.

```
Command: POST
URI: redfish/v1/Systems/System.Embedded.1/Actions/ComputerSystem.Reset
Header: content-type application/json
Auth: Basic or X auth
Body: {
    "ResetType": "GracefulRestart"
}
```

```
}
```

Success status code returned: 204

18.2 Attach Multiple Virtual Media Devices

Get Virtual Media Device Details.

Command: GET

URI: redfish/v1/Systems/System.Embedded.1/VirtualMedia?\$expand=*(levels=1)

Header: content-type application/json

Auth: Basic or X auth

Success status code returned: 200

NOTE: Best practice to get details for virtual media devices first, see if any devices are already attached.

Attach ISO as virtual CD using virtual device index 1.

Command: POST

URI: redfish/v1/Systems/System.Embedded.1/VirtualMedia/{virtual device index ID}/Actions/VirtualMedia.InsertMedia

URI Example:

redfish/v1/Systems/System.Embedded.1/VirtualMedia/1/Actions/VirtualMedia.InsertMedia

Header: content-type application/json

Auth: Basic or X auth

Body: {

 "Image": "{URI path of the ISO}",

 "Inserted": true,

 "WriteProtected": true

}

Body Example: {

```
"Image": "http://128.173.237.17/pub/CentOS/8-stream/isos/x86_64/CentOS-Stream-8-x86_64-latest-boot.iso",  
  
  "Inserted": true,  
  
  "WriteProtected": true  
}
```

Success status code returned: 204

Attach IMG image file using virtual device index 2.

Command: POST

URI: redfish/v1/Systems/System.Embedded.1/VirtualMedia/{virtual device index ID}/Actions/VirtualMedia.InsertMedia

URI Example:

redfish/v1/Systems/System.Embedded.1/VirtualMedia/2/Actions/VirtualMedia.InsertMedia

Header: content-type application/json

Auth: Basic or X auth

Body: {

```
"Image": "{URI path of the ISO}",  
  
  "Inserted": true,  
  
  "WriteProtected": true  
}
```

Body Example: {

```
"Image": "http://192.168.0.130/http_share/testfile.img",  
  
  "Inserted": true,  
  
  "WriteProtected": true  
}
```

Success status code returned: 204

Get Virtual Media Device Details to confirm attach status for both virtual media index ids.

Command: GET

URI: redfish/v1/Systems/System.Embedded.1/VirtualMedia?\$expand=*\$levels=1)

Header: content-type application/json

Auth: Basic or X auth

Success status code returned: 200

18.3 Eject (Detach) Virtual Media Devices

Command: POST

URI: redfish/v1/Systems/System.Embedded.1/VirtualMedia/{virtual device index ID}/Actions/VirtualMedia.EjectMedia

URI example to detach device ID 1:
redfish/v1/Systems/System.Embedded.1/VirtualMedia/1/Actions/VirtualMedia.EjectMedia

Header: content-type application/json

Auth: Basic or X auth

Body: {}

NOTE: Pass in empty JSON body.

Success status code returned: 204

18.4 Boot to Network ISO Using OEM Extension

Attach ISO which is located on either NFS or CIFS share (NFS and CIFS are the only supported network protocols for this action).

Command: POST

URI:
redfish/v1/Systems/System.Embedded.1/Oem/Dell/DellOSDeploymentService/Actions/DellOSDeploymentService.BootToNetworkISO

Header: content-type application/json

Auth: Basic or X auth

Body: {

"IPAddress": "<share ip>",

"ShareType": "<pass in NFS or CIFS>",

"ShareName": "<share path of iso>",

"ImageName": "<ISO name>"

}

Body Example: {

"IPAddress": "192.168.0.130",

"ShareType": "NFS",

"ShareName": "/nfs/RHEL_84_unattend_install",

"ImageName": "RHEL84.iso"

}

Success status code returned: 202

NOTE: In the location headers output, you should see a task URI. Run GET on this URI to monitor the status of this action (Example: /redfish/v1/TaskService/Tasks/OSDeployment).

NOTE: Once you run this action, server will automatically reboot and boot to this ISO after POST completes.

Monitor the task status until it is marked completed.

Command: GET

URI: redfish/v1/TaskService/Tasks/OSDeployment

Header: content-type application/json

Auth: Basic or X auth

Success status code returned: 200

Check the attach status of the ISO

Command: POST

URI:

redfish/v1/Systems/System.Embedded.1/Oem/Dell/DellOSDeploymentService/Actions/DellOSDeploymentService.GetAttachStatus

Header: content-type application/json

Auth: Basic or X auth

Body: {}

NOTE: You must pass in empty JSON body.

Success status code returned: 200

Once the OS installation is complete, you can either wait for the internal timeout to expire to auto detach the ISO (18 hours) or you can detach the ISO using the action below.

Command: POST

URI:

redfish/v1/Systems/System.Embedded.1/Oem/Dell/DellOSDeploymentService/Actions/DellOSDeploymentService.DetachISOImage

Header: content-type application/json

Auth: Basic or X auth

Body: {}

NOTE: You must pass in empty JSON body.

Success status code returned: 200

18.5 Configure BIOS Attributes and Set Onetime Boot to HTTP Boot Device

18.5.1 Enable HTTP Device 1 and Set HTTP Device 1 Path BIOS Attributes

Command: PATCH

URI: redfish/v1/Systems/System.Embedded.1/Bios/Settings

Header: content-type application/json

Auth: Basic or X auth

```
Body: {
  "@Redfish.SettingsApplyTime":{
    "ApplyTime":"OnReset"
  },
  "Attributes":{
    "HttpDev1EnDis":"Enabled",
    "HttpDev1Uri":"<HTTP URI path to bootable image>"
  }
}
```

```
Body Example: {
  "@Redfish.SettingsApplyTime":{
    "ApplyTime":"OnReset"
  },
  "Attributes":{
    "HttpDev1EnDis":"Enabled",
    "HttpDev1Uri":"http://192.168.0.130/shell.efi"
  }
}
```

Success status code returned: 202

JOB ID URI will be returned in JSON response headers location. Run GET command on this URI to monitor the job status to show Scheduled.

Command: GET

URI: redfish/v1/Managers/iDRAC.Embedded.1/Oem/Dell/Jobs/{job ID}

Header: content-type application/json

Auth: Basic or X auth

Success status code returned: 200

Reboot the server to run the config job.

Command: POST

URI: redfish/v1/Systems/System.Embedded.1/Actions/ComputerSystem.Reset

Header: content-type application/json

Auth: Basic or X auth

```
Body: {  
    "ResetType": "GracefulRestart"  
}
```

Success status code returned: 204

Once you reboot the server, run GET command again on the same job ID URI to monitor the status until marked completed.

Command: GET

URI: redfish/v1/Managers/iDRAC.Embedded.1/Oem/Dell/Jobs/{job ID}

Header: content-type application/json

Auth: Basic or X auth

Success status code returned: 200

18.5.2 Get Current Boot Order, Locate HTTP Device 1

Command: GET

URI: redfish/v1/Systems/System.Embedded.1/BootOptions?\$expand=*(levels=1)

Header: content-type application/json

Auth: Basic or X auth

Success status code returned: 200

Example of HTTP Device 1 entry from the boot order.

```
{ '@odata.context': '/redfish/v1/$metadata#BootOption.BootOption',  
  '@odata.id': '/redfish/v1/Systems/System.Embedded.1/BootOptions/Boot0003',  
  '@odata.type': '#BootOption.v1_0_4.BootOption',
```

```
'BootOptionEnabled': True,
'BootOptionReference': 'Boot0003',
'Description': 'Current settings of the UEFI Boot option',
'DisplayName': 'HTTP Device 1: Embedded NIC 1 Port 1 Partition 1',
'Id': 'Boot0003',
'Name': 'Uefi Boot Option',
'UefiDevicePath': 'VenHw(986D1755-B9D0-4F8D-A0DA-D1DB18671111) '}
```

18.5.3 Set Next Onetime Boot to HTTP Boot Entry Device and Reboot Server

Command: PATCH

URI: redfish/v1/Systems/System.Embedded.1

Header: content-type application/json

Auth: Basic or X auth

```
Body: {
  "Boot":{
    "BootSourceOverrideTarget":"UefiTarget",
    "UefiTargetBootSourceOverride":"<UEFI device path string>"
  }
}
```

```
Body Example: {
  "Boot":{
    "BootSourceOverrideTarget":"UefiTarget",
    "UefiTargetBootSourceOverride":"VenHw(986D1755-B9D0-4F8D-A0DA-
D1DB18671111) "
  }
}
```

Success status code returned: 200

Command: POST

URI: redfish/v1/Systems/System.Embedded.1/Actions/ComputerSystem.Reset

Header: content-type application/json

Auth: Basic or X auth

```
Body: {
  "ResetType":"GracefulRestart"
}
```

Success status code returned: 204

19 Server Physical Location Attributes

19.1 Get Server Location Attributes (OEM)

Command: GET

URI:

redfish/v1/Managers/iDRAC.Embedded.1/Oem/Dell/DellAttributes/System.Embedded.1

Header: content-type application/json

Auth: Basic or X auth

Success status code returned: 200

Supported power cap policy attributes:

- ServerTopology.1.AisleName
- ServerTopology.1.DataCenterName
- ServerTopology.1.RackName
- ServerTopology.1.RoomName
- ServerTopology.1.RackSlot

NOTE: For details about getting attribute registry details or using query parameters to get a specific attribute, see section 6 Manage iDRAC Syslog settings.

19.2 Set Server Location Attributes (OEM)

Command: PATCH

URI:

redfish/v1/Managers/iDRAC.Embedded.1/Oem/Dell/DellAttributes/System.Embedded.1

Header: content-type application/json

Auth: Basic or X auth

```
Body: {
  "Attributes":{
    "<attribute name>":"<attribute value>"
  }
}
```

```
Body example: {
  "Attributes":{
    "ServerTopology.1.DataCenterName":"Dell",
    "ServerTopology.1.RoomName":"Server Room",
    "ServerTopology.1.RackName":"Rack 12",
    "ServerTopology.1.RackSlot":1,
    "ServerTopology.1.AisleName":"10"
  }
}
```

Success status code returned: 200

19.3 Get Server Location Attributes (DMTF)

Command: GET

URI: redfish/v1/Chassis/System.Embedded.1?\$select=Location

Header: content-type application/json

Auth: Basic or X auth

Success status code returned: 200

19.4 Set Server Location Attributes (DMTF)

Command: PATCH

URI: redfish/v1/Chassis/System.Embedded.1

Header: content-type application/json

Auth: Basic or X auth

```
Body example: {"Location":{"Placement":{"Rack":"Rack 4G","Row":"Row
99"},"PostalAddress":{"Building":"Round Rock 1", "Room":"Test lab"}}}
```

Success status code returned: 200

19.5 Get Server Slot Details

Get server slot details, this output is helpful to determine which slots are populated/empty in the server. Slot types include CPU, DIMMS, PCIe and Disk Bays.

Command: GET

URI: redfish/v1/Dell/Systems/System.Embedded.1/DellSlotCollection

Header: content-type application/json

Auth: Basic or X auth

Success status code returned: 200

NOTE: This URI returns 50 entries, use the \$skip query parameter to get the next 50 entries if a skip URI is reported in the JSON output.

Examples:

redfish/v1/Dell/Systems/System.Embedded.1/DellSlotCollection?\$skip=50

redfish/v1/Dell/Systems/System.Embedded.1/DellSlotCollection?\$skip=100

20 iDRAC Lifecycle Controller

20.1 Verify Server Status/Lifecycle Controller Status/Real Time Storage Monitor Status

Before performing any iDRAC workflow (get inventory, firmware update, or configuration change) or after an iDRAC reboot, a/c cycle or server reboot, it is recommended to run OEM action

`GetRemoteServicesAPIStatus`. This OEM action confirms that the server is out of POST and Lifecycle Controller (LC) is in a good state (not being used).

Command: POST

URI:

`redfish/v1/Managers/iDRAC.Embedded.1/Oem/Dell/DellLCService/Actions/DellLCService.GetRemoteServicesAPIStatus`

Header: content-type application/json

Auth: Basic or X auth

Body: {}

NOTE: You must pass in an empty body for this OEM action.

Success status code returned: 200

The properties returned in the JSON output:

- `LCStatus`—reports the overall status of the Lifecycle Controller. It is recommended to check LC status before running any update or configuration workflow.
- `RTStatus`—reports the overall status of iDRAC storage real-time monitoring service. Reporting ready, means all storage inventory is current, and you can perform either get storage inventory details or perform RAID real-time config jobs.

Note: If `RTStatus` report is not ready and you attempt to run any RAID operation and create a job, the job type will be staged, and server reboot is required to run the job.

Note: If your server contains many storage devices (for example: multiple external storage controllers connected over 100 hard drives) and you reboot the server, after POST completes `RTStatus` may take up to 20 minutes to report **Ready** status.

- `ServerStatus`—reports the overall status of the server(POST, out of POST, or F2).

Note: If the server status reports in POST or F2 and you attempt to run a firmware update or configuration job, the job may not run and return LC in use message. Correct the server status to report out of POST and then the job will run and complete.

20.2 Insert iDRAC Lifecycle Logs Custom Comment

Command: POST

URI:

redfish/v1/Managers/iDRAC.Embedded.1/Oem/Dell/DellLCService/Actions/DellLCService.InsertCommentInLCLog

Header: content-type application/json

Auth: Basic or X auth

```
Body: {  
  "Comment": "<your custom string message>"  
}
```

Success status code returned: 200

NOTE: Custom string message in the LC logs will be logged with message ID WRK0001.

20.3 View iDRAC Lifecycle Logs

Command: GET

URI: redfish/v1/Managers/iDRAC.Embedded.1/Logs/Lclog

Header: content-type application/json

Auth: Basic or X auth

Success status code returned: 200

NOTE: This URI returns the latest 50 entries, use the `$skip` query parameter to get the next 50 entries.

Examples:

```
/redfish/v1/Managers/iDRAC.Embedded.1/LogServices/Lclog/Entries?$skip=50  
/redfish/v1/Managers/iDRAC.Embedded.1/LogServices/Lclog/Entries?$skip=100  
/redfish/v1/Managers/iDRAC.Embedded.1/LogServices/Lclog/Entries?$skip=150
```

NOTE: If you want to get the complete LC logs using one operation call, leverage the OEM extension.

Command: POST

URI:

redfish/v1/Managers/iDRAC.Embedded.1/Oem/Dell/DellLCService/Actions/DellLCService.ExportLCLog

Header: content-type application/json

Auth: Basic or X auth

```
Body: {  
  "ShareType": "Local"  
}
```

Success status code returned: 202

NOTE: In the header location, you will see URI returned /redfish/v1/Dell/lclog.xml. Run GET on this URI to download LC logs file.

20.4 Prevent iDRAC Lifecycle Logs overflow

This feature prevents iDRAC Lifecycle (LC) Log overflow due to high frequency of logins from the management client. Enabling this feature will not capture login events to the iDRAC from a client. For example: Redfish GET request to the iDRAC will not log USR0030 event in the LC logs.

Command: PATCH

URI:
redfish/v1/Managers/iDRAC.Embedded.1/Oem/Dell/DellAttributes/iDRAC.Embedded.1

Header: content-type application/json

Auth: Basic or X auth

```
Body: {"Attributes":{"Logging.1.LCLogAggregation":"Enabled"}}
```

Success status code returned: 200

NOTE: By default, the attribute is set to disabled.

21 Server POST Code

Command: GET

URI:

redfish/v1/Managers/iDRAC.Embedded.1/Oem/Dell/DellAttributes/iDRAC.Embedded.1?\$select=Attributes/SysInfo.1.POSTCode

Header: content-type application/json

Auth: Basic or X auth

Success status code returned: 200

NOTE: The POST code value will be returned in decimal format. Convert it to hex to identify the POST code as follows.

```
0x00, "Unrecognized Post Code.",
0x01, "System Power On.",
0x02, "CPU Microcode load.",
0x03, "Chipset Initialization.",
0x04, "Memory Configuration.",
0x05, "Shadow BIOS.",
0x06, "Multiprocessor Initialization.",
0x07, "POST processing start.",
0x08, "System Management Mode (SMM) initialization.",
0x09, "PCI bus enumeration & video initialization.",
0x0A, "iDRAC is ready.",
0x0B, "Extended Memory test started.",
0x0C, "Extended Memory test running \\",
0x0D, "Extended Memory test running /",
0x0E, "Extended Memory test completed.",
0x40, "Display sign-on.",
0x41, "PCI configuration.",
0x50, "An issue was detected. System at boot F1/F2 prompt. Requires entry to
continue.",
0x51, "No bootable devices.",
0x52, "In BIOS Setup Menu.",
0x53, "In BIOS Boot Menu.",
0x54, "Automated Task application.",
0x55, "Performing CSIOR.",
0x56, "In Lifecycle Controller.",
0x57, "Initializing iDRAC.",
0x58, "Preparing to Boot.",
0x7D, "The system BIOS is about to start a boot option.",
0x7E, "Giving control to UEFI aware OS.",
0x7F, "Given control to OS.",
0x80, "No memory is detected.",
```

```
0x81, "Memory is detected but is not configurable.",
0x82, "Memory is configured but not usable.",
0x83, "System BIOS shadow failed.",
0x84, "CMOS failed.",
0x85, "DMA controller failed.",
0x86, "Interrupt controller failed.",
0x87, "Timer refresh failed.",
0x88, "Programmable interval timer error.",
0x89, "Parity error.",
0x8A, "SIO failed.",
0x8B, "Keyboard controller failed.",
0x8C, "System management interrupt initialization failed.",
0x8D, "QuickPath Interconnect (QPI) fatal error.",
0x8E, "MRC fatal error.",
0x8F, "Intel Trusted Execution Technology (TXT) fatal error.",
0x90, "Unable to load required BIOS files.",
0xC0, "Shutdown test failed.",
0xC1, "BIOS POST memorytest failed",
0xC2, "Remote access controller configuration failed.",
0xC3, "CPU configuration failed.",
0xC4, "Incorrect memory configuration.",
0xD0, "System BIOS has halted.",
0xD1, "System BIOS has halted due to Non-Maskable Interrupt (NMI).",
0xFE, "General failure after video."
```

21.1 Get Server Last System Boot Timestamp

Command: GET

URI:

```
redfish/v1/Managers/iDRAC.Embedded.1/Oem/Dell/DellAttributes/iDRAC.Embedded.1?$select=Attributes/SystemInfo.1.BootTime
```

Header: content-type application/json

Auth: Basic or X auth

Success status code returned: 200

21.2 Determine Linux OS Run-Level Prior to the Last Reboot

iDRAC does not know what run level the Linux OS is in. This would be logged within the OS logs.

21.3 Get iDRAC First Boot Up Time

Command: GET

URI:

redfish/v1/Managers/iDRAC.Embedded.1/Oem/Dell/DellAttributes/iDRAC.Embedded.1?\$select=Attributes/SupportAssist.1.iDRACFirstPowerUpDateTime

Header: content-type application/json

Auth: Basic or X auth

Success status code returned: 200

21.4 Get/Set iDRAC Date Time Local Offset

Command: GET

URI:

redfish/v1/Managers/iDRAC.Embedded.1/Oem/Dell/DellAttributes/iDRAC.Embedded.1?\$select=Attributes/Time.1.Timezone

Header: content-type application/json

Auth: Basic or X auth

Success status code returned: 200

Command: PATCH

URI:

redfish/v1/Managers/iDRAC.Embedded.1/Oem/Dell/DellAttributes/iDRAC.Embedded.1

Header: content-type application/json

Auth: Basic or X auth

```
Body: {
  "Attributes":{
    "Time.1.Timezone":"CST6CDT"
  }
}
```

NOTE: Currently iDRAC does not support setting property “DateTimeLocalOffset” for URI “redfish/v1/Managers/iDRAC.Embedded.1” to change the offset. You must change OEM time zone attribute stated above to change the offset value.

21.5 Get/Set iDRAC Date Time

Command: POST

URI:
redfish/v1/Managers/iDRAC.Embedded.1/Oem/Dell/DellTimeService/Actions/DellTimeService.ManageTime

Header: content-type application/json

Auth: Basic or X auth

Body: {
 "GetRequest":true
}

Success status code returned: 200

NOTE: To get current iDRAC time, perform POST command above.

Command: POST

URI:
redfish/v1/Managers/iDRAC.Embedded.1/Oem/Dell/DellTimeService/Actions/DellTimeService.ManageTime

Header: content-type application/json

Auth: Basic or X auth

Body: {
 "GetRequest":false,
 "TimeData":"<date time in UTC string format>"
}

Body Example: {
 "GetRequest":false,
 "TimeData":"2022-11-18T17:00:10-06:00"
}

Success status code returned: 200

NOTE: Perform POST command above to change iDRAC time.

NOTE: Once you set iDRAC time, if you reboot the server and BIOS time is not coordinated with iDRAC time, iDRAC time gets overridden by BIOS time and sync.

NOTE: If your OS is not coordinated with iDRAC and BIOS, both iDRAC and BIOS will synchronize with OS time.

The Redfish RESTful API describes many useful open system attributes that provide details about systems monitoring and management of your PowerEdge server. These attributes with Dell specific features, as described above, can increase your overall systems knowledge and efficiency

22 iDRAC Reporting

22.1 Job Queue Management

Get current iDRAC job queue.

Command: GET

URI: redfish/v1/Managers/iDRAC.Embedded.1/Oem/Dell/Jobs?\$expand=*\$levels=1)

Header: content-type application/json

Auth: Basic or X auth

Success status code returned: 200

- Delete one job ID

Command: POST

URI:

redfish/v1/Managers/iDRAC.Embedded.1/Oem/Dell/DellJobService/Actions/DellJobService.DeleteJobQueue

Header: content-type application/json

Auth: Basic or X auth

Body: {

 "JobID": "<job ID>"

}

Body Example: {

 "JobID": "JID_631039262553"

}

Success status code returned: 200

NOTE: You cannot delete a job in running state, only downloaded, scheduled, completed, or failed job can be deleted.

- Delete all job IDs in the job queue.

Command: POST

URI:
redfish/v1/Managers/iDRAC.Embedded.1/Oem/Dell/DellJobService/Actions/DellJobService.DeleteJobQueue

Header: content-type application/json

Auth: Basic or X auth

Body: {
 "JobID": "JID_CLEARALL"
}

Success status code returned: 200

NOTE: If a job is in running state, this command fails.

22.2 Export Server Hardware Inventory

To get complete server hardware inventory, run GET on the URI
"redfish/v1/Chassis/System.Embedded.1?\$expand=*\$levels=1" will return all hardware components but it will not expand all sub URIs for devices like storage or network. The following OEM action, POST call downloads the complete server hardware inventory in XML pretty format.

Command: POST

URI:
redfish/v1/Managers/iDRAC.Embedded.1/Oem/Dell/DellLCService/Actions/DellLCService.ExportHWInventory

Header: content-type application/json

Auth: Basic or X auth

Body: {
 "ShareType": "Local"
}

Success status code returned: 202

NOTE: In the header location, you will see URI returned /redfish/v1/Dell/hwinv.xml. Run GET on this URI to download hardware inventory file.

22.3 Export iDRAC Lifecycle Log

Command: POST

URI:

redfish/v1/Managers/iDRAC.Embedded.1/Oem/Dell/DellLCService/Actions/DellLCService.ExportLCLog

Header: content-type application/json

Auth: Basic or X auth

```
Body: {  
    "ShareType": "Local"  
}
```

Success status code returned: 202

NOTE: In the header location, you will see URI returned /redfish/v1/Dell/lclog.xml. Run GET on this URI to download LC log file.

22.4 SupportAssist Collection

NOTE: Before running SupportAssistCollection for the first time, you must accept End User License Agreement (EULA). Accepting EULA is a one-time operation and this acceptance gets stored in iDRAC. But if iDRAC reset to defaults, the license gets deleted and you must accept again. To accept EULA using Redfish, run POST on URI

redfish/v1/Managers/iDRAC.Embedded.1/Oem/Dell/DellLCService/Actions/DellLCService.SupportAssistAcceptEULA and pass an empty JSON payload.

Command: POST

URI:

redfish/v1/Managers/iDRAC.Embedded.1/Oem/Dell/DellLCService/Actions/DellLCService.SupportAssistCollection

Header: content-type application/json

Auth: Basic or X auth

```
Body: {  
    "ShareType": "Local",  
    "DataSelectorArrayIn": ["<pass in the data types you want to collect in the  
TSR report>"]  
}
```

Body Example: {
 "ShareType": "Local",
 "DataSelectorArrayIn": [

```
        "HWData",  
        "OSAppData",  
        "TTYLogs"  
    ]  
}
```

Success status code returned: 202

NOTE: Job ID URI will be returned in the header location. Run GET command on this URI.

Command: GET

URI: "Header location job ID URI"

Header: content-type application/json

Auth: Basic or X auth

NOTE: Loop checking the job status until marked completed. After completion, the header location will report URI /redfish/v1/Dell/sacollect.zip. Run GET on this URI to download SuperAssist collection file.

23 Capture server screenshot details

Capturing server screenshot details can be helpful to see the current server host status without the need, launch virtual console, or capture the last time the server OS crashed (for example: BSOD or kernel panic).

NOTE: Image output is in base64 string format. You need to convert base64 string to PNG file to view the image.

23.1 Get last crash screenshot

```
Command: POST
URI:
redfish/v1/Managers/iDRAC.Embedded.1/Oem/Dell/DellLCService/Actions/DellLCService.ExportServerScreenShot
Header: content-type application/json
Auth: Basic or X auth
Body: {"FileType":"LastCrashScreenShot"}
```

23.2 Preview server screenshot

```
Command: POST
URI:
redfish/v1/Managers/iDRAC.Embedded.1/Oem/Dell/DellLCService/Actions/DellLCService.ExportServerScreenShot
Header: content-type application/json
Auth: Basic or X auth
Body: {"FileType":"Preview"}
```

23.3 Get current server screenshot

```
Command: POST
URI:
redfish/v1/Managers/iDRAC.Embedded.1/Oem/Dell/DellLCService/Actions/DellLCService.ExportServerScreenShot
Header: content-type application/json
Auth: Basic or X auth
Body: {"FileType":"ServerScreenShot"}
```

24 iDRAC Firmware Operations

24.1 Get Server Current Firmware Inventory All Devices

Command: GET
URI: redfish/v1/UpdateService/FirmwareInventory?\$expand=*\$levels=1)
Header: content-type application/json
Auth: Basic or X auth

Success status code returned: 200

NOTE: URI \$expand may take a few seconds to return all the data depending on the number of nested firmware devices.

NOTE: On most of the server configurations, the \$expand may take longer duration than a few seconds to return data or may time out and return no data. It is recommended to run the GET command on the URI without \$expand to check the number of devices in the collection. If the devices are less than 50, use \$expand to get all data. Else, use scripting to cycle through each members' URI and run the GET command to return the data.

24.2 Get Server Device Sideband Firmware Update Support

Some devices in the server may support sideband firmware updates which means the firmware updates are deployed in real time without the need of a server reboot. Run the GET command to confirm if the device supports sideband firmware update.

NOTE: Some devices that support sideband firmware updates may still need to perform a server operation after the update job is completed. Confirm the final job status and LC logs to know if the server requires a power cycle reboot or a/c cycle.

Command: GET
URI: redfish/v1/UpdateService/FirmwareInventory/<installed device instance ID>?\$select=Oem/Dell/DellSoftwareInventory/SidebandUpdateCapable
URI example: redfish/v1/UpdateService/FirmwareInventory/Installed-104933-15.35.06__NIC.Slot.2-1-1?\$select=Oem/Dell/DellSoftwareInventory/SidebandUpdateCapable
Header: content-type application/json

Auth: Basic or X auth

Success status code returned: 200

NOTE: This property is a Boolean. If the returns true then sideband update is supported for the device.

24.3 Single Device Firmware Update (Immediate)

NOTE: This workflow shows updating firmware for a device that gets applied immediately. Device examples are iDRAC, DIAGs, and DriverPack.

For more details about what devices get firmware applied immediately, see the iDRAC User's Guide Update Section.

Command: POST

```
URI: redfish/v1/UpdateService/MultipartUpload
Header/File: {"UpdateParameters": (None, json.dumps({"Targets": [],
"@Redfish.OperationApplyTime": "Immediate", "Oem": {}},
"application/json"), "UpdateFile": ("<name of firmware image>"),
open("<complete path of directory location and firmware image name>",
"rb"), "application/octet-stream")}
```

```
Header/File Python Example: {"UpdateParameters": (None,
json.dumps({"Targets": [], "@Redfish.OperationApplyTime": "Immediate",
"Oem": {}}, "application/json"), "UpdateFile": ("iDRAC-with-Lifecycle-
Controller_Firmware_JG4FK_WN64_6.00.02.00_A00.EXE"),
open("C:\\Users\\administrator\\Downloads\\iDRAC-with-Lifecycle-
Controller_Firmware_JG4FK_WN64_6.00.02.00_A00.EXE", "rb"),
"application/octet-stream")}
```

Auth: Basic or X auth

Success status code returned: 202

NOTE: See GitHub script for code example about how to pass in reading the firmware file.

<https://github.com/dell/iDRAC-Redfish-Scripting/blob/master/Redfish%20Python/DeviceFirmwareMultipartUploadREDFISH.py>

NOTE: Windows DUP is only supported to update a device using iDRAC interfaces.

NOTE: Job ID URI will be returned in the header location. Run the GET command to validate **JobState** marked as **Completed**.

Command: GET

URI: redfish/v1/Managers/iDRAC.Embedded.1/Oem/Dell/Jobs/{job ID}

Header: content-type application/json

Auth: Basic or X auth

Success status code returned: 200

24.4 Single Device Firmware Update (OnReset)

NOTE: This workflow shows updating firmware for a device that gets applied on reset or server reboot. Device examples: BIOS, Storage Controller, Network Cards. For more details about what devices get firmware applied on reset, see the iDRAC User's Guide Update Section.

Command: POST

URI: redfish/v1/UpdateService/MultipartUpload

Header/File: {"UpdateParameters": (None, json.dumps({"Targets": [],
"@Redfish.OperationApplyTime": "OnReset", "Oem": {}},
"application/json"), "UpdateFile": ("<name of firmware image>", open("<complete
path of directory location and firmware image name>", "rb"), "application/octet-
stream") }

Header/File Python Example: {"UpdateParameters": (None, json.dumps({"Targets":
[], "@Redfish.OperationApplyTime": "OnReset", "Oem": {}},
"application/json"), "UpdateFile": ("BIOS_DHRG5_WN64_1.7.5.EXE"),
open("C:\\Users\\administrator\\Downloads\\BIOS_DHRG5_WN64_1.7.5.EXE", "rb"),
"application/octet-stream") }

Auth: Basic or X auth

Success status code returned: 202

NOTE: See GitHub script for code example about how to pass in reading the firmware file.

<https://github.com/dell/iDRAC-Redfish-Scripting/blob/master/Redfish%20Python/DeviceFirmwareMultipartUploadREDFISH.py>

NOTE: Windows DUP is only supported to update a device using iDRAC interfaces.

NOTE: Job ID URI will be returned in the headers location. Run GET command below on this URI to validate JobState marked Scheduled.

Command: GET

URI: redfish/v1/Managers/iDRAC.Embedded.1/Oem/Dell/Jobs/{job ID}

Header: content-type application/json

Auth: Basic or X auth

Success status code returned: 200

NOTE: Once the job ID is marked scheduled, reboot the server so the update job is performed, and firmware update is applied.

Command: POST

URI: redfish/v1/Systems/System.Embedded.1/Actions/ComputerSystem.Reset

Header: content-type application/json

Auth: Basic or X auth

Body: {
 "ResetType": "GracefulRestart"
}

Success status code returned: 204

NOTE: Once the server reboots, loop checking the job status until JobState marked Completed.

24.5 Rollback Device Firmware

Command: GET

URI: redfish/v1/UpdateService/FirmwareInventory

Header: content-type application/json

Auth: Basic or X auth

Success status code returned: 200

NOTE: In the JSON output Members, look for any URI with "Previous" string, this means that this device supports rollback.

Example:

/redfish/v1/UpdateService/FirmwareInventory/Previous-25227-3.32.32.32__iDRAC.Embedded.1-1

NOTE: Take this URI and pass it in the body for ImageURI value.

Command: POST

URI: redfish/v1/UpdateService/Actions/UpdateService.SimpleUpdate

Header: content-type application/json

Auth: Basic or X auth

Body: {"ImageURI": "<previous device URI>"}

Body Example: {
 "ImageURI": "/redfish/v1/UpdateService/FirmwareInventory/Previous-25227-
3.32.32.32__iDRAC.Embedded.1-1"
}

Success status code returned: 202

NOTE: Job ID URI is returned in the headers location.

Run GET command below on this URI to validate JobState marked as Scheduled or Completed.

If marked Scheduled, you must reboot the server to appl the rollback firmware version.

Command: GET

URI: redfish/v1/Managers/iDRAC.Embedded.1/Oem/Dell/Jobs/{job ID}

Header: content-type application/json

Auth: Basic or X auth

Success status code returned: 200

24.6 Repository Firmware Updates

Workflow example below will be leveraging Dell downloads repository which is recommended to use to get the latest firmware versions for all devices iDRAC supports for updates. You can also create and use your own custom repository using Dell Repository Manager (DRM) utility.

First check the repository for any updates that are available for the server. Setting ApplyUpdate and RebootNeeded to False will achieve this.

Command: POST

URI:

redfish/v1/Systems/System.Embedded.1/Oem/Dell/DellSoftwareInstallationService/Actions/DellSoftwareInstallationService.InstallFromRepository

Header: content-type application/json

Auth: Basic or X auth

Body: {

 "ApplyUpdate":"False",

 "RebootNeeded":false,

 "IPAddress":"downloads.dell.com",

 "ShareType":"HTTPS",

 "ShareName":"catalog"

}

Success status code returned: 202

NOTE: ApplyUpdate is a True or False string value. RebootNeeded is a True or False boolean value.

NOTE: Repository job ID URI is returned in the headers location. Query this job ID until marked completed.

Command: GET

URI: redfish/v1/Managers/iDRAC.Embedded.1/Oem/Dell/Jobs/{job ID}

Header: content-type application/json

Auth: Basic or X auth

Success status code returned: 200

Next step is to get repository update list. This returns what devices can be updated from the repository and what firmware version is available.

Command: POST

URI:

redfish/v1/Systems/System.Embedded.1/Oem/Dell/DellSoftwareInstallationService/Actions/DellSoftwareInstallationService.GetRepoBasedUpdateList

Header: content-type application/json

Auth: Basic or X auth

Body: {}

NOTE: Pass in empty JSON body.

Success status code returned: 200

Example output below which will be in XML format in the JSON.

```
<?xml version="1.0"?>
<CIM xmlns:fo="http://www.w3.org/1999/XSL/Format" CIMVERSION="2.0"
DTDVERSION="2.0">
  <MESSAGE ID="4711" PROTOCOLVERSION="1.0">
    <SIMPLEREQ>
      <VALUE.NAMEDINSTANCE>
        <INSTANCENAME CLASSNAME="DCIM_RepoUpdateSWID"><PROPERTY
NAME="Criticality" TYPE="string"><VALUE>1</VALUE></PROPERTY><PROPERTY
NAME="DisplayName" TYPE="string"><VALUE>Intel(R) Gigabit I350-t LOM -
80:18:44:7A:13:C5</VALUE></PROPERTY><PROPERTY NAME="BaseLocation"
TYPE="string"><VALUE/></PROPERTY><PROPERTY NAME="PackagePath"
TYPE="string"><VALUE>FOLDER08639282M/1/Network_Firmware_3XVMN_WN64_20.5.15_A00.E
XE</VALUE></PROPERTY><PROPERTY NAME="PackageName"
TYPE="string"><VALUE>Network_Firmware_3XVMN_WN64_20.5.15_A00.EXE</VALUE></PROPER
TY><PROPERTY NAME="PackageVersion"
TYPE="string"><VALUE>20.5.15</VALUE></PROPERTY><PROPERTY NAME="RebootType"
TYPE="string"><VALUE>HOST</VALUE></PROPERTY><PROPERTY NAME="JobID"
TYPE="string"><VALUE/></PROPERTY>
        <PROPERTY NAME="Target"
TYPE="string"><VALUE>DCIM:INSTALLED#701__NIC.Embedded.1-1-
1</VALUE></PROPERTY><PROPERTY NAME="ComponentID"
TYPE="string"><VALUE>105516</VALUE></PROPERTY><PROPERTY NAME="ComponentType"
TYPE="string"><VALUE>FRMW</VALUE></PROPERTY><PROPERTY.ARRAY
NAME="ComponentInfoValue"
TYPE="string"><VALUE.ARRAY><VALUE>8086:1521:1028:0757</VALUE></VALUE.ARRAY></PRO
```

```

    Perty.ARRAY><PROPERTY.ARRAY NAME="ComponentInfoName"
    TYPE="string"><VALUE.ARRAY><VALUE>VendorID:DeviceID:SubVendorID:SubDeviceID</VAL
    UE></VALUE.ARRAY></PROPERTY.ARRAY><PROPERTY.ARRAY NAME="ComponentInfoTarget"
    TYPE="string"><VALUE.ARRAY><VALUE>DCIM:INSTALLED#701__NIC.Embedded.1-1-
    1</VALUE></VALUE.ARRAY></PROPERTY.ARRAY><PROPERTY.ARRAY
    NAME="ComponentInstalledVersion"
    TYPE="string"><VALUE.ARRAY><VALUE>20.5.13</VALUE></VALUE.ARRAY></PROPERTY.ARRAY>

    </INSTANCENAME>
    </VALUE.NAMEDINSTANCE>
    <VALUE.NAMEDINSTANCE>
        <INSTANCENAME CLASSNAME="DCIM_RepoUpdateSWID"><PROPERTY
        NAME="Criticality" TYPE="string"><VALUE>1</VALUE></PROPERTY><PROPERTY
        NAME="DisplayName" TYPE="string"><VALUE>iDRAC Service Module
        Installer</VALUE></PROPERTY><PROPERTY NAME="BaseLocation"
        TYPE="string"><VALUE/></PROPERTY><PROPERTY NAME="PackagePath"
        TYPE="string"><VALUE>FOLDER08698095M/1/Systems-
        Management_Application_X2W1R_WN64_4.3.0.0_A00.EXE</VALUE></PROPERTY><PROPERTY
        NAME="PackageName" TYPE="string"><VALUE>Systems-
        Management_Application_X2W1R_WN64_4.3.0.0_A00.EXE</VALUE></PROPERTY><PROPERTY
        NAME="PackageVersion" TYPE="string"><VALUE>4.3.0.0</VALUE></PROPERTY><PROPERTY
        NAME="RebootType" TYPE="string"><VALUE>NONE</VALUE></PROPERTY><PROPERTY
        NAME="JobID" TYPE="string"><VALUE/></PROPERTY>
        <PROPERTY NAME="Target"
        TYPE="string"><VALUE>DCIM:INSTALLED#802__ServiceModule.Embedded.1</VALUE></PROPE
        RTY><PROPERTY NAME="ComponentID"
        TYPE="string"><VALUE>104684</VALUE></PROPERTY><PROPERTY NAME="ComponentType"
        TYPE="string"><VALUE>APAC</VALUE></PROPERTY><PROPERTY.ARRAY
        NAME="ComponentInfoValue"
        TYPE="string"><VALUE.ARRAY/></PROPERTY.ARRAY><PROPERTY.ARRAY
        NAME="ComponentInfoName"
        TYPE="string"><VALUE.ARRAY/></PROPERTY.ARRAY><PROPERTY.ARRAY
        NAME="ComponentInfoTarget"
        TYPE="string"><VALUE.ARRAY><VALUE>DCIM:INSTALLED#802__ServiceModule.Embedded.1</
        VALUE></VALUE.ARRAY></PROPERTY.ARRAY><PROPERTY.ARRAY
        NAME="ComponentInstalledVersion"
        TYPE="string"><VALUE.ARRAY><VALUE>0</VALUE></VALUE.ARRAY></PROPERTY.ARRAY>

    </INSTANCENAME>
    </VALUE.NAMEDINSTANCE>
    <VALUE.NAMEDINSTANCE>
        <INSTANCENAME CLASSNAME="DCIM_RepoUpdateSWID"><PROPERTY
        NAME="Criticality" TYPE="string"><VALUE>1</VALUE></PROPERTY><PROPERTY
        NAME="DisplayName" TYPE="string"><VALUE>OS Collector</VALUE></PROPERTY><PROPERTY
        NAME="BaseLocation" TYPE="string"><VALUE/></PROPERTY><PROPERTY
        NAME="PackagePath"
        TYPE="string"><VALUE>FOLDER06878175M/2/Diagnostics_Application_0G1JH_WN64_6.0_A0
        0_01.EXE</VALUE></PROPERTY><PROPERTY NAME="PackageName"
        TYPE="string"><VALUE>Diagnostics_Application_0G1JH_WN64_6.0_A00_01.EXE</VALUE></
        PROPERTY><PROPERTY NAME="PackageVersion"

```

```

TYPE="string"><VALUE>6.0</VALUE></PROPERTY><PROPERTY NAME="RebootType"
TYPE="string"><VALUE>NONE</VALUE></PROPERTY><PROPERTY NAME="JobID"
TYPE="string"><VALUE/></PROPERTY>
    <PROPERTY NAME="Target"
TYPE="string"><VALUE>DCIM:INSTALLED#802__OSCollector.Embedded.1</VALUE></PROPERT
Y><PROPERTY NAME="ComponentID"
TYPE="string"><VALUE>101734</VALUE></PROPERTY><PROPERTY NAME="ComponentType"
TYPE="string"><VALUE>APAC</VALUE></PROPERTY><PROPERTY.ARRAY
NAME="ComponentInfoValue"
TYPE="string"><VALUE.ARRAY/></PROPERTY.ARRAY><PROPERTY.ARRAY
NAME="ComponentInfoName"
TYPE="string"><VALUE.ARRAY/></PROPERTY.ARRAY><PROPERTY.ARRAY
NAME="ComponentInfoTarget"
TYPE="string"><VALUE.ARRAY><VALUE>DCIM:INSTALLED#802__OSCollector.Embedded.1</VA
LUE></VALUE.ARRAY></PROPERTY.ARRAY><PROPERTY.ARRAY
NAME="ComponentInstalledVersion"
TYPE="string"><VALUE.ARRAY><VALUE>0</VALUE></VALUE.ARRAY></PROPERTY.ARRAY>

</INSTANCENAME>
    </VALUE.NAMEDINSTANCE>
    <VALUE.NAMEDINSTANCE>
        <INSTANCENAME CLASSNAME="DCIM_RepoUpdatesSWID"><PROPERTY
NAME="Criticality" TYPE="string"><VALUE>3</VALUE></PROPERTY><PROPERTY
NAME="DisplayName" TYPE="string"><VALUE>OS Drivers
Pack</VALUE></PROPERTY><PROPERTY NAME="BaseLocation"
TYPE="string"><VALUE/></PROPERTY><PROPERTY NAME="PackagePath"
TYPE="string"><VALUE>FOLDER08823173M/1/Drivers-for-OS-
Deployment_Application_1K4PP_WN64_22.07.08_A00.EXE</VALUE></PROPERTY><PROPERTY
NAME="PackageName" TYPE="string"><VALUE>Drivers-for-OS-
Deployment_Application_1K4PP_WN64_22.07.08_A00.EXE</VALUE></PROPERTY><PROPERTY
NAME="PackageVersion" TYPE="string"><VALUE>22.07.08</VALUE></PROPERTY><PROPERTY
NAME="RebootType" TYPE="string"><VALUE>NONE</VALUE></PROPERTY><PROPERTY
NAME="JobID" TYPE="string"><VALUE/></PROPERTY>
        <PROPERTY NAME="Target"
TYPE="string"><VALUE>DCIM:INSTALLED#802__DriverPack.Embedded.1:LC.Embedded.1</VA
LUE></PROPERTY><PROPERTY NAME="ComponentID"
TYPE="string"><VALUE>18981</VALUE></PROPERTY><PROPERTY NAME="ComponentType"
TYPE="string"><VALUE>APAC</VALUE></PROPERTY><PROPERTY.ARRAY
NAME="ComponentInfoValue"
TYPE="string"><VALUE.ARRAY/></PROPERTY.ARRAY><PROPERTY.ARRAY
NAME="ComponentInfoName"
TYPE="string"><VALUE.ARRAY/></PROPERTY.ARRAY><PROPERTY.ARRAY
NAME="ComponentInfoTarget"
TYPE="string"><VALUE.ARRAY><VALUE>DCIM:INSTALLED#802__DriverPack.Embedded.1:LC.E
mbedded.1</VALUE></VALUE.ARRAY></PROPERTY.ARRAY><PROPERTY.ARRAY
NAME="ComponentInstalledVersion"
TYPE="string"><VALUE.ARRAY><VALUE>0</VALUE></VALUE.ARRAY></PROPERTY.ARRAY>

    </INSTANCENAME>
    </VALUE.NAMEDINSTANCE>

```

```

    <VALUE.NAMEDINSTANCE>
      <INSTANCENAME CLASSNAME="DCIM_RepoUpdatesWID"><PROPERTY
NAME="Criticality" TYPE="string"><VALUE>3</VALUE></PROPERTY><PROPERTY
NAME="DisplayName" TYPE="string"><VALUE>Diagnostics</VALUE></PROPERTY><PROPERTY
NAME="BaseLocation" TYPE="string"><VALUE/></PROPERTY><PROPERTY
NAME="PackagePath"
TYPE="string"><VALUE>FOLDER08224025M/1/Diagnostics_Application_PX25H_WN64_4301A7
0_4301.71.EXE</VALUE></PROPERTY><PROPERTY NAME="PackageName"
TYPE="string"><VALUE>Diagnostics_Application_PX25H_WN64_4301A70_4301.71.EXE</VAL
UE></PROPERTY><PROPERTY NAME="PackageVersion"
TYPE="string"><VALUE>4301A70</VALUE></PROPERTY><PROPERTY NAME="RebootType"
TYPE="string"><VALUE>NONE</VALUE></PROPERTY><PROPERTY NAME="JobID"
TYPE="string"><VALUE/></PROPERTY>
      <PROPERTY NAME="Target"
TYPE="string"><VALUE>DCIM:INSTALLED#802__Diagnostics.Embedded.1:LC.Embedded.1</V
ALUE></PROPERTY><PROPERTY NAME="ComponentID"
TYPE="string"><VALUE>25806</VALUE></PROPERTY><PROPERTY NAME="ComponentType"
TYPE="string"><VALUE>APAC</VALUE></PROPERTY><PROPERTY.ARRAY
NAME="ComponentInfoValue"
TYPE="string"><VALUE.ARRAY/></PROPERTY.ARRAY><PROPERTY.ARRAY
NAME="ComponentInfoName"
TYPE="string"><VALUE.ARRAY/></PROPERTY.ARRAY><PROPERTY.ARRAY
NAME="ComponentInfoTarget"
TYPE="string"><VALUE.ARRAY><VALUE>DCIM:INSTALLED#802__Diagnostics.Embedded.1:LC.
Embedded.1</VALUE></VALUE.ARRAY></PROPERTY.ARRAY><PROPERTY.ARRAY
NAME="ComponentInstalledVersion"
TYPE="string"><VALUE.ARRAY><VALUE>0</VALUE></VALUE.ARRAY></PROPERTY.ARRAY>

    </INSTANCENAME>
  </VALUE.NAMEDINSTANCE>
  <VALUE.NAMEDINSTANCE>
    <INSTANCENAME CLASSNAME="DCIM_RepoUpdatesWID"><PROPERTY
NAME="Criticality" TYPE="string"><VALUE>1</VALUE></PROPERTY><PROPERTY
NAME="DisplayName" TYPE="string"><VALUE>Integrated Remote Access
Controller</VALUE></PROPERTY><PROPERTY NAME="BaseLocation"
TYPE="string"><VALUE/></PROPERTY><PROPERTY NAME="PackagePath"
TYPE="string"><VALUE>FOLDER08759624M/1/iDRAC-with-Lifecycle-
Controller_Firmware_RTTPH_WN64_5.10.50.00_A00.EXE</VALUE></PROPERTY><PROPERTY
NAME="PackageName" TYPE="string"><VALUE>iDRAC-with-Lifecycle-
Controller_Firmware_RTTPH_WN64_5.10.50.00_A00.EXE</VALUE></PROPERTY><PROPERTY
NAME="PackageVersion"
TYPE="string"><VALUE>5.10.50.00</VALUE></PROPERTY><PROPERTY NAME="RebootType"
TYPE="string"><VALUE>iDRAC</VALUE></PROPERTY><PROPERTY NAME="JobID"
TYPE="string"><VALUE/></PROPERTY>
    <PROPERTY NAME="Target"
TYPE="string"><VALUE>DCIM:INSTALLED#iDRAC.Embedded.1-
1#iDRACinfo</VALUE></PROPERTY><PROPERTY NAME="ComponentID"
TYPE="string"><VALUE>25227</VALUE></PROPERTY><PROPERTY NAME="ComponentType"
TYPE="string"><VALUE>FRMW</VALUE></PROPERTY><PROPERTY.ARRAY
NAME="ComponentInfoValue"

```

```

TYPE="string"><VALUE.ARRAY/></PROPERTY.ARRAY><PROPERTY.ARRAY
NAME="ComponentInfoName"
TYPE="string"><VALUE.ARRAY/></PROPERTY.ARRAY><PROPERTY.ARRAY
NAME="ComponentInfoTarget"
TYPE="string"><VALUE.ARRAY><VALUE>DCIM:INSTALLED#iDRAC.Embedded.1-
1#iDRACinfo</VALUE></VALUE.ARRAY></PROPERTY.ARRAY><PROPERTY.ARRAY
NAME="ComponentInstalledVersion"
TYPE="string"><VALUE.ARRAY><VALUE>6.00.02.00</VALUE></VALUE.ARRAY></PROPERTY.ARR
AY>

    </INSTANCENAME>
    </VALUE.NAMEDINSTANCE>
    </SIMPLEREQ>
  </MESSAGE>
</CIM>

```

After you confirm to update these devices, rerun the POST command for action `InstallFromRepository` but this time set **ApplyUpdate** and **RebootNeeded** to **True**. A repository job gets created and, in the background an update job for each device will also get created. Monitor the repository update job and once it is marked completed, monitor the overall job queue for completion of each update job.

Now each update job starts to get applied in a certain order. First devices that get applied immediately (DIAGS, DriverPack), these jobs will run and complete since no server reboot is needed. Next, any update job that needs a server reboot to apply (BIOS, PERC, NIC), server will now automatically reboot and apply the firmware update. Lastly if iDRAC update is detected, it will get applied last in the order due to iDRAC reboots after the update is complete.

```

Command: POST
URI:
redfish/v1/Systems/System.Embedded.1/Oem/Dell/DellSoftwareInstallationService/Actions/DellSoftwareInstallationService.InstallFromRepository
Header: content-type application/json
Auth: Basic or X auth
Body: {
  "ApplyUpdate": "True",
  "RebootNeeded": true,
  "IPAddress": "downloads.dell.com",
  "ShareType": "HTTPS",
  "ShareName": "catalog"
}

```

Success status code returned: 202

NOTE: Repository job ID URI is returned in the headers location. Query this job ID until marked completed.

NOTE: In the background, update job will get created for each individual device that detects an update. Monitor the overall job queue to query the job status for these jobs.

Monitor a single job:

Command: GET

URI: redfish/v1/Managers/iDRAC.Embedded.1/Oem/Dell/Jobs/{job ID}

Header: content-type application/json

Auth: Basic or X auth

Success status code returned: 200

Monitor the complete job queue:

Command: GET

URI: redfish/v1/Managers/iDRAC.Embedded.1/Oem/Dell/Jobs?\$expand=*\$levels=1)

Header: content-type application/json

Auth: Basic or X auth

Success status code returned: 200

25 iDRAC Sessions/Webserver Management

25.1 Get all active sessions

Command: GET

URI: redfish/v1/SessionService/Sessions?\$expand=*\$levels=1

Header: content-type application/json

Auth: Basic or X auth

Success status code returned: 200

25.2 Delete one active session

Command: DELETE

URI: redfish/v1/SessionService/Sessions/{session ID}

Header: content-type application/json

Auth: Basic or X auth

Success status code returned: 200

25.3 Get iDRAC webserver session timeout setting

Command: GET

URI:
redfish/v1/Managers/iDRAC.Embedded.1/Oem/Dell/DellAttributes/iDRAC.Embedded.1?\$select=Attributes/WebServer.1.Timeout

Header: content-type application/json

Auth: Basic or X auth

Success status code returned: 200

NOTE: Default value is 1800 seconds.

25.4 Set iDRAC webserver session timeout setting

Command: PATCH

URI:

redfish/v1/Managers/iDRAC.Embedded.1/Oem/Dell/DellAttributes/iDRAC.Embedded.1

Header: content-type application/json

Auth: Basic or X auth

```
Body: {  
  "Attributes":{  
    "WebServer.1.Timeout":<pass in integer value>  
  }  
}
```

NOTE: Value that is passed is in seconds.

25.5 iDRAC maximum simultaneous sessions

The maximum simultaneous connections (Redfish + GUI) are 8.

This is a recommended number based upon stress testing. Beyond this number, the system is likely to get sluggish and times out on requests.

NOTE: This is not a DMTF specification.

If a ninth request comes in while 8 sessions are already active, you will see an error indicated by HTTP status code 503 with an extended info message of "RAC0218".

25.6 Simultaneous console sessions maximum

We recommend consoles to use not more than four simultaneous sessions for optimum performance.

26 iDRAC User SSH Key Management

26.1 Get current iDRAC user SSH key strings

NOTE: You must leverage iDRAC Server Configuration Profile (SCP) feature to get user SSH keys. This one POST action will return all SSH keys detected for all iDRAC users.

Command: POST

iDRAC9 URI:

redfish/v1/Managers/iDRAC.Embedded.1/Actions/Oem/EID_674_Manager.ExportSystemConfiguration

iDRAC10 URI:

redfish/v1/Managers/iDRAC.Embedded.1/Actions/Oem/OemManager.ExportSystemConfiguration

Header: content-type application/json

Auth: Basic or X token

Body: {"ExportFormat": "<pass in JSON or XML>", "ShareParameters": {"Target": ["iDRAC"], "IPAddress": "<network share IP>", "ShareType": "<pass in share type value, examples, NFS, or HTTP>", "ShareName": "<network share name>", "FileName": "<pass in unique filename>"}}

NOTE: In the Headers Location, job ID URI is returned. Run GET on this URI to check the job status for import operation.

Success status code returned: 200

Snippet example from SCP file showing user 2 SSH string key:

```
<SystemConfiguration Model="R640" ServiceTag="W516C1S" TimeStamp="Mon Nov 28 21:59:22 2022">
```

```
<!--Export type is Normal,XML,Selective-->
```

```
<!--Exported configuration may contain commented attributes. Attributes may be commented due to dependency, destructive nature, preserving server identity or for security reasons.-->
```

```
<Component FQDD="iDRAC.Embedded.1">
```

```
<Attribute Name="IPMILan.1#Enable">Disabled</Attribute>
```

```
<Attribute Name="IPMILan.1#PrivLimit">Administrator</Attribute>
```

```
<Attribute  
Name="IPMILan.1#EncryptionKey">0000000000000000000000000000000000000000000000000000000000000000</Attribu  
te>  
  
<Attribute Name="IPMILan.1#CommunityName">public</Attribute>  
  
<Attribute Name="IPMILan.1#AlertEnable">Disabled</Attribute>  
  
<Attribute Name="Users.2#UserName">root</Attribute>  
  
<!-- <Attribute Name="Users.2#Password">*****</Attribute>-->  
  
<Attribute Name="Users.2#Privilege">511</Attribute>  
  
<Attribute Name="Users.2#IpmiLanPrivilege">Administrator</Attribute>  
  
<Attribute Name="Users.2#Enable">Enabled</Attribute>  
  
<Attribute Name="Users.2#SolEnable">Enabled</Attribute>  
  
<Attribute Name="Users.2#ProtocolEnable">Disabled</Attribute>  
  
<Attribute Name="Users.2#AuthenticationProtocol">SHA</Attribute>  
  
<Attribute Name="Users.2#PrivacyProtocol">AES</Attribute>  
  
<!-- <Attribute Name="Users.2#SSHPublicKey1">ssh-rsa  
AAAAB3NzaClyc2EAAAADAQABAAQGDnZUYFHZ+mYB6dCjKtqoMw9+udigTWRPpY60X8wOE1BevTxDRM  
+foB19wewW2V4AGcznbKrUU3Vd66Mm4X23U4Zj7lxyfSiqOfx9C5eIGVqWfHtuW9MhDXIomLnUP9DMAO  
ypOBWEExwkNPSa9q/SAQLiBg1X3FLLWQtOWTwGA0sXhIz+DDnOs9fMsOYY24nsGYOwfzaDcVPaQks2N8h  
SHVUu+0N52S+awCDLAweFmYlL3QZ9ei+IYaV+W9K9ciVLLCyE7yYNdVamzcKtfSxt+tRdc7wXHGTiVKL  
ufVEHiIW+tXAe/qk/aCb+noUwFABWOXvfVl8BPE8Da7ayOaz4hSDkmv2BuQP1+VinCE/9/sn4pNI9g0p  
F31CMWz+UePbdsCt+D8HdvY9YhQRtU6yMc+Ll0IEQD5uY4j3G/k9g8o8NvEm9xb3YuSq3uYZYBlJooXon  
WPfoq6PX5OpwsITBmCeiiIoAMXu42uKlWLCd5GdW/lxwNFB833abcBDpXs=  
root@SCPexport.example.com</Attribute>-->  
  
<!-- <Attribute Name="Users.2#SSHPublicKey2"></Attribute>-->  
  
<!-- <Attribute Name="Users.2#SSHPublicKey3"></Attribute>-->  
  
<!-- <Attribute Name="Users.2#SSHPublicKey4"></Attribute>-->  
  
<Attribute Name="Users.2#Simple2FA">Disabled</Attribute>  
  
<Attribute Name="Users.2#UseEmail">Disabled</Attribute>  
  
<Attribute Name="Users.2#EmailAddress"/>  
  
<Attribute Name="Users.2#UseSMS">Disabled</Attribute>  
  
<Attribute Name="Users.2#SMSNumber"/>  
  
<Attribute Name="Users.2#RSA SecurID2FA">Disabled</Attribute>
```

26.2 Set iDRAC user SSH key

NOTE: You must leverage iDRAC Server Configuration Profile (SCP) feature to set user SSH keys. You can set SSH keys for 1 user or multiple users using one POST call.

Command: POST

iDRAC9 URI:
redfish/v1/Managers/iDRAC.Embedded.1/Actions/Oem/EID_674_Manager.ExportSystemConfiguration

iDRAC10 URI:
redfish/v1/Managers/iDRAC.Embedded.1/Actions/Oem/OemManager.ExportSystemConfiguration

Header: content-type application/json

Auth: Basic or X token

Body: {"ShareParameters": {"Target": ["iDRAC"], "IPAddress": "<network share IP>", "ShareType": "<pass in share type value, examples, NFS, or HTTP>", "ShareName": "<network share name>", "FileName": "<SCP file name>"}}

NOTE: In the Headers Location, job ID URI is returned. Run GET on this URI to check the job status for import operation.

Success status code returned: 200

27 Monitor Health State for Server Hardware (RollupStatus)

27.1 Get current rollup status for hardware devices

Command: GET

URI: redfish/v1/Systems/System.Embedded.1/Oem/Dell/DellRollupStatus

Header: content-type application/json

Auth: Basic or X auth

Success status code returned: 200

Polling rollup status for hardware devices in the server will alert you if any hardware issues have been detected. Recommended to use query parameter and filter when RollupStatus does not report a value of Ok. By doing this only devices which currently have an issue will be reported in the JSON output.

Example using query parameter URI:

URI:
redfish/v1/Systems/System.Embedded.1/Oem/Dell/DellRollupStatus?\$filter=RollupStatus ne 'Ok'

Example of PSU rollup status error reported:

```
@odata.context "/redfish/v1/$metadata#DellRollupStatus.DellRollupStatus"

@odata.id
  "/redfish/v1/Systems/System.Embedded.1/Oem/Dell/DellRollupStatus/iDRAC.Embedded.1_0x23_SubSystem.1_0x23_PowerSupply"

@odata.type      "#DellRollupStatus.v1_0_0.DellRollupStatus"

CollectionName   "PowerSupplyRollupStatus"

Description      "Represents the subcomponent roll-up statuses."

Id               "iDRAC.Embedded.1_0x23_SubSystem.1_0x23_PowerSupply"

InstanceID       "iDRAC.Embedded.1#SubSystem.1#PowerSupply"

Name              "DellRollupStatus"

RollupStatus     "Error"

SubSystem        "PowerSupply"

@odata.context   "/redfish/v1/$metadata#DellRollupStatus.DellRollupStatus"
```

27.2 Get additional health information for the devices which support rollup health data

Command: GET

URI: redfish/v1/Managers/iDRAC.Embedded.1/LogServices/FaultList/Entries

Header: content-type application/json

Auth: Basic or X auth

Success status code returned: 200

Get on this fault list will return additional information for the device(s) which report a rollup status error. If no rollup status errors are detected, GET on this URI will return an empty members collection.

Example of additional details for PSU error:

```
@odata.id
  "/redfish/v1/Managers/iDRAC.Embedded.1/LogServices/FaultList/Entries/Fault_03
200002_1"

@odata.type      "#LogEntry.v1_12_0.LogEntry"

Created   "2023-03-24T09:42:22-05:00"

Description      "PSU.Slot.2"

EntryType        "SEL"

Id "Fault_03200002_1"

Links

OriginOfCondition

@odata.id        "/redfish/v1/Chassis/System.Embedded.1/Power#/PowerSupplies/1"

Message  "The Power Supply Unit (PSU) 2 firmware is not responding."

MessageArgs

0  "2"

MessageArgs@odata.count  1

MessageId        "IDRAC.2.8.PSU0915"
```

```
Name      "PSU.Slot.2"
Oem
Dell
@odata.type "#DellLogEntry.v1_0_1.DellLogEntry"
FQDD      "PSU.Slot.2"
Id "Fault_03200002_1"
Name      "PSU.Slot.2"
SubSystem  "PowerSupply"
SensorNumber 0
SensorType "Power Supply / Converter"
Severity "Critical"
```

28 Repurpose/Retire The Server Using SystemErase

SystemErase feature will allow you to repurpose / retire the server by performing multiple actions to put the server back into a new state (from the factory).

Actions SystemErase can perform:

- Delete DIAGs or DriverPack
- Reset iDRAC to Default Settings
- Reset BIOS to Default Settings
- Clear iDRAC Lifecycle Logs and rollback firmware images stored in iDRAC
- Erase NVDIMMs
- Clear PERC controller pinned cache
- Crypto erase ISE drives
- Overwrite PD erase non ISE drives
- Erase DPUs
- Reinstall firmware packages which are stored in iDRAC

28.1 Get supported SystemErase Components

Command: GET

URI: redfish/v1/Managers/iDRAC.Embedded.1/Oem/Dell/DellLCService?\$select=Actions

Header: content-type application/json

Auth: Basic or X auth

Success status code returned: 200

NOTE: In the JSON output look for array Component@Redfish.AllowableValues which will list the supported component values. These string values will be required to pass in to run the POST action.

28.2 Perform SystemErase Action

Command: POST

URI:
redfish/v1/Managers/iDRAC.Embedded.1/Oem/Dell/DellLCService/Actions/DellLCService.SystemErase

Header: content-type application/json

Auth: Basic or X auth

Body: {"Component": [<pass in component(s) to erase>]}

Body Example: {"Component": ["iDRAC", "CryptographicErasePD"]}

Success status code returned: 202

NOTE: For Component array you can pass in one or more multiple values.

JOB ID URI is returned in JSON response headers location. Run GET command on this URI to monitor the job status until marked Completed.

Command: GET

URI: redfish/v1/Managers/iDRAC.Embedded.1/Oem/Dell/Jobs/{job ID}

Header: content-type application/json

Auth: Basic or X auth

Success status code returned: 200

NOTE: Once SystemErase action is invoked and job ID is returned, server will automatically reboot to perform erase operations. Once all erase operations are complete the server will power off and reboot the iDRAC. Once the iDRAC is back up server will still be in OFF state.

NOTE: If LCDData component passed in for SystemErase action, this will erase iDRAC configuration database. If using the server after running SystemErase you must power on the server and let it complete POST / Collecting Inventory to write a new configuration database to iDRAC.

NOTE: If BIOS component passed in for SystemErase action, once the job is marked completed and iDRAC reboot, BIOS reset to default (RTD) process is only queued up, power ON the server for BIOS to complete RTD process. Once BIOS completes RTD process server will power OFF.

NOTE: If using SystemErase to erase drives, the operation does not allow you to select which drives to erase. Example: Pass in CryptographicErasePD component, this will erase any (Instant Scramble Erase (ISE) drive detected in the server. If you want to erase a specific drive use SecureErase action.

29 SecureErase Individual Drive

SecureErase DMTF action will allow to erase individual drive which supports Instant Scramble Erase (ISE). SecureErase action is supported on:

- Drives behind PERC
- NVMe direct connect drives
- BOSS M.2 drives
- VOSS drives

NOTE: SecureErase is supported on drives behind HBA controller but only when iDRAC Secure Enterprise Key Manager (SEKM) feature is enabled.

NOTE: If erasing a drive which is part of a RAID volume, you must delete the volume first before you can erase the drive.

NOTE: For more details about ISE process, see the documentation:

<https://infohub.delltechnologies.com/p/instant-scramble-erase/>

29.1 Check if Drive Supports ISE

Command: GET

URI: redfish/v1/Systems/System.Embedded.1/Storage/{controller ID}/Drives/{drive ID}?\$select=Oem/Dell/DellPhysicalDisk/CryptographicEraseCapable

URI Example:

redfish/v1/Systems/System.Embedded.1/Storage/RAID.SL.3-1/Drives/Disk.Bay.3:Enclosure.Internal.0-1:RAID.SL.3-1?\$select=Oem/Dell/DellPhysicalDisk/CryptographicEraseCapable

Header: content-type application/json

Auth: Basic or X auth

Success status code returned: 200

29.2 Secure Erase Drive

Command: POST

URI: redfish/v1/Systems/System.Embedded.1/Storage/{controller ID}/Drives/{drive ID}/Actions/Drive.SecureErase

URI example: redfish/v1/Systems/System.Embedded.1/Storage/RAID.SL.3-1/Drives/Disk.Bay.3:Enclosure.Internal.0-1:RAID.SL.3-1/Actions/Drive.SecureErase

Header: content-type application/json Auth: Basic or X auth

Body: {}

Success status code returned: 202

NOTE: Empty body is required for this action.

JOB ID URI is returned in JSON response headers location. Run GET command on this URI to monitor the job status.

NOTE: For NVMe drives, once a job ID is created you must reboot the server for secure erase to run. For drives behind PERC job will run in real time and no server reboot is required.

Command: GET

URI: redfish/v1/Managers/iDRAC.Embedded.1/Oem/Dell/Jobs/{job ID} Header: content-type application/json

Auth: Basic or X auth

Success status code returned: 200

30 Manage iDRAC Time

30.1 Get iDRAC Time

Command: POST
URI:
redfish/v1/Managers/iDRAC.Embedded.1/Oem/Dell/DellTimeService/Actions/DellTimeService.ManageTime
Header: content-type application/json
Auth: Basic or X auth
Body: {"GetRequest":True}

Success status code returned: 200

NOTE: In the JSON output "TimeData" property will show current iDRAC time in ISO 8601 format.

30.2 Set iDRAC Time

Command: POST
URI:
redfish/v1/Managers/iDRAC.Embedded.1/Oem/Dell/DellTimeService/Actions/DellTimeService.ManageTime
Header: content-type application/json
Auth: Basic or X auth
Body: {"GetRequest":False, "TimeData":"<pass in date time string with UTC offset>"}

Body Example:
{"GetRequest":False, "TimeData":"2023-10-30T09:29:53-05:00"}

Success status code returned: 200

NOTE: Running this command will set the iDRAC time but if you reboot the server and BIOS time is not in sync with iDRAC time, iDRAC will sync with BIOS time and override the time you set for iDRAC.

NOTE: If your OS time is not in sync with iDRAC and BIOS, both iDRAC and BIOS will sync with OS time. Recommend to always have all three aligned with the same date/time (OS, iDRAC, BIOS).

31 iDRAC10 AC power recovery server settings

In iDRAC10, the AC Power recovery settings of a server are managed by iDRAC only. However, in iDRAC9, this was managed by BIOS. To configure BIOS settings, see [Redfish BIOS Settings](#) section.

31.1 Get AC power recovery settings

```
Command: GET
URIs:
redfish/v1/Managers/iDRAC.Embedded.1/Oem/Dell/DellAttributes/System.Embedded.1
?$select=Attributes/ServerPwr.1.AcPwrRcvry
redfish/v1/Managers/iDRAC.Embedded.1/Oem/Dell/DellAttributes/System.Embedded.1
?$select=Attributes/ServerPwr.1.AcPwrRcvryDelay
redfish/v1/Managers/iDRAC.Embedded.1/Oem/Dell/DellAttributes/System.Embedded.1
?$select=Attributes/ServerPwr.1.AcPwrRcvryUserDelay
Header: content-type application/json
Auth: Basic or X auth
```

Success status code returned: 200

31.2 Set AC power recovery settings

```
Command: PATCH
URI:
redfish/v1/Managers/iDRAC.Embedded.1/Oem/Dell/DellAttributes/System.Embedded.1
Header: content-type application/json
Auth: Basic or X auth
Body: {
  "Attributes":{
    "AcPwrRcvry":"<pass in ON, OFF or Last>", "AcPwrRcvryDelay":"<pass in
Immediate, Random or UserDefined>", "AcPwrRcvryUserDelay":"<pass in integer
value from 120 to 600 (value in seconds)>"
  }
}
```

Success status code returned: 200

32 Dell System ID

32.1 Get Dell System ID

Command: GET

URI: redfish/v1/Systems/System.Embedded.1?\$select=Oem/Dell/DellSystem/SystemID

Header: content-type application/json

Auth: Basic or X auth

Success status code returned: 200

33 Server generation details

33.1 Get server generation details

Command: GET

URI: redfish/v1/Managers/iDRAC.Embedded.1?\$select=Model

Header: content-type application/json

Auth: Basic or X auth

Success status code returned: 200

34 Update Active Directory Kerberos key tab

34.1 Set Kerberos key tab

Command: PATCH
URI: redfish/v1/AccountService
Header: content-type application/json
Auth: Basic or X auth
Body: {"ActiveDirectory":{"Authentication":{"KerberosKeytab": "<base64-encoded
file version>"}}}

Success status code returned: 200

35 Debug iDRAC System Management Memory and processor performance

The response speed of some iDRAC interfaces may become slow because of unknown issues in the kernel. To resolve this issue, use the URIs to identify if iDRAC is running out of memory drive space or has a high utilization of processors.

NOTE: If iDRAC is running out of memory drive space or has a high processor utilization, reboot the iDRAC to resolve the issue.

35.1 Get iDRAC kernel memory statistics

Command: GET
URI:
redfish/v1/Managers/iDRAC.Embedded.1/ManagerDiagnosticData?\$select=MemoryStatistics
Header: content-type application/json
Auth: Basic or X auth

Success status code returned: 200

35.2 Get iDRAC kernel processor statistics

Command: GET
URI:
redfish/v1/Managers/iDRAC.Embedded.1/ManagerDiagnosticData?\$select=MemoryStatistics
Header: content-type application/json
Auth: Basic or X auth

Success status code returned: 200

36 Reset iDRAC to custom default settings

When you reset iDRAC to default settings, following three pre-defined static settings are allowed using OEM action `DellManager.ResetToDefaults`:

- **All:** all iDRAC configurations are set to default settings. Root password is reset to the default password set from the factory.
- **ResetAllWithRootDefaults:** all iDRAC configurations including network are set to default. Exception root user password is set to **calvin**.
- **Default:** all configurations are set to default, except users and network settings are preserved.

Besides the pre-defined settings, you can also configure the custom default iDRAC settings which you can reset at any time. See workflow example as follows to configure iDRAC custom default settings:

1. Confirm if custom defaults are uploaded to iDRAC

```
Command: GET
URI: redfish/v1/Managers/iDRAC.Embedded.1?$select=Actions/Oem

Header: content-type application/json

Auth: Basic or X auth
```

Success status code returned: 200

In the JSON output look for

#`DellManager.ResetToDefaults/ResetType@Redfish.AllowableValues`, you get the value `CustomDefaults` this means you already have custom default settings.

2. Create custom default settings using SCP export feature.

```
Command: POST

iDRAC9 URI:
/redfish/v1/Managers/iDRAC.Embedded.1/Actions/Oem/EID_674_Manager.ExportSystemCo
nfiguration

iDRAC10 URI:
/redfish/v1/Managers/iDRAC.Embedded.1/Actions/Oem/OemManager.ExportSystemConfigu
ration

Header: content-type application/json

Auth: Basic or X auth
```

```
Body: {"ExportFormat": "<pass in value XML or  
JSON>", "ShareParameters": {"Target": ["iDRAC"]}}
```

NOTE: After the POST call is invoked, in the headers location a JOB ID URI will be returned. Run the GET command on this URI until the job is completed. After the job ID is marked completed, you must see in the JSON response the complete SCP file contents in string format containing all iDRAC attributes. Modify these attributes as needed to configure your own iDRAC default settings.

3. After you modify all iDRAC settings you need to upload this SCP file to the iDRAC, so iDRAC can store a copy of your custom iDRAC default settings. Use action `DellManager.SetCustomDefaults` to upload the SCP file string.

Command: POST

URI:

`redfish/v1/Managers/iDRAC.Embedded.1/Actions/Oem/DellManager.SetCustomDefaults`

Header: `content-type application/json`

Auth: Basic or X auth

Body: {"CustomDefaults": "<pass in the complete edited
SCP file as a string value>"}

Body Example:

```
{"CustomDefaults": "<SystemConfiguration><Component FQDD=\"iDRAC.Embedded.1\"> <  
Attribute Name=\"IPMILan.1#Enable\">Enabled</Attribute> <Attribute Name=\"SSH.1#  
Enable\">Enabled</Attribute> <Attribute Name=\"WebServer.1#Enable\">Enabled</Att  
ribute> <Attribute Name=\"VirtualConsole.1#Enable\">Enabled</Attribute> <Attrib  
ute Name=\"IPv4.1#Enable\">Enabled</Attribute> <Attribute Name=\"IPv4.1#DHCPEnab  
le\">Enabled</Attribute> <Attribute Name=\"IPv6.1#Enable\">Enabled</Attribute> <  
Attribute Name=\"IPv6.1#AutoConfig\">Enabled</Attribute> </Component></SystemCon  
figuration>"}
```

Note: Once the POST call has been invoked, in the headers location a JOB ID URI will be returned, run GET command on this URI until the job is marked completed.

4. Repeat step 1 and under

`#DellManager.ResetToDefaults/ResetType@Redfish.AllowableValues`, confirm the value is `CustomDefaults`. This value means that the iDRAC custom default settings are stored in iDRAC.

5. If needed, you can get or view custom default settings stored in the iDRAC by running the following command:

Command: GET

URI: `redfish/v1/Managers/iDRAC.Embedded.1/Oem/Dell/CustomDefaultsDownloadURI`

Header: `content-type application/json`

Auth: Basic or X auth

Success status code returned: 200

NOTE: The JSON body output returned will show the stored custom defaults.

6. Reset iDRAC to default custom settings.

Command: POST

URI:

redfish/v1/Managers/iDRAC.Embedded.1/Actions/Oem/DellManager.ResetToDefaults

Header: content-type application/json

Auth: Basic or X auth

Body: {"ResetType":"CustomDefaults"}

37 Graphics Processing Unit (GPU) details

37.1 Get overall GPU details

Command: GET

URI: redfish/v1/Systems/System.Embedded.1/Processors/<GPU instance ID>

URI example: redfish/v1/Systems/System.Embedded.1/Processors/Video.Slot.7-1

Header: content-type application/json

Auth: Basic or X auth

Success status code returned: 200

NOTE: Use the URI redfish/v1/Systems/System.Embedded.1/Oem/Dell/DellVideo to get GPU details. This URI is helpful to get current GPU instance ID(s) installed in the server.

NOTE: More GPU details are also located in server hardware inventory. For more details to export hardware inventory, see section [Export Server Hardware Inventory](#).

37.2 Get GPU sensors

Command: GET

URI: redfish/v1/Systems/System.Embedded.1/Oem/Dell/DellGPUSensors/<GPU instance ID>

Header: content-type application/json

Auth: Basic or X auth

Success status code returned: 200

37.3 Get GPU power metrics

Command: GET

URI: redfish/v1/Systems/System.Embedded.1/Processors/<GPU instance ID>/Oem/Dell/PowerMetrics

Header: content-type application/json

Auth: Basic or X auth

Success status code returned: 200

37.4 Get GPU thermal metrics

Command: GET

URI: /redfish/v1/Systems/System.Embedded.1/Processors/<GPU instance ID>/Oem/Dell/ThermalMetrics

Header: content-type application/json

Auth: Basic or X auth

Success status code returned: 200

38 Get Operating System (OS) network interface details

38.1 Get OS network interface details

Command: GET

URI: redfish/v1/Systems/System.Embedded.1/EthernetInterfaces/<OS network instance ID>

URI example:

redfish/v1/Systems/System.Embedded.1/EthernetInterfaces/OSLogicalNetwork.1

Header: content-type application/json

Auth: Basic or X auth

Success status code returned: 200

NOTE: You must have iDRAC Service Module (iSM) installed in the OS and service running to get OS network details using iDRAC. If iSM is not installed or the service is not running, this URI will only return the hardware network device details. To check if iSM service is running, run GET command on the URI redfish/v1/Managers/iDRAC.Embedded.1/Oem/Dell/DellAttributes/iDRAC.Embedded.1?\$select=Attributes/ServiceModule.1.ServiceModuleState.

39 Set the server onetime boot device

39.1 Get the supported onetime boot device possible values

```
Command: GET
URI:
redfish/v1/Systems/System.Embedded.1?$select=Boot/BootSourceOverrideTarget@Redfi
sh.AllowableValuesHeader: content-type application/json
Auth: Basic or X auth
```

Success status code returned: 200

39.2 Get the current onetime boot device setting

```
Command: GET
URI: redfish/v1/Systems/System.Embedded.1?$select=Boot/BootSourceOverrideTarget
Header: content-type application/json
Auth: Basic or X auth
```

Success status code returned: 200

Note: Default value for this property is None. Also once this property is set to a onetime device and you reboot the server to onetime boot, this property will set it's value back to None.

39.3 Set the server onetime boot device to PXE

```
Command: PATCH
URI: redfish/v1/Systems/System.Embedded.1
Header: content-type application/json
Auth: Basic or X auth
Body: {"Boot":{"BootSourceOverrideTarget":"Pxe"}}
```

Success status code returned: 200

Confirm the `BootSourceOverrideTarget` property reports a value of PXE.

```
Command: GET
URI: redfish/v1/Systems/System.Embedded.1?$select=Boot/BootSourceOverrideTarget
Header: content-type application/json
Auth: Basic or X auth
```

Success status code returned: 200

Reboot the server to onetime boot to PXE device after POST completes.

```
Command: POST
URI: redfish/v1/Systems/System.Embedded.1/Actions/ComputerSystem.Reset
Header: content-type application/json
Auth: Basic or X auth
Body: {"ResetType":"ForceRestart"}
```

39.4 Set the onetime boot device to BIOS setup (F2)

```
Command: PATCH
URI: redfish/v1/Systems/System.Embedded.1
Header: content-type application/json
Auth: Basic or X auth
Body: {"Boot":{"BootSourceOverrideTarget":"BiosSetup"}}
```

Success status code returned: 200

Confirm the `BootSourceOverrideTarget` property reports a value of `BiosSetup`.

```
Command: GET
URI: redfish/v1/Systems/System.Embedded.1?$select=Boot/BootSourceOverrideTarget
Header: content-type application/json
Auth: Basic or X auth
```

Success status code returned: 200

Reboot the server to onetime boot to BIOS Setup.

```
Command: POST
URI: redfish/v1/Systems/System.Embedded.1/Actions/ComputerSystem.Reset
Header: content-type application/json
Auth: Basic or X auth
Body: {"ResetType":"ForceRestart"}
```

39.5 Set the onetime boot device to Virtual CD (ISO Image)

```
Command: PATCH
URI: redfish/v1/Systems/System.Embedded.1
Header: content-type application/json
Auth: Basic or X auth
Body: {"Boot":{"BootSourceOverrideTarget":"Cd"}}
```

Success status code returned: 200

NOTE: Make sure you have virtual CD ISO image attached before running the PATCH command.

Confirm that `BootSourceOverrideTarget` property reports a value of `Cd`.

Command: GET
URI: `redfish/v1/Systems/System.Embedded.1?$select=Boot/BootSourceOverrideTarget`
Header: `content-type application/json`
Auth: Basic or X auth

Success status code returned: 200

Reboot the server to onetime boot to virtual CD (ISO image) after POST completes.

Command: POST
URI: `redfish/v1/Systems/System.Embedded.1/Actions/ComputerSystem.Reset`
Header: `content-type application/json`
Auth: Basic or X auth
Body: `{"ResetType":"ForceRestart"}`

39.6 Set onetime boot device to Virtual Floppy (IMG Image)

Command: PATCH
URI: `redfish/v1/Systems/System.Embedded.1`
Header: `content-type application/json`
Auth: Basic or X auth
Body: `{"Boot":{"BootSourceOverrideTarget":"Floppy"}}`

Success status code returned: 200

NOTE: Make sure you have virtual floppy IMG image attached before running the PATCH command.

Confirm that `BootSourceOverrideTarget` property reports a value of `Floppy`.

Command: GET
URI: `redfish/v1/Systems/System.Embedded.1?$select=Boot/BootSourceOverrideTarget`
Header: `content-type application/json`
Auth: Basic or X auth

Success status code returned: 200

Reboot the server to onetime boot to virtual floppy (IMG image) after POST completes.

Command: POST
URI: `redfish/v1/Systems/System.Embedded.1/Actions/ComputerSystem.Reset`
Header: `content-type application/json`
Auth: Basic or X auth
Body: `{"ResetType":"ForceRestart"}`

40 Block or Disable OS in-band commands to iDRAC

BIOS supports attribute `InBandManageabilityInterface` which allows the user to block or disable Operating System (OS) in-band commands (IPMI, Redfish, RACADM) to the iDRAC. See the following workflow that use Redfish to set the BIOS attribute:

- Get current attribute value which should report **Enabled** (default value).

```
Command: GET
URI:
redfish/v1/Systems/System.Embedded.1/Bios?$select=Attributes/InBandManageability
Interface
Header: content-type application/json
Auth: Basic or X auth
```

Success status code returned: 200

- Set attribute to **Enabled**.

```
Command: PATCH
URI: redfish/v1/Systems/System.Embedded.1/Bios/Settings
Header: content-type application/json
Auth: Basic or X auth
Body: {
  "@Redfish.SettingsApplyTime":{
    "ApplyTime":"OnReset"
  },
  "Attributes":{
    "InBandManageabilityInterface":"Disabled",
  }
}
```

Success status code returned: 202

- JOB ID URI will be returned in JSON response headers location. Run the `GET` command to monitor the job status to show **Scheduled**.

```
Command: GET
URI: redfish/v1/Managers/iDRAC.Embedded.1/Oem/Dell/Jobs/{job ID}
Header: content-type application/json
Auth: Basic or X auth
```

Success status code returned: 200

- Reboot the server to run the **config job**.

```
Command: POST
URI: redfish/v1/Systems/System.Embedded.1/Actions/ComputerSystem.Reset
Header: content-type application/json
Auth: Basic or X auth
Body: {
```

```
"ResetType": "GracefulRestart"  
}
```

Success status code returned: 204

- After you reboot the server, run the `GET` command again on the same job ID URI to monitor the status until marked completed.

Command: `GET`
URI: `redfish/v1/Managers/iDRAC.Embedded.1/Jobs/{job ID}`
Header: `content-type application/json`
Auth: Basic or X auth

Success status code returned: 200

- Get new current attribute value which should now report **Disabled**.

Command: `GET`
URI:
`redfish/v1/Systems/System.Embedded.1/Bios?$select=Attributes/InBandManageability`
`Interface`
Header: `content-type application/json`
Auth: Basic or X auth

Success status code returned: 200

41 Change iDRAC IPv4 network settings from DHCP to Static

Do the following to change iDRAC IPv4 network settings from **DHCP** to **Static**.

- Get current IPv4.1.DHCPEnable attribute value to report **Enable** (default value).

Command: GET

URI:

redfish/v1/Managers/iDRAC.Embedded.1/Oem/Dell/DellAttributes/iDRAC.Embedded.1?\$select=Attributes/IPv4.1.DHCPEnable

Header: content-type application/json

Auth: Basic or X auth

Success status code returned: 200

- Run the PATCH command to disable DHCP and set static IP address, netmask, and gateway.

Command: PATCH

URI:

redfish/v1/Managers/iDRAC.Embedded.1/Oem/Dell/DellAttributes/iDRAC.Embedded.1

Header: content-type application/json

Auth: Basic or X auth

Body: {"Attributes":{"IPv4.1.DHCPEnable":"Disabled","IPv4Static.1.Address":"<new static iDRAC IP>","IPv4Static.1.Netmask":"<new static iDRAC netmask>","IPv4Static.1.Gateway":"<new static iDRAC gateway>"}}

Body Example:

```
{"Attributes":{"IPv4.1.DHCPEnable":"Disabled","IPv4Static.1.Address":"192.168.0.120","IPv4Static.1.Netmask":"255.255.255.0","IPv4Static.1.Gateway":"192.168.0.254"}}
```

Success status code returned: 200

NOTE: It is recommended to use one PATCH command to disable both DHCP and set static IP, netmask, and gateway. If you run the PATCH command and pass the attribute IPv4.1.DHCPEnable to disable DHCP, you will lose iDRAC connection and you must recover it from F2 Device Settings.

42 Redfish Query Parameters

Various DMTF query parameters are available to use when running a GET request. Leverage the query parameters for automation or scripting, it simplifies to retrieve the required data instead of adding your own extra code to parse the JSON and get the same data.

NOTE: Passing in multiple query parameters for one URI is not supported by iDRAC.

42.1 \$select

\$select query parameter can be leveraged to get only a specific property from a URI when performing a GET request.

- Example below using \$select query parameter to get only “PowerState” property current value.

```
[root@localhost ~]# curl -k -X GET -u root:calvin
'https://192.168.0.120/redfish/v1/Systems/System.Embedded.1/?$select=PowerState'
--insecure | jq
  % Total    % Received % Xferd  Average Speed   Time    Time       Time  Current
                                 Dload  Upload   Total   Spent    Left   Speed
100   197    100   197     0     0    398       0 --:--:-- --:--:-- --:--:--   398
{
  "@odata.context": "/redfish/v1/$metadata#ComputerSystem.ComputerSystem",
  "@odata.id": "/redfish/v1/Systems/System.Embedded.1",
  "@odata.type": "#ComputerSystem.v1_22_1.ComputerSystem",
  "PowerState": "On"
}
```

- Example below using \$select query parameter to get only iDRAC attribute “Job.1.JobTimeout” current value.

```
[root@localhost ~]# curl -k -X GET -u root:calvin
'https://192.168.0.120/redfish/v1/Managers/iDRAC.Embedded.1/Oem/Dell/DellAttributes/System.Embedded.1?$select=Attributes/Job.1.JobTimeout' --insecure | jq
  % Total    % Received % Xferd  Average Speed   Time    Time       Time  Current
                                 Dload  Upload   Total   Spent    Left   Speed
100   258    100   258     0     0    158       0 0:00:01 0:00:01 --:--:--   158
{
  "@odata.context": "/redfish/v1/$metadata#DellAttributes.DellAttributes",
  "@odata.id":
"/redfish/v1/Managers/iDRAC.Embedded.1/Oem/Dell/DellAttributes/System.Embedded.1",
  "@odata.type": "#DellAttributes.v1_0_0.DellAttributes",
  "Attributes": {
```

```

    "Job.1.JobTimeout": 120
  }
}

```

- Example below using \$select query parameter to get encryption status for a specific NVMe drive.

```

[root@localhost ~]# curl -k -X GET -u root:calvin
'https://192.168.0.120/redfish/v1/Systems/System.Embedded.1/Storage/CPU.1/Drives
/Disk.Bay.9:Enclosure.Internal.0-1?$select=EncryptionStatus' --insecure | jq
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left   Speed
100   231   100   231    0    0   355      0 --:--:-- --:--:-- --:--:--   355
{
  "@odata.context": "/redfish/v1/$metadata#Drive.Drive",
  "@odata.id":
"/redfish/v1/Systems/System.Embedded.1/Storage/CPU.1/Drives/Disk.Bay.9:Enclosure
.Internal.0-1",
  "@odata.type": "#Drive.v1_19_0.Drive",
  "EncryptionStatus": "Unencrypted"
}

```

42.2 \$top

\$top query parameter can be leveraged to return only a specific number of Members from a URI for a GET request starting from the top if the Members collection.

- Example below using \$top query parameter to return only the first top Member in the collection. You'll notice the member count is 16 and only 1 member was returned in the JSON response.

```

[root@localhost ~]# curl -k -X GET -u root:calvin
'https://192.168.0.120/redfish/v1/Systems/System.Embedded.1/Memory?$top=1' --
insecure | jq
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left   Speed
100   395   100   395    0    0  1329      0 --:--:-- --:--:-- --:--:--  1325
{
  "@odata.context": "/redfish/v1/$metadata#MemoryCollection.MemoryCollection",
  "@odata.id": "/redfish/v1/Systems/System.Embedded.1/Memory",
  "@odata.type": "#MemoryCollection.MemoryCollection",
  "Description": "Collection of memory devices for this system",
  "Members": [
    {
      "@odata.id": "/redfish/v1/Systems/System.Embedded.1/Memory/DIMM.Socket.B7"
    }
  ],
}

```

```

    "Members@odata.count": 16,
    "Name": "Memory Devices Collection"
}

```

Example below using \$top query parameter to return the first two members in the collection. You'll notice the member count is 3 and only 2 members are returned in the JSON response.

```

[root@localhost ~]# curl -k -X GET -u root:calvin
'https://192.168.0.120/redfish/v1/Chassis/System.Embedded.1/NetworkAdapters?$top=2' --insecure | jq

```

% Total	% Received	% Xferd	Average Speed		Time	Time	Time	Current
			Dload	Upload	Total	Spent	Left	Speed
100	517	100	517	0	0	2779	0	--:--:-- --:--:-- --:--:-- 2779

```

{
  "@odata.context":
"/redfish/v1/$metadata#NetworkAdapterCollection.NetworkAdapterCollection",
  "@odata.id": "/redfish/v1/Chassis/System.Embedded.1/NetworkAdapters",
  "@odata.type": "#NetworkAdapterCollection.NetworkAdapterCollection",
  "Description": "Collection Of Network Adapter",
  "Members": [
    {
      "@odata.id":
"/redfish/v1/Chassis/System.Embedded.1/NetworkAdapters/NIC.Embedded.1"
    },
    {
      "@odata.id":
"/redfish/v1/Chassis/System.Embedded.1/NetworkAdapters/NIC.Integrated.1"
    }
  ],
  "Members@odata.count": 3,
  "Name": "Network Adapter Collection"
}

```

42.3 \$expand

\$expand query parameter can be leveraged to expand nested URIs detected in the response when you run GET request on a URI.

NOTE: Depending on the number of nested URIs \$expand may take a few seconds to return all the data which is expected behavior.

NOTE: On max server configurations \$expand may take longer than a few seconds to return data or may timeout and return no data. Recommend to first perform GET on the URI without \$expand to check the number of items in the collection and if it is less than 50, proceed with \$expand to get all data. Otherwise, using scripting to cycle through each Members URI by performing GET to return the data.

- Example below shows first running GET request without using \$expand which shows only the Members URIs. Using the same GET request add \$expand and in the output you will see each member URI expanded and shows all the properties for each URI.

```
[root@localhost ~]# curl -k -X GET -u root:calvin
'https://192.168.0.120/redfish/v1/Systems/System.Embedded.1/Processors' --
insecure | jq
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left   Speed
100   481    100   481     0     0   1871      0 --:--:-- --:--:-- --:--:--  1871
{
  "@odata.context":
"/redfish/v1/$metadata#ProcessorCollection.ProcessorCollection",
  "@odata.id": "/redfish/v1/Systems/System.Embedded.1/Processors",
  "@odata.type": "#ProcessorCollection.ProcessorCollection",
  "Description": "Collection of Processors for this System",
  "Members": [
    {
      "@odata.id":
"/redfish/v1/Systems/System.Embedded.1/Processors/CPU.Socket.1"
    },
    {
      "@odata.id":
"/redfish/v1/Systems/System.Embedded.1/Processors/CPU.Socket.2"
    }
  ],
  "Members@odata.count": 2,
  "Name": "ProcessorsCollection"
}
```

```
[root@localhost ~]# curl -k -X GET -u root:calvin
'https://192.168.0.120/redfish/v1/Systems/System.Embedded.1/Processors?$expand=*
($levels=1)' --insecure | jq
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left   Speed
100  7525    100  7525     0     0  14726      0 --:--:-- --:--:-- --:--:-- 14726
{
  "@odata.context":
"/redfish/v1/$metadata#ProcessorCollection.ProcessorCollection",
  "@odata.id": "/redfish/v1/Systems/System.Embedded.1/Processors",
  "@odata.type": "#ProcessorCollection.ProcessorCollection",
  "Description": "Collection of Processors for this System",
```

```

"Members": [
  {
    "@odata.id":
"/redfish/v1/Systems/System.Embedded.1/Processors/CPU.Socket.1",
    "@odata.type": "#Processor.v1_20_0.Processor",
    "@odata.context": "/redfish/v1/$metadata#Processor.Processor",
    "Assembly": {
      "@odata.id": "/redfish/v1/Chassis/System.Embedded.1/Assembly"
    },
    "Description": "Represents the properties of a Processor attached to this
System",
    "Enabled": true,
    "EnvironmentMetrics": {
      "@odata.id":
"/redfish/v1/Systems/System.Embedded.1/Processors/CPU.Socket.1/EnvironmentMetric
s"
    },
    "Id": "CPU.Socket.1",
    "InstructionSet": "x86-64",
    "Links": {
      "Chassis": {
        "@odata.id": "/redfish/v1/Chassis/System.Embedded.1"
      },
      "Memory": [
        {
          "@odata.id":
"/redfish/v1/Systems/System.Embedded.1/Memory/DIMM.Socket.A3"
        },
        {
          "@odata.id":
"/redfish/v1/Systems/System.Embedded.1/Memory/DIMM.Socket.A4"
        },
        {
          "@odata.id":
"/redfish/v1/Systems/System.Embedded.1/Memory/DIMM.Socket.A6"
        },
        {
          "@odata.id":
"/redfish/v1/Systems/System.Embedded.1/Memory/DIMM.Socket.A2"
        },
        {
          "@odata.id":
"/redfish/v1/Systems/System.Embedded.1/Memory/DIMM.Socket.A8"
        },
        {
          "@odata.id":
"/redfish/v1/Systems/System.Embedded.1/Memory/DIMM.Socket.A5"
        }
      ]
    }
  }
]

```

```

        "@odata.id":
"/redfish/v1/Systems/System.Embedded.1/Memory/DIMM.Socket.A7"
    },
    {
        "@odata.id":
"/redfish/v1/Systems/System.Embedded.1/Memory/DIMM.Socket.A1"
    }
],
    "Memory@odata.count": 8
},
    "Manufacturer": "Intel",
    "MaxSpeedMHz": 4000,
    "MemorySummary": {
        "TotalCacheSizeMiB": 79,
        "TotalMemorySizeMiB": 131072
    },
    "Model": "Intel(R) Xeon(R) Gold 6330 CPU @ 2.00GHz",
    "Name": "CPU 1",
    "Oem": {
        "Dell": {
            "@odata.type": "#DellOem.v1_3_0.DellOemResources",
            "DellProcessor": {
                "CPUFamily": "Intel(R) Xeon(TM)",
                "CPUStatus": "CPUEnabled",
                "Cache1Associativity": "12-WaySet-Associative",
                "Cache1ErrorMethodology": "Parity",
                "Cache1InstalledSizeKB": 2240,
                "Cache1Level": "L1",
                "Cache1Location": "Internal",
                "Cache1PrimaryStatus": "OK",
                "Cache1SRAMType": "Unknown",
                "Cache1SizeKB": 2240,
                "Cache1Type": "Unified",
                "Cache1WritePolicy": "WriteBack",
                "Cache2Associativity": "20-WaySet-Associative",
                "Cache2ErrorMethodology": "Single-bitECC",
                "Cache2InstalledSizeKB": 35840,
                "Cache2Level": "L2",
                "Cache2Location": "Internal",
                "Cache2PrimaryStatus": "OK",
                "Cache2SRAMType": "Unknown",
                "Cache2SizeKB": 35840,
                "Cache2Type": "Unified",
                "Cache2WritePolicy": "WriteBack",
                "Cache3Associativity": "12-WaySet-Associative",
                "Cache3ErrorMethodology": "Single-bitECC",
                "Cache3InstalledSizeKB": 43008,
                "Cache3Level": "L3",
                "Cache3Location": "Internal",
                "Cache3PrimaryStatus": "OK",

```

```

        "Cache3SRAMType": "Unknown",
        "Cache3SizeKB": 43008,
        "Cache3Type": "Unified",
        "Cache3WritePolicy": "WriteBack",
        "CurrentClockSpeedMhz": 2000,
        "ExecuteDisabledCapable": "Yes",
        "ExecuteDisabledEnabled": "Yes",
        "ExternalBusClockSpeedMhz": 11200,
        "HyperThreadingCapable": "Yes",
        "HyperThreadingEnabled": "Yes",
        "Id": "CPU.Socket.1",
        "LastSystemInventoryTime": "2024-12-06T04:51:57+00:00",
        "LastUpdateTime": "2024-10-01T23:48:24+00:00",
        "Name": "DellProcessor",
        "TurboModeCapable": "Yes",
        "TurboModeEnabled": "Yes",
        "VirtualizationTechnologyCapable": "Yes",
        "VirtualizationTechnologyEnabled": "Yes",
        "Volts": "1.6",
        "@odata.context":
"/redfish/v1/$metadata#DellProcessor.DellProcessor",
        "@odata.type": "#DellProcessor.v1_2_0.DellProcessor",
        "@odata.id":
"/redfish/v1/Systems/System.Embedded.1/Processors/CPU.Socket.1/Oem/Dell/DellProc
essors/CPU.Socket.1"
    }
}
},
"OperatingSpeedMHz": 2000,
"ProcessorArchitecture": "x86",
"ProcessorId": {
    "EffectiveFamily": "6",
    "EffectiveModel": "106",
    "IdentificationRegisters": "0x000606A6",
    "MicrocodeInfo": "0xD0003E7",
    "ProtectedIdentificationNumber": "60053DE09E921819",
    "Step": "6",
    "VendorId": "GenuineIntel"
},
"ProcessorType": "CPU",
"Socket": "CPU.Socket.1",
"Status": {
    "Health": "OK",
    "State": "Enabled"
},
"TotalCores": 28,
"TotalEnabledCores": 28,
"TotalThreads": 56,
"TurboState": "Enabled",
"Version": "Model 106 Stepping 6"

```

```

    },
    {
        "@odata.id":
"/redfish/v1/Systems/System.Embedded.1/Processors/CPU.Socket.2",
        "@odata.type": "#Processor.v1_20_0.Processor",
        "@odata.context": "/redfish/v1/$metadata#Processor.Processor",
        "Assembly": {
            "@odata.id": "/redfish/v1/Chassis/System.Embedded.1/Assembly"
        },
        "Description": "Represents the properties of a Processor attached to this
System",
        "Enabled": true,
        "EnvironmentMetrics": {
            "@odata.id":
"/redfish/v1/Systems/System.Embedded.1/Processors/CPU.Socket.2/EnvironmentMetric
s"
        },
        "Id": "CPU.Socket.2",
        "InstructionSet": "x86-64",
        "Links": {
            "Chassis": {
                "@odata.id": "/redfish/v1/Chassis/System.Embedded.1"
            },
            "Memory": [
                {
                    "@odata.id":
"/redfish/v1/Systems/System.Embedded.1/Memory/DIMM.Socket.B7"
                },
                {
                    "@odata.id":
"/redfish/v1/Systems/System.Embedded.1/Memory/DIMM.Socket.B8"
                },
                {
                    "@odata.id":
"/redfish/v1/Systems/System.Embedded.1/Memory/DIMM.Socket.B3"
                },
                {
                    "@odata.id":
"/redfish/v1/Systems/System.Embedded.1/Memory/DIMM.Socket.B5"
                },
                {
                    "@odata.id":
"/redfish/v1/Systems/System.Embedded.1/Memory/DIMM.Socket.B1"
                },
                {
                    "@odata.id":
"/redfish/v1/Systems/System.Embedded.1/Memory/DIMM.Socket.B2"
                }
            ]
        }
    }

```

```

        "@odata.id":
"/redfish/v1/Systems/System.Embedded.1/Memory/DIMM.Socket.B6"
    },
    {
        "@odata.id":
"/redfish/v1/Systems/System.Embedded.1/Memory/DIMM.Socket.B4"
    }
],
    "Memory@odata.count": 8
},
    "Manufacturer": "Intel",
    "MaxSpeedMHz": 4000,
    "MemorySummary": {
        "TotalCacheSizeMiB": 79,
        "TotalMemorySizeMiB": 131072
    },
    "Model": "Intel(R) Xeon(R) Gold 6330 CPU @ 2.00GHz",
    "Name": "CPU 2",
    "Oem": {
        "Dell": {
            "@odata.type": "#DellOem.v1_3_0.DellOemResources",
            "DellProcessor": {
                "CPUFamily": "Intel(R) Xeon(TM)",
                "CPUStatus": "CPUEnabled",
                "Cache1Associativity": "12-WaySet-Associative",
                "Cache1ErrorMethodology": "Parity",
                "Cache1InstalledSizeKB": 2240,
                "Cache1Level": "L1",
                "Cache1Location": "Internal",
                "Cache1PrimaryStatus": "OK",
                "Cache1SRAMType": "Unknown",
                "Cache1SizeKB": 2240,
                "Cache1Type": "Unified",
                "Cache1WritePolicy": "WriteBack",
                "Cache2Associativity": "20-WaySet-Associative",
                "Cache2ErrorMethodology": "Single-bitECC",
                "Cache2InstalledSizeKB": 35840,
                "Cache2Level": "L2",
                "Cache2Location": "Internal",
                "Cache2PrimaryStatus": "OK",
                "Cache2SRAMType": "Unknown",
                "Cache2SizeKB": 35840,
                "Cache2Type": "Unified",
                "Cache2WritePolicy": "WriteBack",
                "Cache3Associativity": "12-WaySet-Associative",
                "Cache3ErrorMethodology": "Single-bitECC",
                "Cache3InstalledSizeKB": 43008,
                "Cache3Level": "L3",
                "Cache3Location": "Internal",
                "Cache3PrimaryStatus": "OK",

```

```

        "Cache3SRAMType": "Unknown",
        "Cache3SizeKB": 43008,
        "Cache3Type": "Unified",
        "Cache3WritePolicy": "WriteBack",
        "CurrentClockSpeedMhz": 2000,
        "ExecuteDisabledCapable": "Yes",
        "ExecuteDisabledEnabled": "Yes",
        "ExternalBusClockSpeedMhz": 11200,
        "HyperThreadingCapable": "Yes",
        "HyperThreadingEnabled": "Yes",
        "Id": "CPU.Socket.2",
        "LastSystemInventoryTime": "2024-12-06T04:51:57+00:00",
        "LastUpdateTime": "2024-10-01T23:48:24+00:00",
        "Name": "DellProcessor",
        "TurboModeCapable": "Yes",
        "TurboModeEnabled": "Yes",
        "VirtualizationTechnologyCapable": "Yes",
        "VirtualizationTechnologyEnabled": "Yes",
        "Volts": "1.6",
        "@odata.context":
"/redfish/v1/$metadata#DellProcessor.DellProcessor",
        "@odata.type": "#DellProcessor.v1_2_0.DellProcessor",
        "@odata.id":
"/redfish/v1/Systems/System.Embedded.1/Processors/CPU.Socket.2/Oem/Dell/DellProc
essors/CPU.Socket.2"
    }
}
},
"OperatingSpeedMHz": 2000,
"ProcessorArchitecture": "x86",
"ProcessorId": {
    "EffectiveFamily": "6",
    "EffectiveModel": "106",
    "IdentificationRegisters": "0x000606A6",
    "MicrocodeInfo": "0xD0003E7",
    "ProtectedIdentificationNumber": "5FB635E0E8E36687",
    "Step": "6",
    "VendorId": "GenuineIntel"
},
"ProcessorType": "CPU",
"Socket": "CPU.Socket.2",
"Status": {
    "Health": "OK",
    "State": "Enabled"
},
"TotalCores": 28,
"TotalEnabledCores": 28,
"TotalThreads": 56,
"TurboState": "Enabled",
"Version": "Model 106 Stepping 6"

```

```

    }
  ],
  "Members@odata.count": 2,
  "Name": "ProcessorsCollection"
}

```

42.4 \$filter

`$filter` query parameter can be leveraged when you only want to get specific members and their content from a member collection. `$filter` is helpful for parsing the iDRAC lifecycle logs to get only specific entries.

- Example below shows using `$filter` on iDRAC lifecycle logs to get all entries where severity is reported as critical.

NOTE: If `$filter` response returns an empty Members collection, this means the search was unable to locate any data within the filter parameters set.

NOTE: In the example below URI

“redfish/v1/Managers/iDRAC.Embedded.1/LogServices/Lclog/Entries?\$filter=Severity eq 'Critical'” is converted so Curl command will pass (whitespace in the query filter will fail the command).

```

[root@localhost ~]# curl -k -X GET -u root:calvin
'https://192.168.0.120/redfish/v1/Managers/iDRAC.Embedded.1/LogServices/Lclog/En
tries?$filter=Severity%20eq%20%27Critical%27' --insecure | jq
  % Total    % Received % Xferd  Average Speed   Time    Time       Time  Current
                                 Dload  Upload  Total   Spent    Left   Speed
100 1050    100 1050      0      0   1988      0 --:--:-- --:--:-- --:--:-- 1988
{
  "@odata.context":
"/redfish/v1/$metadata#LogEntryCollection.LogEntryCollection",
  "@odata.id":
"/redfish/v1/Managers/iDRAC.Embedded.1/LogServices/Lclog/Entries",
  "@odata.type": "#LogEntryCollection.LogEntryCollection",
  "Description": "LC Logs for this manager",
  "Members": [
    {
      "@odata.id":
"/redfish/v1/Managers/iDRAC.Embedded.1/LogServices/Lclog/Entries/2425",
      "@odata.type": "#LogEntry.v1_16_1.LogEntry",

```

```

    "Created": "2024-11-26T14:26:37-06:00",
    "Description": "Log Entry 2425",
    "EntryType": "Oem",
    "Id": "2425",
    "Links": {
      "OriginOfCondition": {
        "@odata.id":
"/redfish/v1/Systems/System.Embedded.1/Processors/CPU.Socket.1"
      }
    },
    "Message": "CPU 1 temperature is less than the lower critical threshold.",
    "MessageArgs": [
      "1"
    ],
    "MessageArgs@odata.count": 1,
    "MessageId": "IDRAC.2.10.TMP0201",
    "Name": "Log Entry 2425",
    "Oem": {
      "Dell": {
        "@odata.type": "#DellLCLogEntry.v1_2_0.DellLCLogEntry",
        "AgentID": "SEL",
        "BitMask": "4101",
        "Category": "SystemHealth",
        "Comment": null,
        "LastUpdatedByUser": null
      }
    },
    "OemRecordFormat": "Dell",
    "Severity": "Critical"
  }
],
"Members@odata.count": 1,
"Name": "Log Entry Collection"
}

```

- Example below shows using \$filter on iDRAC lifecycle logs to get all entries which occurred during a specific date/time.

NOTE: In the example below URI

“redfish/v1/Managers/iDRAC.Embedded.1/LogServices/Lclog/Entries?\$filter=Created%20ge%20\"2024-12-06T23:33:27-06:00\"%20and%20Created%20le%20\"2024-12-07T01:46:41-06:00\"” is converted so Curl command will pass (whitespace in the query filter will fail the command).

```

[root@localhost ~]# curl -k -X GET -u root:calvin
'https://192.168.0.120/redfish/v1/Managers/iDRAC.Embedded.1/LogServices/Lclog/En
tries?$filter=Created%20ge%20\'\'2024-12-06T23:33:27-

```

```

06:00'\''%20and%20Created%20le%20'\''2024-12-07T01:46:41-06:00'\'' --insecure |
jq
% Total      % Received % Xferd  Average Speed   Time    Time       Time  Current
           Dload  Upload   Total     Spent    Left     Speed
100  3560  100  3560    0      0  14412      0 --:--:-- --:--:-- --:--:-- 14354
{
  "@odata.context":
"/redfish/v1/$metadata#LogEntryCollection.LogEntryCollection",
  "@odata.id":
"/redfish/v1/Managers/iDRAC.Embedded.1/LogServices/Lclog/Entries",
  "@odata.type": "#LogEntryCollection.LogEntryCollection",
  "Description": "LC Logs for this manager",
  "Members": [
    {
      "@odata.id":
"/redfish/v1/Managers/iDRAC.Embedded.1/LogServices/Lclog/Entries/56378",
      "@odata.type": "#LogEntry.v1_16_1.LogEntry",
      "Created": "2024-12-07T01:46:41-06:00",
      "Description": "Log Entry 56378",
      "EntryType": "Oem",
      "Id": "56378",
      "Links": {
        "OriginOfCondition": {
          "@odata.id": "/redfish/v1/Managers/iDRAC.Embedded.1"
        }
      },
      "Message": "Successfully logged in using root, from 192.168.0.130 and
REDFISH.",
      "MessageArgs": [
        "root",
        "192.168.0.130",
        "REDFISH"
      ],
      "MessageArgs@odata.count": 3,
      "MessageId": "IDRAC.2.9.USR0030",
      "Name": "Log Entry 56378",
      "Oem": {
        "Dell": {
          "@odata.type": "#DellLCLogEntry.v1_1_0.DellLCLogEntry",
          "Category": "Audit",
          "Comment": null,
          "LastUpdatedByUser": null
        }
      },
      "OemRecordFormat": "Dell",
      "Severity": "OK"
    },
    {
      "@odata.id":
"/redfish/v1/Managers/iDRAC.Embedded.1/LogServices/Lclog/Entries/56377",

```

```

"@odata.type": "#LogEntry.v1_16_1.LogEntry",
"Created": "2024-12-07T01:46:41-06:00",
"Description": "Log Entry 56377",
"EntryType": "Oem",
"Id": "56377",
"Links": {},
"Message": "The previous log entry was repeated 1 times.",
"MessageArgs": [
  "1"
],
"MessageArgs@odata.count": 1,
"MessageId": "IDRAC.2.9.LOG007",
"Name": "Log Entry 56377",
"Oem": {
  "Dell": {
    "@odata.type": "#DellLCLogEntry.v1_1_0.DellLCLogEntry",
    "Category": "Audit",
    "Comment": null,
    "LastUpdatedByUser": null
  }
},
"OemRecordFormat": "Dell",
"Severity": "OK"
},
{
  "@odata.id":
"/redfish/v1/Managers/iDRAC.Embedded.1/LogServices/Lclog/Entries/56376",
  "@odata.type": "#LogEntry.v1_16_1.LogEntry",
  "Created": "2024-12-06T23:47:52-06:00",
  "Description": "Log Entry 56376",
  "EntryType": "Oem",
  "Id": "56376",
  "Links": {
    "OriginOfCondition": {
      "@odata.id": "/redfish/v1/Managers/iDRAC.Embedded.1"
    }
  },
  "Message": "Successfully logged in using root, from 192.168.0.130 and
REDFISH.",
  "MessageArgs": [
    "root",
    "192.168.0.130",
    "REDFISH"
  ],
  "MessageArgs@odata.count": 3,
  "MessageId": "IDRAC.2.9.USR0030",
  "Name": "Log Entry 56376",
  "Oem": {
    "Dell": {
      "@odata.type": "#DellLCLogEntry.v1_1_0.DellLCLogEntry",

```

```

        "Category": "Audit",
        "Comment": null,
        "LastUpdatedByUser": null
    }
},
    "OemRecordFormat": "Dell",
    "Severity": "OK"
},
{
    "@odata.id":
"/redfish/v1/Managers/iDRAC.Embedded.1/LogServices/Lclog/Entries/56375",
    "@odata.type": "#LogEntry.v1_16_1.LogEntry",
    "Created": "2024-12-06T23:47:52-06:00",
    "Description": "Log Entry 56375",
    "EntryType": "Oem",
    "Id": "56375",
    "Links": {},
    "Message": "The previous log entry was repeated 3 times.",
    "MessageArgs": [
        "3"
    ],
    "MessageArgs@odata.count": 1,
    "MessageId": "IDRAC.2.9.LOG007",
    "Name": "Log Entry 56375",
    "Oem": {
        "Dell": {
            "@odata.type": "#DellLCLogEntry.v1_1_0.DellLCLogEntry",
            "Category": "Audit",
            "Comment": null,
            "LastUpdatedByUser": null
        }
    },
    "OemRecordFormat": "Dell",
    "Severity": "OK"
},
{
    "@odata.id":
"/redfish/v1/Managers/iDRAC.Embedded.1/LogServices/Lclog/Entries/56374",
    "@odata.type": "#LogEntry.v1_16_1.LogEntry",
    "Created": "2024-12-06T23:33:27-06:00",
    "Description": "Log Entry 56374",
    "EntryType": "Oem",
    "Id": "56374",
    "Links": {
        "OriginOfCondition": {
            "@odata.id": "/redfish/v1/Managers/iDRAC.Embedded.1"
        }
    },
    "Message": "Successfully logged in using root, from 192.168.0.130 and
REDFISH.",

```

```

    "MessageArgs": [
      "root",
      "192.168.0.130",
      "REDFISH"
    ],
    "MessageArgs@odata.count": 3,
    "MessageId": "IDRAC.2.9.USR0030",
    "Name": "Log Entry 56374",
    "Oem": {
      "Dell": {
        "@odata.type": "#DellLCLogEntry.v1_1_0.DellLCLogEntry",
        "Category": "Audit",
        "Comment": null,
        "LastUpdatedByUser": null
      }
    },
    "OemRecordFormat": "Dell",
    "Severity": "OK"
  }
],
"Members@odata.count": 5,
"Name": "Log Entry Collection"
}

```

43 Redfish Error Codes

43.1 HTTP status codes and error messages

HTTP defines the status codes that are returned in response messages. When the HTTP status code indicates a failure, the response body contains an extended error resource, which provides meaningful and deterministic error semantics.

The extended-error information for the Redfish service that Dell has implemented contains error or exception information that is unique to Dell. This information provides more details and recommendations for error resolution. To learn more about extended error information, see the Event and Error Message Reference Guide available at www.dell.com/manuals.

For more details about the error messages, see the following:

- Base messages: Registries > BaseMessage schema. Examples of base messages are GeneralError, PropertyUnknown, InsufficientPrivilege, and so on.
- Event and Error messages: Registries > Message schema. Examples of Event and Error messages are AMP0300, ASR0001, HWC7014, and so on.

For more information about supported status codes and base messages, see the Redfish Scalable Platforms Management API Specification document available at www.dmtf.org/standards/redfish.

For information about OEM event IDs and error messages, see the KB (Knowledge Based) article [link](#).

43.2 Message Registry/Error Message Index Error Codes

This section contains information about various Event and Error Message indicator (EEMI) Redfish error codes. EEMI error codes can be found in the iDRAC9 API specification. This documented list can be found at this [EEMI link](#). This content is online at the API developer's portal developers.dell.com.

NOTE: To get latest supported message/message IDs for your iDRAC version installed, run GET command:

```
Command: GET
iDRAC9 URI: redfish/v1/Registries/Messages/EEMIRegistry
iDRAC10 URI: redfish/v1/Registries/Messages/iDRACMessageRegistry
Header: content-type application/json
Auth: Basic or X auth
```

Success status code returned: 200

43.3 Redfish Service Error payload

The [error payload schema](#) from a Redfish Service.

44 Frequently Asked Questions (FAQ)

This section provides questions and suggestions for using Redfish to get you to your Redfish knowledge goals.

44.1 Strings the API is expecting for the ComputerSystem Reset action?

What are the strings the API is expecting for the ComputerSystem Reset action?

Run the GET command on URI `redfish/v1/Systems/System.Embedded.1?$select=Actions` will return action `ComputerSystem.Reset`. In the JSON output you will see property [ResetType@Redfish.AllowableValues](#) which will return supported possible values. This list is dynamic and will report only the values that iDRAC supports.

Also, DMTF specification will list other values for this action that iDRAC does not support like Pause and Resume.

```
@odata.context          "/redfish/v1/$metadata#ComputerSystem.ComputerSystem"
@odata.id               "/redfish/v1/Systems/System.Embedded.1"
@odata.type             "#ComputerSystem.v1_12_0.ComputerSystem"
Actions
#ComputerSystem.Reset
target                  "/redfish/v1/Systems/System.Embedded.1/Actions/ComputerSystem.Reset"
ResetType@Redfish.AllowableValues
0                       "On"
1                       "ForceOff"
2                       "ForceRestart"
3                       "GracefulRestart"
4                       "GracefulShutdown"
5                       "PushPowerButton"
6                       "Nmi"
7                       "PowerCycle"
```

Since `ResetType` is a string property, therefore the API guide does not list possible values, today. You will see the same behavior if you run GET on “`redfish/v1/Schemas/ComputerSystem_v1.xml`,” will not list possible string values for `ResetType` property.

44.2 Set BIOS attributes but no changes applied, no config job created

When setting BIOS attributes, ensure to pass in the JSON body `@Redfish.SettingsApplyTime` to create the config job along with setting pending values. If you do not pass in `SettingsApplyTime` only pending values will be set but no config job will get created.

Example of correct JSON body for PATCH operation:

```
{"@Redfish.SettingsApplyTime":{"ApplyTime":"OnReset"},  
"Attributes":{"MemTest":"Disabled"}}
```

44.3 Created a BIOS config job but no changes were applied

When setting BIOS attributes and a config job gets created, once you confirm that the job is marked scheduled, you must reboot the server for the job to be performed and applied.

NOTE: For setting BIOS attributes @Redfish.SettingsApplyTime does not support "Immediate" value.

44.4 BIOS config job created which is still in scheduled state, unable to set more BIOS attributes

While setting BIOS attributes if a config job is created, setting more BIOS attributes is not allowed. Once a job ID is created, the FQDD assigned to that job ID locks the session.

To configure more BIOS attributes, delete the current BIOS config job, set the new BIOS attributes along with previous attributes that were set in the deleted job and then create the config job.

44.5 What version of iDRAC supports configuring OEM attributes using PATCH?

This support is only available for iDRAC9. If you are using iDRAC 7 or iDRAC 8, you must leverage Server Configuration Profile (SCP) feature to set these attributes.

44.6 Details on YAML spec/schema?

Where can we find the YAML spec schemas?

It is available under: https://redfish.dmtf.org/redfish/schema_index

44.7 What type of methods are supported per URI?

To find out what type of methods are supported per URI, run GET on the URI and in the Header output you should see Allow property. This shows what methods are supported for this URI.

Examples:

```
>>> response = requests.get('https://192.168.0.120/redfish/v1/Managers',  
auth=("root", "calvin"), verify=False)  
>>> response.headers["Allow"]  
'GET, HEAD'
```

```
>>> response =
requests.get('https://192.168.0.120/redfish/v1/Managers/iDRAC.Embedded.1',
auth=("root", "calvin"), verify=False)
>>> response.headers["Allow"]
'GET, HEAD, PATCH'
```

44.8 What are the supported parameters/values for an action?

To find out supported parameters or values for an action, run the `GET` command on the URI service where it derives from or get more detailed information from running `GET` command on the schema for that action.

Following is the workflow example using the OEM action `DellLCService.SupportAssistCollection`:

1. Run the `GET` command on the URI

`redfish/v1/Managers/iDRAC.Embedded.1/Oem/Dell/DellLCService?$select=Actions`, look for action `DellLCService.SupportAssistCollection` which should show supported parameters and allowable values.



#DellLCService.SupportAssistCollection:	
▼ DataSelectorArrayIn@Redfish.AllowableValues:	
0:	"DebugLogs"
1:	"HwData"
2:	"OSAppData"
3:	"TTYLogs"
4:	"TelemetryReports"
▼ Filter@Redfish.AllowableValues:	
0:	"No"
1:	"Yes"
▼ IgnoreCertWarning@Redfish.AllowableValues:	
0:	"Off"
1:	"On"
▼ ProxySupport@Redfish.AllowableValues:	
0:	"DefaultProxy"
1:	"Off"
2:	"ParametersProxy"
▼ ProxyType@Redfish.AllowableValues:	
0:	"HTTP"
1:	"SOCKS"
▼ ShareType@Redfish.AllowableValues:	
0:	"CIFS"
1:	"FTP"
2:	"HTTP"
3:	"HTTPS"
4:	"Local"
5:	"NFS"
6:	"TFTP"
▼ Transmit@Redfish.AllowableValues:	
0:	"No"
1:	"Yes"
target:	"/redfish/v1/Managers/iDRAC.Embedded.1/Oem/Dell/DellLCService/Actions/DellLCService.SupportAssistCollection"

2. Run the `GET` command on schema URI `redfish/v1/Schemas/DellLCService_v1.xml` will return the detailed information for the action.

```

<Action Name="SupportAssistCollection" IsBound="true">
  <Annotation String="This method is used for triggering a SupportAssist collection and, optionally, send the collection to Dell or a network share." Term="OData.Description"/>
  <Annotation String="This method is used for triggering a SupportAssist collection and, optionally, send the collection to Dell or a network share." Term="OData.LongDescription"/>
  <Parameter Name="DellLCService" Type="DellLCService.v1_0_0.Actions"/>
  <Annotation Term="Redfish Revisions">
    <Collection>
      <Record>
        <PropertyValue Property="Kind" EnumMember="Redfish.RevisionKind/Added"/>
        <PropertyValue Property="Version" String="v1_1_0"/>
      </Record>
    </Collection>
  </Annotation>
  <Parameter Name="DataSelectorArrayIn" Type="Collection(DellLCService.DataSelectorArrayIn)">
    <Annotation String="This property allows to select the different type of logs in the collection. Default value for this parameter is HW Data and MUT logs are part of every collection." Term="OData.Description"/>
    <Annotation String="This property allows to select the different type of logs in the collection. Default value for this parameter is HW Data and MUT logs are part of every collection." Term="OData.LongDescription"/>
  </Parameter>
  <Parameter Name="Filter" Type="DellLCService.Filter">
    <Annotation String="This property specifies whether the collection should be filtered for PII (Personal Identification Information). Filter = 1 indicates that the collection is filtered for PII and excludes all debug logs and storage logs. When this parameter is not passed, default value is 0 (No filtering)." Term="OData.Description"/>
    <Annotation String="This property specifies whether the collection should be filtered for PII (Personal Identification Information). Filter = 1 indicates that the collection is filtered for PII and excludes all debug logs and storage logs. When this parameter is not passed, default value is 0 (No filtering)." Term="OData.LongDescription"/>
  </Parameter>
  <Parameter Name="IPAddress" Type="Edm.String">
    <Annotation String="The IP address of network share. This parameter is mandatory when ShareType is NFS or CIFS." Term="OData.Description"/>
    <Annotation String="The IP address of network share. This parameter is mandatory when ShareType is NFS or CIFS." Term="OData.LongDescription"/>
  </Parameter>
  <Parameter Name="IgnoreCertWarning" Type="DellLCService.IgnoreCertWarning">
    <Annotation String="Specifies if certificate warning should be ignored when HTTPS is specified. If IgnoreCertWarning is On, warnings are ignored. Default is 2 (On)." Term="OData.Description"/>
    <Annotation String="Specifies if certificate warning should be ignored when HTTPS is specified. If IgnoreCertWarning is On, warnings are ignored. Default is 2 (On)." Term="OData.LongDescription"/>
  </Parameter>
  <Parameter Name="Password" Type="Edm.String">
    <Annotation String="Password of the account to access the share. This parameter is mandatory if ShareType is CIFS." Term="OData.Description"/>
    <Annotation String="Password of the account to access the share. This parameter is mandatory if ShareType is CIFS." Term="OData.LongDescription"/>
  </Parameter>
  <Parameter Name="Port" Type="Edm.Int64"></Parameter>
  <Parameter Name="ProxyPasswd" Type="Edm.String">
    <Annotation String="The password to login into proxy server." Term="OData.Description"/>
    <Annotation String="The password to login into proxy server." Term="OData.LongDescription"/>
  </Parameter>
  <Parameter Name="ProxyPort" Type="Edm.String">
    <Annotation String="Port for the proxy server. Default is set to 80." Term="OData.Description"/>
    <Annotation String="Port for the proxy server. Default is set to 80." Term="OData.LongDescription"/>
  </Parameter>
  <Parameter Name="ProxyServer" Type="Edm.String">
    <Annotation String="The IP address of the proxy server." Term="OData.Description"/>
    <Annotation String="The IP address of the proxy server." Term="OData.LongDescription"/>
  </Parameter>
  <Parameter Name="ProxySupport" Type="DellLCService.ProxySupport">
    <Annotation String="Specifies if proxy should be used. Default is 1 (Off)." Term="OData.Description"/>
    <Annotation String="Specifies if proxy should be used. Default is 1 (Off)." Term="OData.LongDescription"/>
  </Parameter>

```

Figure 6 Schema example details

44.9 What are the supported parameters for creating a RAID volume using POST?

Supported parameters for creating a RAID volume are:

- DisplayName (custom name for the VD, if not passed in default name will be assigned by the controller)
- RAIDType (RAID level of the volume)
- Drives (array list of drives)
- CapacityBytes (size of the RAID volume, if not passed in full disk size will be used)
- OptimumIOSizeBytes (stripe size of the volume, if not passed in default size will be assigned by the controller)
- DiskCachePolicy (OEM property)
- ReadCachePolicy

- WriteCachePolicy
- Encrypted (secure a volume, controller encryption must be enabled first)
- MediaSpanCount (must have iDRAC9 6.10.00 or later to use this property)

44.10 Operation not starting, Lifecycle Controller in use error message returned

If an operation is not starting or you see a command or job message stating **Lifecycle Controller in use**, make sure the server is either off, out of POST, out of F2 System Setup or in the OS. To see the server and Lifecycle Controller (LC) status, see the [Verify Server Status/Lifecycle Controller Status/Real Time Storage Monitor Status](#) section.

44.11 All supported controller attributes are not listed under URI “redfish/v1/Systems/System.Embedded.1/Storage/{controller FQDD}/Controllers/{controller FQDD}?\$select=Oem/Dell/DellStorageController”

URI `redfish/v1/Systems/System.Embedded.1/Storage/{controller FQDD}/Controllers/{controller FQDD}?$select=Oem/Dell/DellStorageController` does not list controller attributes `CheckConsistencyRate` and `RebuildRate`, but it lists all the other supported rate attributes. `CheckConsistencyRate` and `RebuildRate` attributes are listed under the URI `redfish/v1/Systems/System.Embedded.1/Storage/{controller FQDD}`.

44.12 redfish/v1/TaskService/Tasks/<task ID> is getting auto deleted after a certain period of time

This is expected behavior and leverages `TaskAutoDeleteTimeoutMinutes` property setting under **TaskService** schema. But you can modify the timeout setting by running the `PATCH` command to either decrease or increase when the Task URI gets auto deleted.

NOTE: Even though the task URI gets auto deleted, it does not get deleted from iDRAC job queue. You can still query the job ID using `redfish/v1/JobService/Jobs/<job ID>` URI.

44.13 Configuration or Update job stuck in running state, takes 2 hours to get marked as failed due to timeout

Intermittently a configuration or update job may get stuck in running state due to unknown iDRAC issue. When a job is stuck in running state you are unable to create any new configuration or update jobs using iDRAC interfaces. By default, the timeout to mark this job as failed is 2 hours. You can modify this timeout value by running the `PATCH` command on attribute `Job.1.JobTimeout`. This attribute is located under URI `redfish/v1/Managers/iDRAC.Embedded.1/Oem/Dell/DellAttributes/System.Embedded.1`.

A Technical support and resources

Dell Technologies iDRAC Redfish API portal main page. Once you click on the link below select your iDRAC version from the version dropdown box.

<https://developer.dell.com/apis/2978/versions/7.xx/docs/0WhatsNew.md>

Whitepapers on [Redfish Events \(Legacy\)](#) and [Redfish Lifecycle Events](#) (RLCE in iDRAC9 version 5.xx series)
Redfish Events

Redfish User Guide: https://www.dmtf.org/sites/default/files/standards/documents/DSP2060_1.0.0.pdf

iDRAC RESTful API with Redfish automation tools -

- Dell Open source: [Redfish PowerShell and Python Scripting on GitHub](#)
- <https://www.powershellgallery.com/packages?q=idrac>
- [Python “requests” library](#)
- [OpenManage Ansible Modules](#)
- [Advanced REST Client for Google Chrome](#)
- [Postman REST Client for Google Chrome](#)
- [cURL Command Line Interface](#)
- iDRAC Redfish mockup clients and setup documentation: [iDRAC Redfish mockup clients](#)

Redfish: It is What's for Dinner; How Does it Work and First Steps: [Customer benefits video](#)

DMTF materials on Redfish - these documents and videos explain the basics of the Redfish standard.

- [Introduction to Redfish presentation](#)
 - [DMTF Redfish specification](#)
 - Online seminar – [Redfish overview](#):
 - YouTube [Redfish School](#):
- iDRAC OEM schema for developer reference; <https://downloads.dell.com/redfish/bmc/schemas/>

The iDRAC support home page provides access to product documents, technical white papers, how-to videos, and more. www.dell.com/support/idrac

iDRAC User Guides and other manuals: www.dell.com/idracmanuals

Select iDRAC9, then the firmware version. Under “Documentation”

- RACADM CLI Guide
- Attribute Registry

- Both guides provide the list of sub commands, supported RACADM interfaces, and property database groups and object definitions.
- Also, on this page are the iDRAC User's Guide, along with Release Notes and other helpful documents.

iDRAC Service Module (iSM)

Start at www.dell.com/idracmanuals and select "iDRAC Service Module." The iSM User Guide has details on how to install, iSM features, supported Operating Systems, and more.

Lifecycle controller Best Practices Guide https://dl.dell.com/manuals/common/lc-best_practices_guide.pdf

WS-Man to Redfish API whitepaper provides a [mapping between WS-Man to Redfish](#)

Dell Technical Support: www.Dell.com/support